

**IAR ASSEMBLER,
LINKER, & LIBRARIAN
FOR THE Z80/64180**

COPYRIGHT NOTICE

© Copyright 1991-1997 IAR Systems. All rights reserved.

No part of this document may be reproduced without the prior written consent of IAR Systems. The software described in this document is furnished under a license and may only be used or copied in accordance with the terms of such a license.

DISCLAIMER

The information in this document is subject to change without notice and does not represent a commitment on any part of IAR Systems. While the information contained herein is assumed to be accurate, IAR Systems assumes no responsibility for any errors or omissions.

In no event shall IAR Systems, its employees, its contractors, or the authors of this document be liable for special, direct, indirect, or consequential damage, losses, costs, charges, claims, demands, claim for lost profits, fees, or expenses of any nature or kind.

TRADEMARKS

C-SPY is a trademark of IAR Systems.

MS-DOS is a trademark of Microsoft Corp.

All other product names are trademarks or registered trademarks of their respective owners.

Second edition: March 1997

Part number: AZ80-2

ABOUT THIS GUIDE

This guide describes how to install and use the IAR Systems macro assembler for the Zilog Z80, Z80180 series, Z64180, and Hitachi 64180 microprocessors.

The guide is divided into two parts: the first part, *IAR Z80 Assembler*, describes the assembler, and the second part, *IAR Linker & Librarian*, describes the XLINK Linker and XLIB Librarian.

IAR Z80/64180 ASSEMBLER

This part consists of the following chapters:

The *Introduction* describes the general features of the IAR Assemblers, and provides a brief summary of the specific features provided for the Z80/64180. It also describes the IAR XLINK Linker and XLIB Librarian and their use in a typical configuration.

Getting started describes how to install the assembler, XLINK, and XLIB.

The *Tutorial* shows how to use the most important features of the assembler to develop simple Z80/64180 machine-code programs. It also describes a typical development cycle using XLINK and XLIB.

Command line options summary gives a summary of the assembler command line options.

Command line options gives complete reference information about each command line option.

Assembler source format describes the source format for the Z80 Assembler, and the format of assembler listings.

Assembler operator summary gives a summary of the assembler operators, arranged in order of precedence.

Assembler operator reference then gives a complete alphabetical list of operators, with a full description of each one.

ABOUT THIS GUIDE

Assembler directives reference gives complete reference information about the IAR Assembler directives, classified into groups according to their function.

Assembler mnemonics lists the Z80 and 64180 instruction mnemonics, with details of the addressing modes that can be used with each one.

Finally, *Assembler diagnostics* provides a list of error messages specific to the Z80 Assembler.

IAR LINKER AND LIBRARIAN

The *Introduction* describes the general features of XLINK and XLIB and their use in a typical configuration.

XLINK Linker then introduces the XLINK Linker, and describes the XLINK listing format.

XLINK environment variables describes the XLINK environment variables.

XLINK command summary gives a summary of the XLINK commands.

XLINK command reference then gives complete reference information about each XLINK command.

XLINK output formats summarizes the output formats available from XLINK.

XLIB Librarian introduces the XLIB Librarian, and describes how to run it in interactive mode or batch mode.

XLIB environment variables lists the XLIB environment variables.

XLIB command summary gives a summary of the XLIB commands, and lists the XLINK environment variables.

XLIB command reference then gives complete reference information about each XLIB command.

Finally *XLINK and XLIB diagnostics* describes the error and warning messages produced by XLINK and XLIB, together with explanations and suggested courses of action in each case.

ASSUMPTIONS

This guide assumes that you already have a working knowledge of the following:

- ◆ The target processor you are using with the IAR Assembler.
- ◆ The target processor assembler language.
- ◆ MS-DOS or UNIX, depending on your host system.

CONVENTIONS

This user guide uses the following typographical conventions:

<i>Style</i>	<i>Used for</i>
<code>computer</code>	Text that you type in, or that appears on the screen.
<code>parameter</code>	What you should type as part of a command.
<code>[option]</code>	An optional part of a command.
<code>{a b c}</code>	Alternatives in a command.
<code>reference</code>	A cross reference to another part of this user guide, or to another guide.

ABOUT THIS GUIDE

CONTENTS

IAR Z80/64180 ASSEMBLER

Introduction	1-1
Key features	1-1
General characteristics	1-2
Overview	1-3
System requirements	1-4
Getting started	1-5
Installation	1-5
Running the assembler	1-8
Tutorial	1-11
Introduction	1-11
Using macros	1-15
Using structured assembly	1-19
Using modules	1-23
Command line options summary	1-29
Command line options	1-31
Assembler source format	1-43
Source format	1-43
Expressions and operators	1-44
Listing format	1-49
Output formats	1-51
Assembler operator summary	1-53
Assembler operator reference	1-57
Assembler directives reference	1-77
Syntax conventions	1-78
Module control directives	1-80
Symbol control directives	1-83
Segment control directives	1-85
Value assignment directives	1-90

CONTENTS

Conditional assembly directives	1-93
Macro processing directives	1-96
Structured assembly directives	1-104
Listing control directives	1-113
C preprocessor directives	1-123
Data definition or allocation directives	1-127
Assembler control directives	1-130
Assembler mnemonics	1-135
Addressing modes	1-135
Instruction mnemonics	1-138
Additional 64180 instructions	1-144
Assembler diagnostics	1-145
Introduction	1-145
Warning messages	1-147
Error messages	1-150

IAR LINKER AND LIBRARIAN

Introduction	2-1
XLINK Linker	2-1
XLIB Librarian	2-2
Overview	2-3
System requirements	2-4
XLINK Linker	2-5
Introduction	2-5
Using XLINK	2-7
Input files and modules	2-9
Listing format	2-12
XLINK environment variables	2-17
XLINK command summary	2-23
XLINK command reference	2-25
XLINK output formats	2-61

CONTENTS

XLIB Librarian	2-65
Introduction	2-65
Running XLIB	2-67
XLIB environment variables	2-69
XLIB command summary	2-71
XLIB command reference	2-73
XLINK and XLIB diagnostics	2-91
XLINK messages	2-91
XLIB messages	2-105
INDEX	I

CONTENTS

IAR Z80/64180 ASSEMBLER

INTRODUCTION

The IAR Assemblers are a family of powerful relocating macro assemblers, supporting a range of different microprocessors.

The IAR Assemblers all have a common user interface as well as standardized relocation and macro facilities, making it easy to work with different target processors.

Despite providing a standardized user interface, the IAR Assemblers also incorporate a high degree of compatibility with the microprocessor manufacturer's own assembler, to ensure that software originally developed using them can be transferred to the IAR Assembler with little or no modification.

KEY FEATURES

The IAR Assemblers offer the following features:

USER INTERFACE

- ◆ One pass assembly, for faster execution.
- ◆ Integrates with XLINK Linker and XLIB Librarian.
- ◆ Integrates with all other IAR software.
- ◆ Self explanatory error messages.

ASSEMBLER FEATURES

- ◆ Supports structured control directives.
- ◆ Up to 255 relocatable segments per module.
- ◆ 32 bit arithmetic and IEEE floating point constants.
- ◆ 255 significant characters in symbols.
- ◆ Powerful recursive macro facilities.

INTRODUCTION

- ◆ Number of symbols and program size limited only by available memory.
- ◆ Complex expressions with external references supported.
- ◆ Forward references allowed to any depth.
- ◆ C language pre-processor directives and the `sfr` keyword supported.
- ◆ Macros in Motorola/Intel style.

GENERAL CHARACTERISTICS

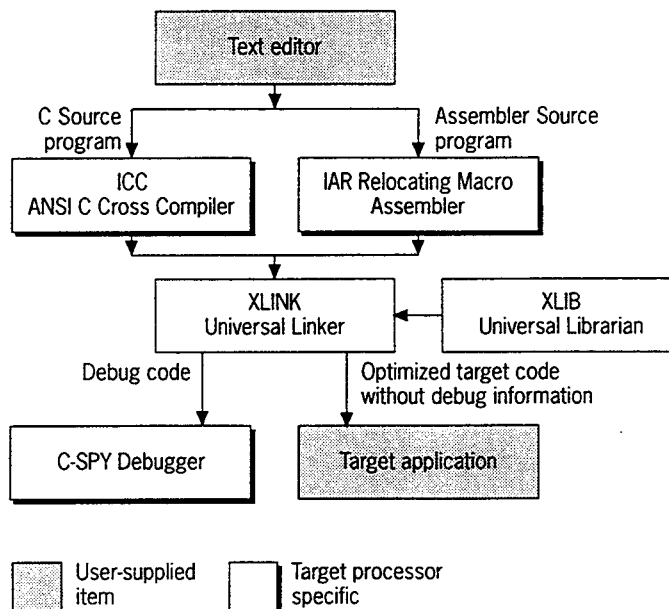
Irrespective of the target processor supported, the IAR Assemblers share a common user interface and general characteristics. They are all of the single pass type, executing as a single program, and no temporary files are created during execution.

The assemblers include directives for structured flow control, or structured assembly, and a pre-processor for the C language.

Despite these common characteristics, the IAR Assemblers also provide a high level of compatibility with each manufacturer's assembler. Where it has not been possible to retain complete compatibility, the differences are described in this guide.

OVERVIEW

The following diagram shows how the IAR Assembler is used in conjunction with the XLINK Linker, XLIB Librarian, and C-SPY debugger:



For information about the XLINK Linker and XLIB Librarian see the guide *IAR Linker & Librarian*.

For information about C-SPY see the guide *Using C-SPY*.

INTRODUCTION

SYSTEM REQUIREMENTS

The IAR Assembler, XLINK Linker, and XLIB Librarian are available in versions for two different host computer systems: IBM PC compatibles under MS-DOS or UNIX-based workstations. The versions supplied for the IBM PC provide default support for up to 16 Mbyte of extended memory.

IBM PC

Operating system:	Minimum MS-DOS 4.01.
Memory:	1 Mbyte minimum. Extended memory is supported up to 16 Mbyte.

UNIX

Operating system:	No special requirements.
-------------------	--------------------------

GETTING STARTED

This chapter describes how to install the assembler, XLINK, and XLIB from the installation disk, and how to run the assembler.

INSTALLATION

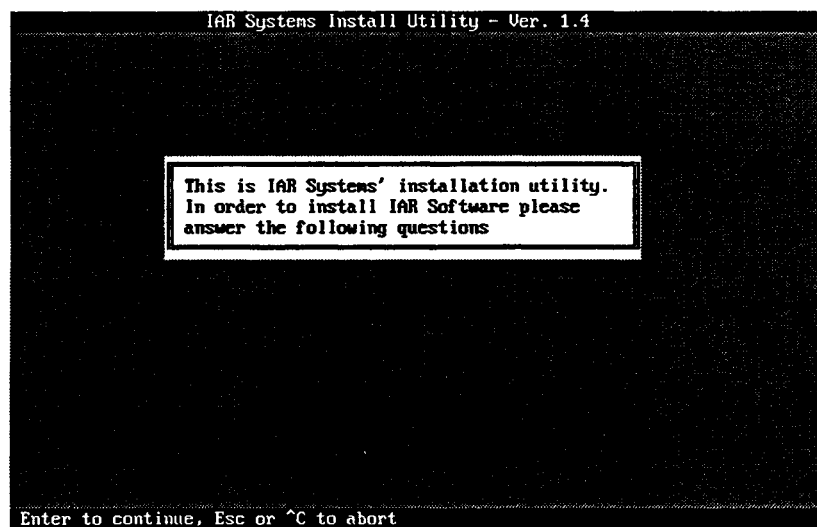
This section describes how to install the assembler.

INSTALLATION UNDER MS-DOS

- ◆ Insert the installation disk into the floppy disk drive and type:

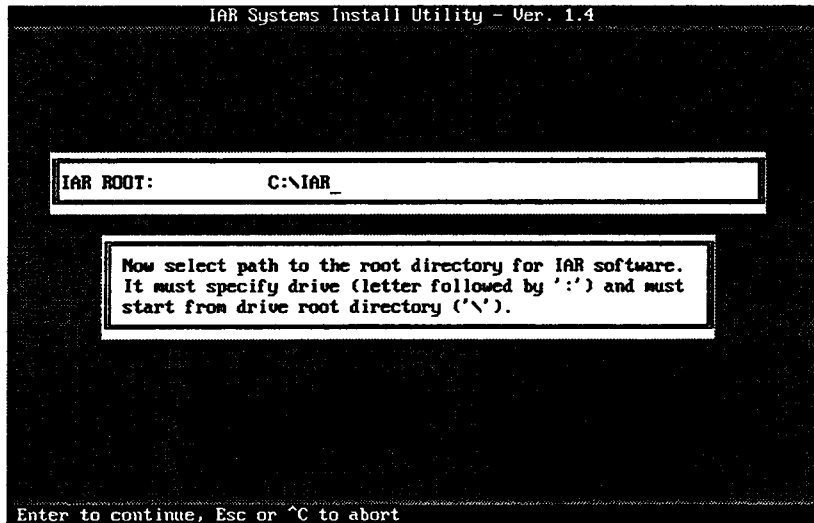
a:\install ☐

The startup screen is displayed:



- ◆ Press ☐. You will then be prompted to enter the path for installing the IAR files:

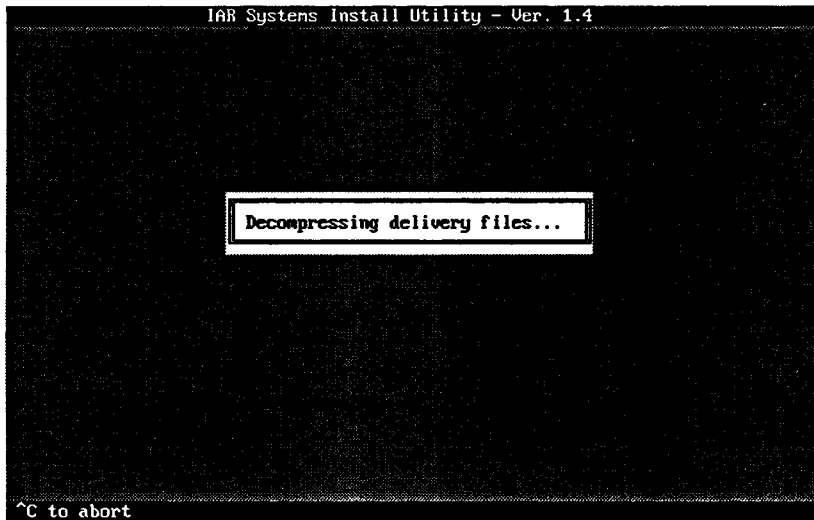
GETTING STARTED



By default the files are installed in c:\iar.

◆ Edit the path, or press to use the default.

The files are then decompressed:



GETTING STARTED

The following screen then shows the default paths for each subdirectory into which the files will be installed:

IAR Systems Install Utility - Ver. 1.4

Select target directories for different file types. Use arrows or TAB to choose path to edit. Press ENTER after accepting all paths.

FILE TYPE	DIRECTORY TO INSTALL
EXECUTABLE:	C:\IAR\EXE
ETC:	C:\IAR\ETC
ASSEMBLER:	C:\IAR\A280

This directory contains executable files

Esc or ^C to abort

Note that not all the options will be present if you have only purchased the IAR Assembler.

You may edit any of the paths to suit your requirements. You will not normally need to do this, and this guide assumes you have chosen the defaults.

- ◆ Press to proceed.
- ◆ Confirm with Y to the prompts:

Ready to install (Y/N)?

Confirm overwriting files (Y/N)?

The files will then be installed in the directories you specified.

GETTING STARTED

The following changes should be made to the `autoexec.bat` file to make the IAR files accessible from any directory in your system:

- ◆ Modify the path command by adding:

```
c:\iar\exe
```

or whatever other path you specified for executable files.

- ◆ Define an environment variable for the IAR XLINK Linker as follows:

```
set XLINK_DFLTDIR=c:\iar\lib\
```

Again, if you changed the default path use the path you specified.

INSTALLATION UNDER UNIX

Follow the separate printed installation documentation supplied with the delivery media.

RUNNING THE ASSEMBLER

The assembler is run with the following command:

```
az80 [options] sourcefile [options]
```

PARAMETERS

sourcefile The source file with the default extension `.msa` or `.s01`.

options Command line options; see *Command line options summary*, page 1-29.

FILE NAMING CONVENTIONS

The Z80 Assembler and XLINK Linker use the following naming conventions for the various files they process:

<i>Extension</i>	<i>Description</i>
.s01	Source files read by the assembler (default); this is also the default file type for include source files.
.msa	Optional source file default file type.
.inc	Optional file type for include source files.
.r01	Relocatable output files created by the assembler.
.lst	Listing files created by the assembler (default).
.a01	Absolute, linked code created by XLINK.
.d01	Absolute, linked code in the IAR debug format.
.xcl	XLINK command files (default).

When using the Z80 Assembler from within a batch file, you may need to determine whether the assembly was successful in order to decide what step to take next. For this reason the assembler returns the following error return codes:

<i>Return codes</i>	<i>Meaning</i>
0	Assembly successful.
1	There were warnings; returns 0 if -w (disable warning messages) assembler option is used.
2	There were errors.
3	Fatal error detected (assembly aborted).

These codes can be used in conjunction with the make facility.

GETTING STARTED

EXTENDING THE COMMAND LINE

The environment variable ASMZ80 can be used for extending the command line or setting user-defined defaults. For example:

```
az80 -v0 -l foo.lst foo
```

This assembles the source file `foo.s01`, or `foo.msa`, for the Z80 processor. The object code is listed in the file `foo.lst`, and the object file is `foo.r01`.

```
az80 -v1 -cC -o temp.r01 fie.s01 > fie.out
```

This assembles the source file `fie.s01` for the 64180 processor. The object file is `temp.r01`. The screen output is saved in the file `fie.out`, which is a useful procedure when assembling large files that may initially contain many errors.

By adding the following line:

```
set ASMZ80=-l temp.lst
```

to your `autoexec.bat` file, the option `-l temp.lst` will always be automatically added as the first option of the command line. Thus, typing:

```
az80 foo.s01
```

will have the effect of the command line:

```
az80 -l tmp.lst foo.s01
```

TUTORIAL

This tutorial illustrates how you might use the Z80 Assembler to develop a series of simple machine code programs, and it illustrates some of the assembler's most important features.

Before reading this chapter you should:

- ◆ Have installed the assembler software; see *Getting started*, page 1-5.
- ◆ Be familiar with the architecture and instruction set of the Z80 or 64180 processor. For more information see the chapter *Assembler mnemonics*, page 1-135, and the manufacturer's data book.

This tutorial assumes that you have the optional C-SPY simulator, and explains how to run the example programs using C-SPY.

Alternatively, you can run the examples by linking them without the -r option to give a file about .a01, which can be downloaded to an emulator or PROM programmer. Use the XLINK -F option to specify a format other than the default INTEL-STD.

INTRODUCTION

The first example program is a simple count loop to illustrate how you write an assembler program, and how you then assemble, link, and run it. Enter the following using any standard text editor (such as the MS-DOS EDIT editor). Alternatively, a copy is provided in the c:\iar\az80 directory with the name first.s01:

	NAME	first
	ORG	0
main	LD	BC,0
loop	LD	A,C
	INC	A
	DAA	
	LD	C,A

TUTORIAL

```
LD      A,B
ADC     A,0
DAA
LD      B,A
JR      NC,loop
RET
```

```
END
```

This simple program is a loop, which counts up the B and C registers in binary-coded decimal.

The ORG directive assembles the program starting at address 0, so it is executed on reset.

Save the source program in a file `first.s01`. The files associated with the Z80 Assembler have extensions `.s01`, `.a01`, `.d01`, and `.r01` to identify them. You now have a source file which is ready to assemble.

RUNNING THE ASSEMBLER

To run the assembler, type the following command at the MS-DOS prompt:

```
az80 first -l first
```

This will send a listing to the file `first.lst`. If you look at this file you will see that it contains the following:

TUTORIAL

```
#####  
#                                                                 #  
#IAR Systems Z80/64180 Assembler X3.00A/DOS [Apr 14 1994] 16/Aug/94 13:56:44#  
#                                                                 #  
#           Target option - z80                                   #  
#           Source file   - first.s01                           #  
#           List file     - first.lst                             #  
#           Object file   - first.r01                             #  
#           Command line  - first -l first                       #  
#                                                                 #  
#                                                                 (c) Copyright IAR Systems 1993 #  
#####  
  
      1      000000                NAME      first  
      2      000000  
      3      000000                ORG        0  
      4      000000 010000      main  LD      BC,0  
      5      000003 79          loop  LD      A,C  
      6      000004 3C                INC      A  
      7      000005 27                DAA  
      8      000006 4F                LD      C,A  
      9      000007 78                LD      A,B  
     10      000008 CE00             ADC      A,0  
     11      00000A 27                DAA  
     12      00000B 47                LD      B,A  
     13      00000C 30F5             JR      NC,loop  
     14      00000E C9                RET  
     15      00000F  
     16      00000F                END  
#####  
#           CRC:31FC      #  
#           Errors:   0    #  
#           Warnings: 0    #  
#####
```

This shows the machine code instructions generated by each of the source code statements. Note that the CRC number depends on the date of assembly, and may vary.

TUTORIAL

The format of the listing is as follows:

5	000003	79	loop	LD	A,C
6	000004	3C		INC	A
7	000005	27		DAA	
8	000006	4F		LD	C,A

		Address field		Source line	
Source line number		Data field			

Assuming that the source assembled successfully, a further file, `first.r01`, will also be created, containing the linkable object code.

If you made any errors when entering the program, these will be displayed on the screen during the assembly. If this happens, return to the editor, check carefully through the source code to locate and correct all the mistakes, resave the source file using the same name, and try assembling it again.

LINKING THE OBJECT FILE

To link the object file to produce code that can be executed by the C-SPY debugger, give the command:

```
xlink first -cz80 -r -DCBAR=0 -DCBR=0 
```

The `-c` option specifies the target processor, and `-r` specifies that debugging information should be included for use by C-SPY.

The `-D` options define symbols needed by C-SPY.

In this simple example there is only one segment, the default absolute segment, which will be located in memory starting at address 0.

By default, the output code will be placed in a file `out.d01`.

RUNNING THE PROGRAM

To run the example program using the C-SPY debugger give the command:

cs64180 aout 

In C-SPY give the command:

WINDOW REG ON 

and then type STEP or press  to step through the program and watch the B and C registers count in binary-coded decimal.

USING MACROS

The second sample program will be used to demonstrate the use of a simple macro. It defines a recursive routine to calculate binomial coefficients.

The binomial coefficient nC_r gives the number of different combinations of n things taken r at a time. For example, the number of combinations of the letters X, Y, and Z taken 2 at a time is 3C_2 or 3; namely XY, YZ, and XZ.

The function can be defined recursively as follows:

$$\begin{aligned} {}^nC_r &= 1 \text{ if } r=0 \\ &= 1 \text{ if } n=r \\ &= {}^{n-1}C_r + {}^{n-1}C_{r-1} \text{ otherwise.} \end{aligned}$$

The ncr subroutine uses the stack to store intermediate results when the routine is re-entered recursively. It is called with the B and C registers containing the values of n and r respectively, and the result is returned in register HL.

The program defines the following simple macro to test whether the A register is zero:

```
; Test A register
test    MACRO
        OR      A
        ENDM
```

TUTORIAL

The full listing of the ncr assembler program is as follows:

```
; Test A register
test    MACRO
        OR      A
        ENDM

main    ORG      0
        LD       SP,2000
        LD       BC,703H ; Test 3C7
        CALL    ncr
        RET

;
; Recursive routine to calculate
; binomial coefficients
;
ncr     LD       HL,1
        LD       A,C
        test
        RET     Z
        CP      B          ; B=C?
        RET     Z
;
none    DEC      B
        PUSH    BC          ; Save B,C
        CALL   ncr
        POP     BC          ; Restore B,C
        PUSH    HL          ; Save result
        DEC     C
        CALL   ncr
        POP     BC          ; Get result in BC
        ADD     HL,BC
        RET

END
```

The main program, `main`, sets up the stack and then calls the `ncr` subroutine with $B = 7$ and $C = 3$ to calculate 7C_3 .

Type in this listing and save it in a file ncr.s01. Alternatively, a copy of the source is provided in the c:\iar\az80 directory.

ASSEMBLE THE PROGRAM

To assemble the source program with default options give the command:

```
az80 ncr -l ncr
```

The following output will be produced in the file ncr.lst. In this and subsequent listings the header information is omitted for clarity:

```
1  000000          ; Test A register
5  000000
6  000000          ORG      0
7  000000 31D007  main  LD      SP,2000
8  000003 010307          LD      BC,703H ; Test 3C7
9  000006 CD0A00          CALL    ncr
10 000009 C9           RET
11 00000A          ;
12 00000A          ; Recursive routine to calculate
13 00000A          ; binomial coefficients
14 00000A          ;
15 00000A 210100  ncr   LD      HL,1
16 00000D 79           LD      A,C
17 00000E           test
17.1 00000E B7         OR      A
17.2 00000F           ENDM
18 00000F C8           RET     Z
19 000010 B8           CP      B      ; B-C?
20 000011 C8           RET     Z
21 000012          ;
22 000012 05          none  DEC     B
23 000013 C5           PUSH    BC      ; Save B,C
24 000014 CD0A00       CALL    ncr
25 000017 C1           POP     BC      ; Restore B,C
26 000018 E5           PUSH    HL      ; Save result
27 000019 0D           DEC     C
28 00001A CD0A00       CALL    ncr
29 00001D C1           POP     BC      ; Get result in BC
```

TUTORIAL

```
30  00001E 09      ADD    HL,BC
31  00001F C9      RET
32  000020
33  000020      END
```

The macro-generated lines are numbered with a decimal suffix; eg 17.1, 17.2, etc.

LINKING THE PROGRAM

In order to be able to execute the program, the relocatable file produced by the assembler needs to be converted to an object code program with all the addresses resolved. This is done using XLINK.

Run XLINK to produce code for debugging with the command:

```
xlink ncr -cz80 -r -l ncr.map -DCBAR=0 -DCBR=0 ↵
```

This generates a file aout.d01.

RUNNING THE PROGRAM

If you have the C-SPY simulator you can run the program with the command:

```
cs64180 aout ↵
```

Set a breakpoint at the RET instruction at the end of the main program by moving the source cursor to the RET line with **[Ctrl]N**, and then typing:

```
BREAK SET ↵
```

Display the registers by typing:

```
WINDOW REG ON ↵
```

Then execute from main by typing:

```
GO ↵
```

The result will be returned in register HL, and should be 23 (hexadecimal).

USING STRUCTURED ASSEMBLY

This example demonstrates how to take full advantage of the IAR Assembler's powerful structured assembly facilities.

A simple routine is first written in conventional assembler instructions. The example then shows how it can be rewritten using structured assembly directives to reduce the number of source lines, make the program logic clearer, and reduce the possibility of programmer error.

WRITE THE PROGRAM

The following program implements Euclid's algorithm for finding the greatest common divisor of two numbers in the A and B registers:

```
/* Euclid's Algorithm */
/* Numbers in A and B */
/* Result in B          */

        NAME    gcd
        ORG     1000H

begin    CP      B
        JR      NC,noswap
        LD      C,A
        LD      A,B
        LD      B,C
noswap    SUB     B
        CP      0
        JR      NZ,begin
        RET

        END     begin
```

Type the program into a file gcd1.s01. Alternatively, a copy is supplied on the installation disk.

TUTORIAL

ASSEMBLE THE PROGRAM

Assemble the gcd1 program with the following command:

az80 gcd1 -l gcd1 

The listing will be sent to the file gcd1.lst. This is as follows:

```
1  001000          /* Euclid's Algorithm */
2  001000          /* Numbers in A and B */
3  001000          /* Result in B          */
4  001000
5  001000          NAME    gcd
6  001000          ORG     1000H
7  001000
8  001000 B8      begin  CP      B
9  001001 3003      JR      NC,noswap
10 001003 4F      LD      C,A
11 001004 78      LD      A,B
12 001005 41      LD      B,C
13 001006 90      noswap SUB    B
14 001007 FE00      CP      0
15 001009 20F5      JR      NZ,begin
16 00100B C9      RET
17 00100C
18 00100C          END     begin
```

USING STRUCTURES

The IAR Assembler provides the following constructs for structured assembly:

<i>Construct</i>	<i>Description</i>
IF ... ELSEIF ... ELSE ... ENDIF	Specifies instructions to be executed if a condition is true.
WHILE ... ENDW	Executes a loop as long as an expression is true.
REPEAT ... UNTIL	Executes a loop until an expression is true.
FOR ... ENDF	Repeats subsequent instructions a specified number of times.
SWITCH ... CASE ... DEFAULT ... ENDS	Multiple case switch.

The gcd program can be rewritten to use the REPEAT ... UNTIL loop, and an IF ... ENDIF test, as follows:

```
/* Euclid's Algorithm */  
/* Numbers in A and B */  
/* Result in B          */
```

```
NAME    gcd  
ORG     1000H  
  
begin   REPEAT  
        IF      A < B  
        LD      C,A  
        LD      A,B  
        LD      B,C  
        ENDIF  
        SUB     B  
        UNTIL   A = 0  
        RET  
  
        END     begin
```

TUTORIAL

Save this in a file gcd2.s01.

Assemble this with the command:

az80 gcd2 -l gcd2 

This produces the following code:

```
1  001000      /* Euclid's Algorithm */
2  001000      /* Numbers in A and B */
3  001000      /* Result in B      */
4  001000
5  001000          NAME    gcd
6  001000          ORG     1000H
7  001000
8  001000      begin  REPEAT
8.1 001000      _?0
9  001000          IF     A < B
9.1 001000 B8      CP     B
9.2 001001 F20A10   JP     P,_?2
9.3 001004 CA0A10   JP     Z,_?2
10 001007 4F      LD     C,A
11 001008 78      LD     A,B
12 001009 41      LD     B,C
13 00100A          ENDIF
13.1 00100A      _?2
14 00100A 90      SUB     B
15 00100B          UNTIL  A = 0
15.1 00100B FE00   CP     0
15.2 00100D C20010  JP     NZ,_?0
15.3 001010      _?1
16 001010 C9      RET
17 001011
18 001011          END    begin
```

The code generated by the structured assembly directives is listed on lines with a decimal suffix: 9.1, 9.2, etc. The assembler generates extra labels _?0, _?1, etc to implement the generated branch instructions.

The main REPEAT ... UNTIL loop produces similar code for the loop as in the original example, but in a much more convenient and easy to read manner.

Similarly, the test in the middle of the body of the loop has been replaced with an IF ... ENDIF structure. This tests register A against register B. The < test results in a CP instruction followed by JP instructions around the body of the section (which swaps the registers A and B).

USING MODULES

The final example demonstrates how to create library modules and use the XLIB Librarian to maintain files of modules.

CREATE THE MAIN PROGRAM

First create the main program which simply uses a routine called `rightshift` to shift the contents of the BC register to the right. The data in register BC is set to `$ABCD` and the `rightshift` routine is called to shift it to the right by four places, as specified by the contents of the A register.

```
NAME      main

EXTERN    rightshift

main      RSEG    PROM
          LD      BC,H'ABCD
          LD      A,4
          CALL    rightshift
          RET

END       main
```

The EXTERN directive declares `rightshift` as an external symbol, to be resolved at link time.

Save this in a file `main.s01`.

TUTORIAL

CREATE THE LIBRARY ROUTINES

The second program is used to form a separately assembled library. This contains two library routines: the `rightshift` routine called by `main`, and a corresponding `leftshift` routine. These both operate on the contents of register `BC` by repeatedly shifting it to the right or left. The number of shifts performed is controlled by decrementing register `A` to zero.

It would be possible to simply assemble both source files together but if this were a large project such a method would soon become very wasteful of time. By assembling the library routines separately, changes to the main program only require reassembly of the main source file.

```
MODULE rightshift
PUBLIC rightshift
RSEG PROM

rightshift
WHILE A > 0
SRL B
RR C
DEC A
ENDW
RET
ENDMOD
```

```
MODULE leftshift
PUBLIC leftshift
RSEG PROM

leftshift
WHILE A > 0
SLA C
RL B
DEC A
ENDW
RET

END
```

The routines are defined as library modules by the `MODULE` directives; these instruct the XLINK Linker to include them only if they are called by another module.

The `rightshift` and `leftshift` entry addresses are made public to other modules with a `PUBLIC` directive.

Save these modules in a file `shifts.s01`.

ASSEMBLE AND LINK THE SOURCE FILES

Next you need to assemble both of the above source files.

To assemble the main program type:

```
az80 main -l main 
```

Similarly, to assemble the library routines type:

```
az80 shifts -l shifts 
```

Assembling the files creates two relocatable files. You need to link these together to produce a single executable object file containing the main program and the library routine it references, with all of the cross references resolved. In this case the only reference from one section to the other is the call of the `rightshift` subroutine. The `leftshift` routine is not used at all.

To link the files in a single step use the following:

```
xlink -cz80 main shifts -ZPROM=2000 -xsm -l main.map 
```

The following table explains the options which define the addresses for the code and data segments:

<i>Parameter</i>	<i>Description</i>
<code>-ZPROM=2000</code>	Defines that the code segment is to be relocated to the hex address 2000.
<code>-xsm</code>	Requests a cross reference listing.
<code>-l main.map</code>	Directs the listing output to <code>main.map</code> .

TUTORIAL

For more information about the XLINK commands see *XLINK command reference* in the *IAR Linker & Librarian* guide.

If you list the cross reference listing, `main.map`, you will see that the module created by XLINK includes the `main` program module and the `rightshift` library module, but not the unused `leftshift` library module.

USING LIBRARIES

If you are working on a large project you will soon accumulate a collection of useful routines that are used by several of your programs.

To avoid the need to assemble a routine each time you need it you can store such routines as object files; ie assembled but not linked.

A collection of routines in a single object file is referred to as a library. It is recommended that you use library files to create collections of related routines, such as graphical or maths libraries.

You can use the XLIB Librarian to manipulate libraries; it allows you to:

- ◆ Change modules from PROGRAM to LIBRARY type, and vice versa.
- ◆ Add or remove modules from a library file.
- ◆ Change the names of entries.
- ◆ List module names, entry names, etc.

USING THE XLIB LIBRARIAN

Once you have assembled and debugged a module intended for general use, like the `rightshift` and `leftshift` modules previously described, you can add them to a library. To do this you start the XLIB Librarian by typing:

XLIB 

XLIB runs in an interactive mode, and displays a `*` prompt for you to enter your command.

The first thing you need to do within XLIB is define the CPU you are using:

DEF-CPU Z80

Then, extract the modules you want from `shifts.r01` into a library called `math.r01`. To do this use:

FETCH-MODULES

This prompts for a number of arguments which it requires:

<i>Prompt</i>	<i>What you type</i>
Source file	<code>shifts</code>
Destination file	<code>math</code>
Start module	To use the default, which is the first in the file, press
End module	To use the default, which is the last in the file, press

This creates the file `math.r01` which contains the code for the `leftshift` and `rightshift` routines.

You can confirm this by typing:

LIST-MODULES

This prompts for the following arguments:

<i>Prompt</i>	<i>What you type</i>
Object file	<code>math</code>
List file	(to use the screen).
Start module	(to start from the first module).
End module	(to end at the last module).

Finally, exit from XLIB by typing:

EXIT

You could use the same procedure to add further modules to the `math` library at any time.

TUTORIAL

COMMAND LINE OPTIONS SUMMARY

This chapter gives a summary of the assembler command line options.

You can list the available options by typing:

az80 

ASSEMBLER CONTROL

- <i>E</i> <i>number</i>	Limits the number of errors.
- <i>f</i> <i>filename</i>	Reads commands from a file.
- <i>G</i>	Opens standard input as source.
- <i>M</i> <i>l</i> <i>r</i>	Changes macro quote characters.
- <i>S</i>	Selects silent operation of the assembler.
- <i>uu</i>	Allows use of undocumented instructions.
- <i>vn</i>	Selects processor type.
- <i>w</i> [<i>string</i>]	Disables warnings.

LISTING CONTROL

- <i>B</i>	Prints debug information for macros.
- <i>c</i> <i>options</i>	Defines listing options.
- <i>i</i>	Lists included files.
- <i>l</i> <i>filename</i>	Generates a listing file.
- <i>L</i> [<i>prefix</i>]	Defines the listing file prefix.
- <i>N</i>	Disables header in listing.

COMMAND LINE OPTIONS SUMMARY

<code>-pnn</code>	Sets the number of lines per page.
<code>-T</code>	Lists active lines only.
<code>-tn</code>	Sets the tab spacing.
<code>-xoptions</code>	Generates a cross reference listing.

OBJECT FILE

<code>-b</code>	Makes the object code a library module.
<code>-o filename</code>	Defines the object file.
<code>-Oprefix</code>	Defines the object file prefix.
<code>-r</code>	Enables debugger output.

PREPROCESSOR CONTROL

<code>-Dsymb[=value]</code>	Defines a symbol.
<code>-Iprefix</code>	Adds an include file search prefix.
<code>-Usymb</code>	Undefines a symbol.

COMMAND LINE OPTIONS

This chapter lists the assembler command line options.

-B

Prints debug information for assembler macros.

SYNTAX

-B

DESCRIPTION

This option lists debugging information about each macro call to the terminal.

For each macro call the following information is shown:

- ◆ Name of the macro.
- ◆ Macro definition.
- ◆ Arguments to the macro.
- ◆ Expanded macro.

-b

-b

Makes the object code a library module.

SYNTAX

-b

DESCRIPTION

The effect of this option is overridden by NAME or PROGRAM directives.

-c

Defines listing options.

SYNTAX

-c*options*

PARAMETERS

<i>options</i>	Any of the following:
A	List only assembled part.
C	List total cycle count.
D	Disable listing.
E	Disable macro expansions.
M	List macro definitions.
O	List multi-line lines of code.
S	No structured assembly list.

-D

Defines a symbol.

SYNTAX

-D*symb*[=*value*]

PARAMETERS

symb Symbol to be defined

value Value to be assigned to symbol; if omitted it defaults to 1.

DESCRIPTION

-D*symb* is equivalent to **#define** *symb* 1

-D*symb=**value* is equivalent to **#define** *symb value*

-E

Limits the number of errors.

SYNTAX

-E*number*

PARAMETERS

number Maximum number of errors to be listed.

-f

-f

Reads commands from a file.

SYNTAX

-f filename

PARAMETERS

filename Filename of command file; if omitted the extension
defaults to .xcl.

EXAMPLE

To read commands from a file runasm.xcl:

az80 -f runasm

-G

Opens standard input as source.

SYNTAX

-G

-I

Adds an include file search prefix.

SYNTAX

-Iprefix

PARAMETERS

prefix The #include file prefix to be added.

-i

Lists included files.

SYNTAX

-i

-L

Defines the listing file prefix.

SYNTAX

*-L[*prefix*]*

PARAMETERS

prefix Prefix for listing files. The listing is created on
prefix source.lst.

-l

-l

Generates a listing file.

SYNTAX

-l *filename*

PARAMETERS

filename File to which listing should be sent; default extension
 .lst.

-M

Changes macro quote characters.

SYNTAX

-M*l r*

PARAMETERS

l Left quote character; default <.
r Right quote character; default >.

EXAMPLE

To use [and] as the quote characters, assemble with:

-M[]

You could then pass the argument a , b to a macro print by writing:

print [a,b]

-N

Disables header in listing.

SYNTAX

-N

-O

Defines the object file prefix.

SYNTAX

-O*prefix*

PARAMETERS

prefix Prefix for object files. The object file is created as
prefix source.r01.

-O

Defines the object file.

SYNTAX

-o *filename*

-p

PARAMETERS

filename Name of object file. The default extension is .r01.

EXAMPLE

To assemble to an output file patch:

```
az80 prog -o patch
```

-p

Sets the number of lines per page.

SYNTAX

-pnn

PARAMETERS

nn The number of lines per page, from 10 to 150.

-r

Enables debugger output.

SYNTAX

-r

-S

Selects silent operation of the assembler.

SYNTAX

-S

DESCRIPTION

Use -S to disable the sign-on message.

-T

Lists active lines only.

SYNTAX

-T

DESCRIPTION

Use the -T option to list only the active lines; ie #if etc true.

-t

Sets the tab spacing.

SYNTAX

-tn

PARAMETERS

n Tab spacing: between 2 and 9. The default is 8.

-U

-U

Undefines a symbol.

SYNTAX

-U *syb*

PARAMETERS

syb Symbol to be undefined.

DESCRIPTION

Equivalent to *#undef syb*.

-uu

Allows use of undocumented instructions.

SYNTAX

-uu

DESCRIPTION

For a full list of undocumented instructions see *Assembler mnemonics*, page 1-135.

-V

Selects processor type.

SYNTAX

-v n

PARAMETERS

<i>n</i>	Processor type:
0	Z80 (the default).
1	64180.

-W

Disables warnings.

SYNTAX

*-w[*string*]*

PARAMETERS

<i>string</i>	Warnings to be disabled, specified as a list of the following separated by commas:
+	Enable all warnings.
-	Disable all warnings.
<i>+n</i>	Enable warning <i>n</i> .
<i>-n</i>	Disable warning <i>n</i> .
<i>+m-n</i>	Enable warnings <i>m</i> to <i>n</i> .
<i>-m-n</i>	Disable warnings <i>m</i> to <i>n</i> .

-x

EXAMPLES

-w turns all warnings off.

-w-,+10-12,+20 turns off all but 10, 11, 12, and 20.

-X

Generates a cross reference listing.

SYNTAX

-xoptions

PARAMETERS

<i>options</i>	Any of the following:
D	Show all #defines.
I	Show internal table.
2	Dual line space listing.

DESCRIPTION

Use -x to enable the listing of symbols. See also the LSTXRF directive.

The I option gives an alphabetical list of all internal symbols; ie, symbols that have been defined before assembly, such as register, directive, and instruction names.

This can be helpful in detecting whether a label has the same name as an internal symbol.

ASSEMBLER SOURCE FORMAT

This chapter describes the source format for the Z80 Assembler, and the format of assembler listings.

SOURCE FORMAT

The format of an assembler source line is as follows:

[label [:]] operation [operands] [: comment]

where the components are as follows:

<i>label</i>	A label, which is assigned the value and type of the current location counter (PLC). The : (colon) is optional if the label starts in the first column.
<i>operation</i>	An assembler instruction or directive. This must not start in the first column.
<i>operands</i>	One or more operands, separated by commas.
<i>comment</i>	A comment, preceded by a ; (semicolon).

The fields can be separated by spaces or tabs.

A source line may not exceed 255 characters.

Tab characters (ASCII 09H), are expanded according to the most common practice; ie to columns 8, 16, 24 etc.

EXPRESSIONS AND OPERATORS

Expressions can consist of operands and operators.

The assembler will accept a wide range of expressions, including both arithmetic and logical operations. All operators use 32-bit two's complement integers, and range checking is only performed when a value is used to generate code.

Expressions are evaluated from left to right, unless this order is overridden by the priority of operators.

The valid operands in an expression are:

- ◆ User-defined symbols and labels.
- ◆ Constants, excluding floating point constants.
- ◆ The location counter (PLC) symbol, \$.
- ◆ The target identity symbol __TID__.

These are described in greater detail in the following sections.

The valid operators are described in the chapters *Assembler operator summary*, page 1-53, and *Assembler operator reference*, page 1-57.

SYMBOLS

User-defined symbols can be up to 255 characters long, and all characters are significant.

Symbols must begin with a letter, a-z or A-Z, ? (question mark), @ (at sign), comma, or full stop. Symbols can include the digits 0-9, and \$ (dollar). For user-defined symbols, case is insignificant unless the directive CASEON has been used. For built-in symbols like instructions, registers, operators, and directives case is insignificant.

ASSEMBLER SOURCE FORMAT

LABELS

Symbols used for memory locations are referred to as labels.

Location counter

The location counter is called \$. For example:

```
LD      HL,$      ; Load this address
```

INTEGER CONSTANTS

Since all IAR Assemblers use 32-bit two's complement internal arithmetic, integers have a (signed) range from -2147483648 to 2147483647.

Constants are written as a sequence of digits with an optional - (minus) sign in front to indicate a negative number.

Commas and decimal points are not permitted.

The IAR Assemblers support C-language style constants as well as the formats of chip manufacturers. If no explicit base is given (eg 10 as opposed to 0x10 or 10Q etc) the default base is used. The default base is 10 at the start of assembly but can be changed via the RADIX directive.

The following formats are supported:

Hexadecimal

Hexadecimal numbers can be written in any of the following formats:

<i>Format</i>	<i>Example</i>	<i>Value</i>
<i>0xhex-digits</i>	0x20	32 in decimal.
<i>H'hex-digits</i>	H'80	128 in decimal.
<i>hex-digitsH</i>	20H	32 in decimal*.

*Note that if the first digit is A–F, a leading zero must be included; for example, 0AH.

ASSEMBLER SOURCE FORMAT

Octal

Octal numbers can be written in any of the formats:

<i>Format</i>	<i>Example</i>	<i>Value</i>
<i>'\octal-digits'</i>	<i>'\10'</i>	8 in decimal.
<i>Q'octal-digits</i>	<i>Q'10</i>	8 in decimal.
<i>octal-digitsQ</i>	<i>10Q</i>	8 in decimal.

Decimal

Decimal numbers can be written in the formats:

<i>Format</i>	<i>Example</i>	<i>Value</i>
<i>digits</i>	<i>123</i>	123 in decimal.
<i>D'digits</i>	<i>D'78</i>	78 in decimal.

Binary

Binary numbers can be written in any of the formats:

<i>Format</i>	<i>Example</i>	<i>Value</i>
<i>B'binary-digits</i>	<i>B'10</i>	2 in decimal.
<i>binary-digitsB</i>	<i>10B</i>	2 in decimal.

ASCII CHARACTER CONSTANTS

ASCII constants can consist of between zero and four characters enclosed in single or double quotes. Enclosing a string in double quotes terminates it with a null character. Only printable characters and spaces may be used in ASCII strings.

ASSEMBLER SOURCE FORMAT

If the quote character itself is to be accessed, two consecutive quotes must be used:

<i>Format</i>	<i>Value</i>
'ABCD'	ABCD (four characters).
"ABCD"	ABCD'\0' (five characters, the last ASCII null).
'A''B'	A'B
'A''''	A'
'''' (4 quotes)	'
'' (2 quotes)	Empty string (value = 0).
""	Empty string (an ASCII null character).
\'	'
\\	\

REAL NUMBER CONSTANTS

The IAR Assembler will accept real numbers as constants and convert them into IEEE single-precision (signed 32-bit) real number format.

Floating point numbers can be written in the format:

$[+|-][digits].[digits][\{E|e\}[+|-]digits]$

or:

$:hex-digits$

Some valid examples are as follows:

<i>Format</i>	<i>Value</i>
10.23	1.023×10^1
1.23456E-24	1.23456×10^{-24}
:3F800000	Explicit hexadecimal floating point (1.0).

ASSEMBLER SOURCE FORMAT

The floating point numbers above with a period must have a digit on each side of the period. This restriction has been introduced to avoid conflict with operators like `.AND.` that are used for some targets.

THE TARGET IDENTITY SYMBOL

The `__TID__` symbol consists of two bytes:

High byte: the target identity; for the Z80 this is 0x04.

Low byte:

Bit 4 is the processor option *n* from the `-vn` command line option.

Bit 7 is set if the `-uu` command line option is specified.

Thus, for the Z80 the values of `__TID__` are as follows:

<i>Processor</i>	<i>Without -uu</i>	<i>With -uu</i>
Z80	0x0400	0x0480
64180	0x0410	0x0490

LISTING FORMAT

The format of the IAR Assembler listing is as follows:

```
#####
#
#   IAR Systems Z80/64180 Assembler V3.00A/DOS 25/May/94  10:13:37   #
#
#   Target option - 64180
#   Source file   - power2.s01
#   List file    - power2.lst
#   Object file   - power2.r01
#   Command line  - power2 -l power2 -v1
#
#                                     (c) Copyright IAR Systems 1993 #
#####
```

Header

```

1  000000          NAME    power2
2  000000
3  000000          LSTXRF+
9  000000
10 000000 3E17      begin LD    A,23
11 000002          power2 3
11.1 000002          REPT   3
11.2 000002          SLA    A
11.3 000002          ENDR
11.4 000002 CB27       SLA    A
11.5 000004 CB27       SLA    A
11.6 000006 CB27       SLA    A
11.7 000008          ENDM
12 000008 C9          RET
13 000009
14 000009          END     begin
```

Assembler generated lines

```

Segment      Type  Mode
-----
ASEG          CODE      ABS Org:0

Label        Mode  Type      Segment  Value
-----
begin        ABS   CONST UNTYP.  ASEG      0
```

Segment list

Symbol list

```
#####
#   CRC:21B      #
#   Errors: 0    #
#   Warnings: 0  #
#####
```

CRC

ASSEMBLER SOURCE FORMAT

The segment list and symbol list are only produced if the LSTXRF+ directive has been specified.

The header, with assembly parameters, is only output on listings directed to files other than the terminal.

Assembly list information is put into five fields:

11	0 000000			LSTCYC+
12	0 000000 21....	capture	LD	HL,inbuf+256
13	1 000003 3A....	capt2	LD	A,(ioport)
14	2 000006 77		LD	(HL),A
15	4 000007 10FA		DJNZ	capt2
16	6 000009 C9		RET	

Source line number	Cycle count	Address field	Data field	Source line
-----------------------	----------------	------------------	---------------	-------------

Source line number

The line number in the source file.

Lines generated by macros and directives for structured assembly will, if listed, have their source line number fields extended with a decimal part; eg 10.5.

Cycle count

The instruction cycle count.

This is only shown if the LSTCYC+ directive, or -cC command line option, has been used.

Address and data fields

These are always listed in hexadecimal notation.

Source line

Lists the source file line.

OUTPUT FORMATS

The relocatable and absolute output is in the same format for all assemblers, because object code is always meant to be processed by the IAR XLINK Linker.

The output from XLINK, however, is in absolute formats normally compatible with the chip vendor's debugger programs (monitors), as well as with PROM programmers and stand-alone emulators from independent sources.

ASSEMBLER SOURCE FORMAT

ASSEMBLER OPERATOR SUMMARY

This chapter summarizes the assembler operators, classified according to their precedence. A full alphabetical reference list of operators is given in the next chapter.

PRECEDENCE OF OPERATORS

Each operator has a precedence number assigned to it which determines the order in which the operator and its operands are evaluated. The precedence numbers range from 1 (the highest precedence, or first evaluated) to 7 (the lowest precedence, or last evaluated).

The following rules determine how expressions are evaluated:

- ◆ The highest precedence (lowest number) operators are evaluated first, then the next highest precedence operators, and so on until the lowest precedence operators are evaluated.
- ◆ Operators of equal precedence are evaluated from left to right in the expression.
- ◆ Parentheses (and) can be used to group operators and operands and to control the order in which the expressions are evaluated. For example, the following expression evaluates to 1:

$8 / ((1+1) * (2+2))$

The following tables give a summary of the operators, in order of priority. Synonyms, where available, are shown in brackets after the operator name:

ASSEMBLER OPERATOR SUMMARY

UNARY OPERATORS – 1

-	Unary minus.
+	Unary plus.
NOT(!)	Logical NOT.
LOW	Low byte of.
HIGH	High byte of.
BYTE3	Third byte of.
LWRD	Low word of.
HWRD	High word of.
DATE	Current date/time.
SFB	Segment begin.
SFE	Segment end.
SIZEOF	Segment size.
BINNOT(~)	Bitwise NOT.

MULTIPLICATIVE ARITHMETIC OPERATORS – 3

*	Multiplication.
/	Division.
MOD (%)	Modulo.
SHR (>>)	Logical shift right.
SHL (<<)	Logical shift left.

ADDITIVE ARITHMETIC OPERATORS – 4

+	Addition.
-	Subtraction.

ASSEMBLER OPERATOR SUMMARY

AND OPERATORS – 5

AND(&&)	Logical AND.
BINAND(&)	Bitwise AND.

OR OPERATORS – 6

OR ()	Logical OR.
XOR	Logical exclusive OR.
BINOR()	Bitwise OR.
BINXOR(^)	Bitwise XOR.

COMPARISON OPERATORS – 7

EQ (=, ==)	Equal.
NE (<>, !=)	Not equal.
GT (>)	Greater than.
LT (<)	Less than.
UGT	Unsigned greater than.
ULT	Unsigned less than.
GE (>=)	Greater than or equal.
LE (<=)	Less than or equal.

ASSEMBLER OPERATOR SUMMARY

ASSEMBLER OPERATOR REFERENCE

This section gives an alphabetical list of the assembler operators with a full description of each one.

The format of each operator description is as follows:

Name	Precedence	
DATE	1	
Date of assembly.		
DESCRIPTION		Description
Use the DATE operator to give the moment when the current assembly began.		
The DATE operator takes an absolute argument (expression) and returns:		
DATE 1 Current second (0-59).		
DATE 2 Current minute (0-59).		
DATE 3 Current hour (0-23).		
DATE 4 Current day (1-31).		
DATE 5 Current month (1-12).		
DATE 6 Current year MOD 100 (1983 → 83).		
EXAMPLES		Examples
To assemble the date of assembly:		
today DEFB DATE 5, DATE 4, DATE 3		

NAME

The operator name, and where appropriate, any synonyms for the operator, and the operator precedence.

Note that all symbolic operators may alternatively be written with leading and trailing periods; ie NOT and DATE may be written .NOT. and .DATE..

The operator name is followed by a description of the operator.

ASSEMBLER OPERATOR REFERENCE

DESCRIPTION

A detailed description covering the operator's most general use.

EXAMPLES

Examples, illustrating typical applications of the operator and clarifying any special cases.

3

Multiplication.

DESCRIPTION

* produces the product of its two operands. The operands are taken as signed 32-bit integers and the result is also a signed 32-bit integer.

EXAMPLES

$2 * 2 \rightarrow 4$

$-2 * 2 \rightarrow -4$

+

1

Unary plus.

DESCRIPTION

Unary plus operator.

+

4

Addition.

DESCRIPTION

The + addition operator produces the sum of the two operands which surround it. The operands are taken as signed 32-bit integers and the result is also a signed 32-bit integer.

ASSEMBLER OPERATOR: -

EXAMPLES

$92+19 \rightarrow 111$

$-2+2 \rightarrow 0$

$-2+-2 \rightarrow -4$

-

1

Unary minus.

DESCRIPTION

The - unary minus operator performs arithmetic negation on its operand.

The operand is interpreted as a 32-bit signed integer and the result of the operator is the 2's complement negation of that integer.

-

4

Subtraction.

DESCRIPTION

The - subtraction operator produces the difference when the right-hand operand is taken away from the left-hand operand. The operands are taken as signed 32-bit integers and the result is also signed 32-bit integer.

EXAMPLES

$92-19 \rightarrow 73$

$-2-2 \rightarrow -4$

$-2--2 \rightarrow 0$

/**3**

Division.

DESCRIPTION

/ produces the integer quotient of the left-hand operand divided by the right-hand operator. The operands are taken as signed 32-bit integers and the result is also a signed 32-bit integer.

EXAMPLES

8/2 → 4

-12/3 → -4

AND (&&)**5**

Logical AND.

DESCRIPTION

Use AND to perform logical AND between its two integer operands. If both operands are non-zero the result is 1; otherwise it is zero.

EXAMPLES

1010B AND 0011B → 1

1010B AND 0101B → 1

1010B AND 0000B → 0

BINAND (&)

BINAND (&)

5

Bitwise AND.

DESCRIPTION

Use BINAND to perform bitwise AND between the integer operands.

EXAMPLES

1010B BINAND 0011B → 0010B
1010B BINAND 0101B → 0000B
1010B BINAND 0000B → 0000B

BINNOT (~)

1

Bitwise NOT.

DESCRIPTION

Use BINNOT to perform bitwise NOT on its operand.

EXAMPLES

BINNOT 1010B → 111111111110101B

BINOR (|)

6

Bitwise OR.

DESCRIPTION

Use BINOR to perform bitwise OR on operands.

EXAMPLES

1010B BINOR 0101B → 1111B

1010B BINOR 0000B → 1010B

BINXOR (^)**6**

Bitwise exclusive-OR.

DESCRIPTION

Use BINXOR to perform bitwise XOR on its operands.

EXAMPLES

1010B BINXOR 0101B → 1111B

1010B BINXOR 0011B → 1001B

BYTE3**1**

Third byte of.

DESCRIPTION

BYTE3 returns the third byte of its operand.

EXAMPLES

BYTE3 1234ABCDh → 34h

DATE

DATE

1

Date of assembly.

DESCRIPTION

Use the DATE operator to give the moment when the current assembly began.

The DATE operator takes an absolute argument (expression) and returns:

DATE 1 Current second (0-59).
DATE 2 Current minute (0-59).
DATE 3 Current hour (0-23).
DATE 4 Current day (1-31).
DATE 5 Current month (1-12).
DATE 6 Current year MOD 100 (1983 → 83).

EXAMPLES

To assemble the date of assembly:

today DEFB DATE 5, DATE 4, DATE 3

EQ (=,==)

7

Equal.

DESCRIPTION

EQ evaluates to 1 (true) if its two operands are identical in value, or to 0 (false) if its two operands are not identical in value.

EXAMPLES

1 EQ 2 → 0
2 EQ 2 → 1
'ABC' EQ 'ABCD' → 0

GE (>=)

7

Greater or equal.

DESCRIPTION

GE evaluates to 1 (true) if the left-hand operand is equal to or has a higher numeric value than the right-hand operand.

EXAMPLES

1 GE 2 → 0
2 GE 1 → 1
1 GE 1 → 0

GT (>)

7

Greater than.

DESCRIPTION

GT evaluates to 1 (true) if the left-hand operand has a higher numeric value than the right-hand operand.

EXAMPLES

-1 GT 1 → 0
2 GT 1 → 1
1 GT 1 → 0

HIGH

HIGH

1

High byte of.

DESCRIPTION

HIGH takes a single operand to its right which is interpreted as an unsigned, 16-bit integer value. The result is the unsigned 8-bit integer value of the higher order byte of the operand.

EXAMPLES

HIGH 1234ABCDh → ABh

HWRD

1

High word of.

DESCRIPTION

HWRD takes a single operand to its right, which is interpreted as an unsigned, 32-bit integer value. The result is the unsigned 16-bit integer value of the higher order word of the operand.

EXAMPLES

HWRD 1234ABCDh → 1234h

LE (<=)

7

Less or equal.

DESCRIPTION

LE evaluates to 1 (true) if the left-hand operand has a lower numeric value or is equal to the right-hand operand.

EXAMPLES

1 LE 2 → 1
2 LE 1 → 0
1 LE 1 → 1

LOW

1

Low byte of.

DESCRIPTION

LOW takes a single operand, which is interpreted as an unsigned, 32-bit integer value. The result is the unsigned 8-bit integer value of the lower order byte of the operand.

EXAMPLES

LOW 1234ABCDh → CDh

LT (<)

LT (<)

7

Less than.

DESCRIPTION

LT evaluates to 1 (true) if the left-hand operand has a lower numeric value than the right-hand operand.

EXAMPLES

-1 LT 2 → 1

2 LT 1 → 0

2 LT 2 → 0

LWRD

1

Low word of.

DESCRIPTION

LWRD takes a single operand which is interpreted as an unsigned, 32-bit integer value. The result is the unsigned 16-bit integer value of the lower order word of the operand.

EXAMPLES

LWRD 1234ABCDh → ABCDh

MOD (%)

3

Modulo.

DESCRIPTION

MOD produces the remainder from the integer division of the left-hand operand by the right-hand operand. The operands are taken as signed 32-bit integers and the result is also a signed 32-bit integer.

$X \text{ MOD } Y$ is equivalent to $X - Y * (X / Y)$ using integer division.

EXAMPLES

2 MOD 2 $\rightarrow$ 0
12 MOD 7 $\rightarrow$ 5
3 MOD 2 $\rightarrow$ 1

NE (<>, !=)

7

Not equal.

DESCRIPTION

NE evaluates to 0 (false) if its two operands are identical in value or to 1 (true) if its two operands are not identical in value.

EXAMPLES

1 NE 2 $\rightarrow$ 1
2 NE 2 $\rightarrow$ 0
'A' NE 'B' $\rightarrow$ 1

NOT (!)

NOT (!)

1

Logical NOT.

DESCRIPTION

Use NOT to negate a logical argument.

EXAMPLES

NOT 0101B → 0

NOT 0000B → 1

OR (||)

6

Logical OR.

DESCRIPTION

Use OR to perform a logical OR between two integer operands.

EXAMPLES

1010B OR 0000B → 1

0000B OR 0000B → 0

SFB

1

Segment begin.

SYNTAX

SFB(*segment* [{+ | -} *offset*])

PARAMETERS

segment The name of a relocatable segment.

offset An optional offset from the start address. The parentheses are optional if *offset* is omitted.

DESCRIPTION

SFB accepts a single operand to its right. The operand must be the name of a relocatable segment. The operator evaluates to the absolute address of the first byte of that segment. This evaluation takes place at linking time.

EXAMPLES

```
NAME demo
RSEG CODE
start DEFL SFB(CODE)
```

Even if the above code is linked with many other modules, *start* will still be set to the address of the first byte of the segment.

SFE**1**

Segment end.

SYNTAX

SFE(*segment* [{+ | -} *offset*])

PARAMETERS

segment The name of a relocatable segment.

offset An optional offset from the start address. The parentheses are optional if *offset* is omitted.

SHL (<<)

DESCRIPTION

SFE accepts a single operand to its right. The operand must be the name of a relocatable segment. The operator evaluates to the segment start address plus the segment size. This evaluation takes place at linking time.

EXAMPLES

```
NAME demo
RSEG CODE
end DEFL SFE(CODE)
```

Even if the above code is linked with many other modules, end will still be properly set to the address of the last byte of the segment.

SHL (<<)

3

Logical shift left.

DESCRIPTION

Use SHL to shift the left-hand operand to the left. The number of bits to shift is specified by the right-hand operand, interpreted as an integer value between 0 and 32.

EXAMPLES

```
00011100B SHL 3 → 11100000B
0000011111111111B SHL 5 → 1111111111100000B
14 SHL 1 → 28
```

SHR (>>)

3

Logical shift right.

DESCRIPTION

SHR shifts the left-hand operand to the right. The number of bits to shift specified by the right-hand operand, interpreted as an integer value between 0 and 32.

EXAMPLES

01110000B SHR 3 → 00001110B
1111111111111111B SHR 20 → 0
14 SHR 1 → 7

SIZEOF

1

Segment size.

SYNTAX

`SIZEOF segment`

PARAMETERS

segment The name of a relocatable segment.

DESCRIPTION

SIZEOF generates SFE-SFB for its argument, which should be the name of a relocatable segment; ie it calculates the size in bytes of a segment. This is done when modules are linked together.

UGT

EXAMPLES

```
NAME    demo
RSEG    CODE
size    DEFL    SIZEOF CODE
sets size to the size of segment CODE.
```

UGT

7

Unsigned greater than.

DESCRIPTION

UGT evaluates to 1 (true) if the left-hand operand has a larger absolute value than the right-hand operand. It assumes that the arguments are unsigned.

EXAMPLES

```
2 UGT 1 → 1
1 UGT 2 → 0
```

ULT

7

Unsigned less than.

DESCRIPTION

ULT evaluates to 1 (true) if the left-hand operand has a smaller absolute value than the right-hand operand. It assumes that the arguments are unsigned.

EXAMPLES

1 ULT 2 $\rightarrow$ 1

2 ULT 1 $\rightarrow$ 0

XOR**6**

Logical exclusive-OR.

DESCRIPTION

Use XOR to perform logical XOR on its two operands.

EXAMPLES

0101B XOR 1010B $\rightarrow$ 0

0101B XOR 0000B $\rightarrow$ 1

XOR

ASSEMBLER

DIRECTIVES REFERENCE

This section gives a list of the Z80 Assembler directives, classified according to their function, with a full description of their operation, and the options available for each one.

The format of each section is as follows:

SYMBOL CONTROL DIRECTIVES	Class
These directives control how symbols are shared between modules.	
PUBLIC Exports symbols to other modules.	Description
EXTERN Imports an external symbol.	
SYNTAX	Syntax
PUBLIC <i>symbol</i> [<i>.symbol</i>] ...	
EXTERN <i>label</i> [<i>.label</i>] ...	
PARAMETERS	Parameters
<i>symbol</i> Symbol to be exported.	
<i>label</i> Label to be exported.	
DESCRIPTION	Full description
Use PUBLIC to make one or more symbols available to other modules. The symbols declared as PUBLIC can only be assigned values by using them as labels. PUBLIC declared symbols can be relocated or absolute, and can also be used in expressions (with the same rules as for other symbols).	
Use EXTERN to import an untyped external label.	
EXAMPLES	Examples
The following example defines a subroutine to print an error message, and exports the entry address <i>err</i> so that it can be called from other modules.	
1 000000 NAME error	
2 000000 EXTERN print	
3 000000 PUBLIC err	
4 000000	
5 000000 CD.... err CALL print	
6 000003 2A2A2A20 DEFM "*** Error ***"	
7 000011 C9 RET	

ASSEMBLER DIRECTIVES REFERENCE

CLASS

The class of directives.

The class is followed by a description of the class, and a description of each directive in the class.

SYNTAX

A full syntax definition of each directive.

PARAMETERS

Details of each parameter in the syntax definitions.

DESCRIPTION

A detailed description covering each directive's most general use. This includes information about what the directives are useful for, and a discussion of any special conditions and common pitfalls.

EXAMPLES

Examples, illustrating typical applications of the directives and clarifying any special cases.

SYNTAX CONVENTIONS

In the syntax definitions the following conventions are used:

Parameters, representing what you would type, are shown in italics. So, for example, in:

`RADIX expr`

expr represents an arbitrary expression.

Optional parameters are shown in square brackets. So, for example, in:

`END [expr]`

the *expr* parameter is optional.

ASSEMBLER DIRECTIVES REFERENCE

An ellipsis indicates that the previous item can be repeated an arbitrary number of times. For example:

`LOCAL symbol [.symbol] ...`

indicates that `LOCAL` can be followed by one or more symbols, separated by commas.

Alternatives are enclosed in { and } brackets, separated by a vertical bar. For example:

`LSTOUT{+ | -}`

indicates that the directive must be followed by either + or -.

LABELS AND COMMENTS

Where a directive must be preceded by a label, this is indicated in the syntax, as in:

`label SET expr`

All other directives can be preceded by an optional label, which will assume the value and type of the current location counter (PLC), and for clarity this is not included in each syntax definition.

In addition, unless specified all directives can be followed by an optional comment preceded by ;.

PARAMETERS

The following table shows the correct form of the most commonly-used types of parameter:

<i>Parameter</i>	<i>What it consists of</i>
<i>symbol</i>	An assembler symbol.
<i>label</i>	A symbolic label.
<i>expr</i>	An expression; see <i>Expressions and operators</i> , page 1-44.

MODULE CONTROL DIRECTIVES

MODULE CONTROL DIRECTIVES

Module control directives are used to mark the beginning and end of source program modules, and to assign names and types to them.

NAME	Begins a program module.
MODULE	Begins a library module.
ENDMOD	Terminates the assembly of the current module.
END	Terminates the assembly of the last module in a file.

SYNTAX

NAME *symbol* [(*expr*)]

MODULE *symbol* [(*expr*)]

ENDMOD [*label*]

END [*label*]

PARAMETERS

<i>symbol</i>	Name assigned to module, used by XLIB when referencing the module.
<i>expr</i>	Optional expression (0-255) used by the IAR C Compiler.
<i>label</i>	An expression or label which can be resolved at assembly time. It is output in the object code as a program entry address.

DESCRIPTION

Beginning a program module

Use NAME to begin a program module, and assign a name for future reference by XLINK and XLIB.

Program modules are unconditionally linked by XLINK, even if they are not referenced by other modules.

MODULE CONTROL DIRECTIVES

Beginning a library module

Use `MODULE` to create libraries containing lots of small modules (like run time systems for high level languages), where each module also often represent a single routine. With the multi-module facility you can significantly reduce the number of source and object files needed.

Library modules are only copied into the linked code if a public symbol in the module is referenced by other modules.

Terminating a module

Use `ENDMOD` to define the end of a module.

Terminating the last module

Use `END` to indicate the end of the source file. Any lines after the `END` directive are ignored.

You can specify a label after the `END` directive to define it as a program entry address. Program entries must be either relocatable or absolute (no externals allowed), and will show up in `XLINK` load maps, as well as in some of the hexadecimal absolute output formats.

The following rules apply to multi-module assemblies:

- ◆ At the beginning of a new module all user symbols are deleted, except for those created by `DEFINE`, `#define`, or `MACRO`, the location counters are cleared, and the mode is set to absolute.
- ◆ List control directives remain in effect throughout the assembly.

Note that `END` must always be used in the *last* module, and that there must not be any source lines (except for comments and list control directives) between an `ENDMOD` and a `MODULE` directive.

If the `NAME` or `MODULE` directive is missing, the module will be assigned the name of the source file and the attribute `program`.

MODULE CONTROL DIRECTIVES

EXAMPLES

The following example defines three modules:

```
MODULE
.
. Module #1
.
ENDMOD
MODULE
.
. Module #2
.
ENDMOD
MODULE
.
. Last module
.
END
```

SYMBOL CONTROL DIRECTIVES

These directives control how symbols are shared between modules.

PUBLIC	Exports symbols to other modules.
EXTERN	Imports an external symbol.

SYNTAX

PUBLIC *symbol* [*,symbol*] ...

EXTERN *label* [*,label*] ...

PARAMETERS

symbol Symbol to be exported.

label Label to be imported.

DESCRIPTION

Exporting symbols to other modules

Use **PUBLIC** to make one or more symbols available to other modules. The symbols declared as **PUBLIC** can only be assigned values by using them as labels. **PUBLIC** declared symbols can be relocated or absolute, and can also be used in expressions (with the same rules as for other symbols).

The **PUBLIC** directive always exports full 32-bit values, which makes it feasible to use global 32-bit constants also in assemblers for 8 and 16-bit processors. With the **LOW**, **HIGH**, **LWRD**, and **HWRD** operators any part of such a constant can be loaded in a 8 or 16-bit register or word.

There are no restrictions on the number of **PUBLIC** declared symbols in a module.

Importing symbols

Use **EXTERN** to import an untyped external label.

SYMBOL CONTROL DIRECTIVES

EXAMPLES

The following example defines a subroutine to print an error message, and exports the entry address `err` so that it can be called from other modules.

It defines `print` as an external routine; the address will be resolved at link time.

```
1  000000          NAME  error
2  000000          EXTERN print
3  000000          PUBLIC err
4  000000
5  000000 CD....   err  CALL  print
6  000003 2A2A2A20  DEFM   "**** Error ****"
7  000011 C9        RET
8  000012
9  000012          END   err
```

SEGMENT CONTROL DIRECTIVES

The segment directives control how code and data are generated.

ASEG	Begins an absolute segment.
RSEG	Begins a relocatable segment.
STACK	Begins a stack segment.
COMMON	Begins a common segment.
ORG	Sets the location counter.

SYNTAX

ASEG [*start*]
RSEG *segment* [:*type*]
STACK *segment* [:*type*]
COMMON *segment* [:*type*]
ORG *expr*

PARAMETERS

<i>start</i>	A start address which has the same effect as using an ORG directive at the beginning of the absolute segment.
<i>segment</i>	The name of the segment.
<i>type</i>	The memory type; one of: UNTYPED (the default), CODE, or DATA. In addition, the following types are provided for compatibility with the IAR C Compilers: XDATA, IDATA, BIT, REGISTER, and CONST.
<i>expr</i>	Address to set location counter to.

SEGMENT CONTROL DIRECTIVES

DESCRIPTION

Beginning an absolute segment

Use ASEG to set the absolute mode of assembly, which is the default at the beginning of a module.

If the parameter is omitted, the start address of the first segment is 0, and subsequent segments continue after the last address of the previous segment.

Beginning a relocatable segment

Use RSEG to set the current mode of the assembly to relocatable assembly mode. The assembler maintains separate location counters (initially set to zero) for all segments, which makes it possible to switch segments and mode anytime without the need to save the current segment location counter.

Up to 256 unique, relocatable segments may be defined in a single module.

Beginning a stack segment

Use STACK to allocate code or data allocated from high to low addresses (vs. the RSEG directive which causes low-to-high allocation).

Note that the contents of the segment are not generated in reverse order.

Beginning a common segment

Use COMMON to place data in memory at the same location as COMMON segments from other modules that have the same name. In other words, all COMMON segments of the same name will start at the same location in memory and overlay each other.

Obviously, the COMMON segment type should not be used for overlaid executable code. A typical application would be where you desire to have a number of different routines share a reusable, common area of memory for data.

It can be practical to have the interrupt vector table in a COMMON segment, thereby allowing access from several routines.

The final size of the COMMON segment is determined by the size of largest occurrence of this segment. The location in memory is determined by the XLINK -Z command; see *XLINK command reference*, in the *IAR Linker & Librarian* guide.

SEGMENT CONTROL DIRECTIVES

Setting the location counter

Use **ORG** to set the location counter of the current segment to the value of an expression. The optional label will assume the value and type of the new location counter.

The result of the expression must be of the same type as the current segment, that is, it is not valid to use **ORG 10** during **RSEG**, since the expression is absolute; instead use **ORG \$+10**. The expression must not contain any forward or external references.

All location counters are set to zero at the beginning of an assembly module.

EXAMPLES

Beginning an absolute segment

The following example assembles vector addresses at the first three Z80 restart addresses, using an absolute segment starting at address 00H:

```
        EXTERN  rst1,rst2

        ASEG
        ORG     00H
        CALL    main

        ORG     08H
        CALL    rst1

        ORG     10H
        CALL    rst2

        ORG     40H
main    LD      A,0
        END
```

The main power-on code is assembled in memory starting at 40H. Note that the first **ORG** directive is optional, since if no origin is specified an absolute segment starts at address 0.

SEGMENT CONTROL DIRECTIVES

Beginning a relocatable segment

In the following example the data following the first RSEG directive is placed in a relocatable segment called `table`; the `ORG` directive is used to create a gap of 6 bytes in the `table`.

The code following the second RSEG directive is placed in a relocatable segment called `code`:

```
        EXTERN  divrtn,mulrtn

        RSEG    table
        DEFW    divrtn,mulrtn

        ORG     $+6
        DEFW    subrtn

        RSEG    code
subrtn  LD      A,1
        SUB     B
        END
```

Beginning a stack segment

The following example defines two 100-byte stacks in a relocatable segment called `rpnstack`:

```
        STACK   rpnstack
parms  DEFS     100
opers  DEFS     100

        END
```

The data is allocated from high to low addresses.

SEGMENT CONTROL DIRECTIVES

Beginning a common segment

The following example defines two common segments containing variables:

```
count    NAME    common1
         COMMON  data
         DEFS    4
         ENDMOD
```

```
up       NAME    common2
         COMMON  data
         DEFS    1
         ORG     $+2
down     DEFS    1
         END
```

Because the common segments have the same name, data, the variables up and down refer to the same locations in memory as count.

VALUE ASSIGNMENT DIRECTIVES

These directives are used to assign values to symbols.

DEFL	Defines a temporary value.
EQU	Defines a permanent value local to a module.
=	Synonym for EQU.
DEFINE	Defines a file-wide value.

SYNTAX

label DEFL *expr*

label EQU *expr*

label = *expr*

label DEFINE *expr*

PARAMETERS

label Symbol to be defined.

expr Value assigned to symbol.

DESCRIPTION

Defining a temporary value

Use DEFL to define a symbol which may be redefined, such as for use with macro variables. Symbols defined with DEFL cannot be declared PUBLIC.

Defining a permanent local value

Use EQU or = to assign a value to a symbol. The symbol is only valid in the module containing the EQU or = statement.

A symbol which has been given a value with EQU or = can be made available to other modules with PUBLIC.

To import symbols from other modules use EXTERN.

VALUE ASSIGNMENT DIRECTIVES

Defining a permanent global value

Use **DEFINE** to define symbols that should be known to all modules in the source file.

A symbol which has been given a value with **DEFINE** can be made available to modules in files with the **PUBLIC** directive.

Symbols defined with **DEFINE** cannot be redefined.

EXAMPLES

Redefining a symbol

The following example uses **DEFL** to redefine the symbol **const** in a **REPT** loop to generate a table of the first 8 powers of 3:

```

      NAME    table
main   ; Generate table of powers of 3
const  DEFL   1
      REPT   8
      DEFW   const
const  DEFL   const * 3
      ENDR
      END    main
```

It generates the following code:

```

1  000000      NAME    table
2  000000      main   ; Generate table of powers of 3
3  000001      const  DEFL   1
4  000000      REPT   8
5  000000      DEFW   const
6  000000      const  DEFL   const * 3
7  000000      ENDR
7.1 000000 0100      DEFW   const
7.2 000003      const  DEFL   const * 3
7.3 000002 0300      DEFW   const
7.4 000009      const  DEFL   const * 3
7.5 000004 0900      DEFW   const
7.6 000018      const  DEFL   const * 3
7.7 000006 1800      DEFW   const
7.8 000051      const  DEFL   const * 3
```

VALUE ASSIGNMENT DIRECTIVES

```
7.9 000008 5100      DEFW  const
7.10 0000F3          const DEFL  const * 3
7.11 00000A F300      DEFW  const
7.12 0002D9          const DEFL  const * 3
7.13 00000C D902      DEFW  const
7.14 00088B          const DEFL  const * 3
7.15 00000E 8B08      DEFW  const
7.16 0019A1          const DEFL  const * 3
8    000010          END    main
```

Using local symbols

In the following example the symbol `value` defined in module `add1` is local to that module; a distinct symbol of the same name is defined in module `add2`:

```
NAME      add1
locn      DEFINE 100H
value     EQU     77
          LD      A,(locn)
          ADD     A,value
          RET
          ENDMOD
```

```
NAME      add2
value     EQU     88
          LD      A,(locn)
          ADD     A,value
          RET
          END
```

The symbol `locn` defined in the module `add1` is also available to module `add2`.

CONDITIONAL ASSEMBLY DIRECTIVES

These directives provide logical control over the selective assembly of source code.

<code>.IF</code>	Assembles instructions if a condition is true.
<code>.ELSE</code>	Assembles instructions if a condition is false.
<code>.ENDIF</code>	Ends a <code>.IF</code> block.

SYNTAX

`.IF condition`
`.ELSE`
`.ENDIF`

PARAMETERS

<i>condition</i>	One of the following:	
	An absolute expression	The expression must not contain forward or external references, and any non-zero value is considered as true.
	<i>string1=string2</i>	The condition is true if <i>string1</i> and <i>string2</i> have the same length and contents.
	<i>string1<>string2</i>	The condition is true if <i>string1</i> and <i>string2</i> have different length or contents.

CONDITIONAL ASSEMBLY DIRECTIVES

DESCRIPTION

Use the `.IF ... .ELSE ... .ENDIF` directives to control the assembly process at assembly time. If the condition following the `.IF` directive is not true, the subsequent instructions will not generate any code (ie it will not be assembled or syntax checked) until a `.ENDIF` or `.ELSE` directive is found.

Conditional assembler directives may be used anywhere in an assembly, but have their greatest use in conjunction with macro processing.

All assembler directives (except for `END`), and file inclusion, may be disabled by the conditional directives. Each `.IF` directive must be terminated by a `.ENDIF` directive. The `.ELSE` directive is optional, and if used, it must be inside a `.IF ... .ENDIF` block.

`.IF ... .ENDIF` and `.IF ... .ELSE ... .ENDIF` blocks may be nested to any level.

EXAMPLES

The following macro multiplies A by a constant:

```
/* 8-bit constant multiply */
```

```
        EXTERN  multab

mult    MACRO   times
        .IF     times = 2
        SLA     A
        .ELSE
        LD      B,times
        CALL    multab
        .ENDIF
        ENDM
```

If the argument to the macro is 2 it generates an `SLA` instruction to save instruction cycles; otherwise it calls an external multiply routine.

CONDITIONAL ASSEMBLY DIRECTIVES

It could be tested with the following program:

```
main    LD      A,17
        mult    2
        LD      A,23
        mult    133
        RET
        END     main
```

MACRO PROCESSING DIRECTIVES

These directives allow user macros to be defined.

MACRO	Defines a macro.
ENDM	Ends a macro definition.
EXITM	Exits prematurely from a macro.
LOCAL	Creates symbols local to a macro.
REPT	Assembles instructions a specified number of times.
REPTC	Repeats and substitutes characters.
REPTI	Repeats and substitutes strings.
ENDR	Ends a repeat structure.

SYNTAX

name MACRO [*argument*] ...

ENDM

EXITM

LOCAL *symbol* [,*symbol*] ...

REPT *expr*

REPTC *formal,actual*

REPTI *formal,actual* [,*actual*] ...

ENDR

MACRO PROCESSING DIRECTIVES

PARAMETERS

<i>name</i>	The name of the macro.
<i>argument</i>	A symbolic argument name.
<i>symbol</i>	Symbol to be local to the macro.
<i>expr</i>	An expression.
<i>formal</i>	Argument into which each character of <i>actual</i> (REPTC) or each <i>actual</i> (REPTI) is substituted.
<i>actual</i>	String to be substituted.

DESCRIPTION

A macro is a user-defined symbol that represents a block of one or more assembler source lines. Once you have defined a macro you can use it in your program just like an assembler directive or assembler mnemonic.

When the assembler encounters a macro, it looks up the macro's definition, and inserts the lines that the macro represents as if they were included in the source file at that position.

Although macros effectively perform simple text substitution, you can control what they substitute by supplying parameters to them.

Defining a macro

You define a macro with the statement:

```
macroname MACRO [arg] [arg] ...
```

Here *macroname* is the name you are going to use for the macro, and *arg* is an argument for values you want to pass to the macro when it is expanded.

For example, you could define a macro ERROR as follows:

```
error    MACRO    text
          CALL     abort
          DEFM     text,0
          ENDM
```

MACRO PROCESSING DIRECTIVES

This uses a parameter text to set up an error message for a routine abort. You would call the macro with a statement such as:

```
error    'Disk not ready'
```

This will be expanded by the assembler to:

```
CALL     abort
DEFM     'Disk not ready',0
```

If you omit a list of one or more arguments, the arguments you supply when calling the macro are called \1 to \9 and \A to \Z.

The previous example could therefore be written as follows:

```
error    MACRO
          CALL     abort
          DEFM     \1,0
        ENDM
```

Use the EXITM directive to generate a premature exit from a macro.

EXITM is not allowed inside REPT ... ENDR, REPTC ... ENDR, or REPTI ... ENDR.

Use LOCAL to create symbols local to a macro. The LOCAL directive must be used before the symbol is used.

Each time a macro is expanded new instances of local symbols are created by the LOCAL directive, so it is legal to use local symbols in recursive macros.

It is illegal to *redefine* a macro.

Passing special characters

Macro arguments that include commas or white space can be forced to be interpreted as one argument by using the matching quote characters < and > in the macro call.

For example:

```
mac1     MACRO      regs
          ADD  regs  ; add contents of regs
        ENDM
```

MACRO PROCESSING DIRECTIVES

It could be called using:

```
mac1      <HL,BC>
END
```

You can redefine the macro quote characters with the `-M` command line option; see `-M`, page 1-36.

How macros are processed

There are three distinct phases in the macro process:

- ◆ Scanning and saving of macro definitions is performed by the assembler. The text between `MACRO` and `ENDM` is saved but not syntax checked. Include file references `$file` are recorded and will be included during macro *expansion*.
- ◆ A macro call forces the assembler to invoke the macro processor (expander) which switches (if not already in a macro) the assembler input stream from a source file to the output from the macro expander (which takes its input from the requested macro definition).

The macro expander has no knowledge of assembler symbols since it only deals with text substitutions at source level. Before a line from the called macro definition is handed over to the assembler, the expander scans the line for all occurrences of symbolic macro arguments, and replaces them with their expansion arguments.

- ◆ The expanded line is then processed as any other assembler source line. The input stream to the assembler will continue to be the output from the macro processor, until all lines of the current macro definition have been read.

Repeating statements

Use the `REPT ... ENDR` structure to assemble the same block of instructions a number of times. If *expr* evaluates to 0 nothing will be generated.

Use `REPTC` to assemble a block of instructions once for each character in a string. If the string contains a comma it should be enclosed in quotation marks.

Use `REPTI` to assemble a block of instructions once for each string in a series of strings. Strings containing commas should be enclosed in quotation marks.

MACRO PROCESSING DIRECTIVES

EXAMPLES

This section gives examples of the different ways in which macros can make assembler programming easier.

Extending the instruction set

Because of the way in which microprocessor instruction sets evolve, they are not always as symmetrical as one would like. By writing macros you can add definitions for instructions that you would like to have included in the instruction set, and use them just like the built-in instructions.

For example, the Z80 does not include a TST instruction to test the A register and set flags. You could define a `tsta` macro to do this as follows:

```
NAME      tsta
tsta      MACRO
          OR      A
          ENDM
```

This could then be used in a program as follows:

```
main      tsta
          RET
```

Coding in-line for efficiency

In time-critical code it is often desirable to code routines in-line to avoid the overhead of a subroutine call and return. Macros provide a convenient way of doing this.

For example, the following subroutine outputs bytes from a buffer to a port:

```
NAME      play
EXTERN    port

; Plays B bytes from (HL)
play      LD      A,(HL)
          LD      (port),A
          INC     HL
          DJNZ    play
          RET
```

MACRO PROCESSING DIRECTIVES

The main program calls this routine as follows:

```
buffer  DEFS    256
test    LD      HL,buffer
        LD      B,200
        CALL    play
```

For efficiency we can recode this as the following macro:

```
        NAME     play
        EXTERN   port

; Plays B bytes from (HL)
play    MACRO
        LOCAL    loop
loop    LD      A,(HL)
        LD      (port),A
        INC     HL
        DJNZ    loop
        ENDM
```

Note the use of LOCAL to make the label loop local to the macro; otherwise an error will be generated if the macro is used twice, as the loop label will already exist.

To use in-line code the main program is then simply altered to:

```
buffer  DEFS    256
test    LD      HL,buffer
        LD      B,200
        play
```

MACRO PROCESSING DIRECTIVES

Using REPTC and REPTI

The following example assembles a series of calls to a subroutine plot to plot each character in a string:

```
NAME      reptc

          EXTERN  plotc
banner    REPTC   chr,"Welcome"
          LD      A,'chr'
          CALL    plotc
          ENDR

          END
```

This produces the following code:

```
1  000000          NAME      reptc
2  000000
3  000000          EXTERN  plotc
4  000000      banner    REPTC   chr,"Welcome"
5  000000          LD      A,'chr'
6  000000          CALL    plotc
7  000000          ENDR
7.1 000000 3E57      LD      A,'W'
7.2 000002 CD....    CALL    plotc
7.3 000005 3E65      LD      A,'e'
7.4 000007 CD....    CALL    plotc
7.5 00000A 3E6C      LD      A,'l'
7.6 00000C CD....    CALL    plotc
7.7 00000F 3E63      LD      A,'c'
7.8 000011 CD....    CALL    plotc
7.9 000014 3E6F      LD      A,'o'
7.10 000016 CD....   CALL    plotc
7.11 000019 3E6D      LD      A,'m'
7.12 00001B CD....   CALL    plotc
7.13 00001E 3E65      LD      A,'e'
7.14 000020 CD....   CALL    plotc
8  000023
9  000023          END
```

MACRO PROCESSING DIRECTIVES

The following example uses REPTI to clear a number of memory locations:

```
NAME      repti

          EXTERN  base,count,init
          LD      A,0
clear     REPTI   adds,base,count,init
          LD      (adds),A
          ENDR

          END      clear
```

This produces the following code:

```
1  000000          NAME      repti
2  000000
3  000000          EXTERN  base,count,init
4  000000 3E00          LD      A,0
5  000002          clear    REPTI  adds,base,count,init
6  000002          LD      (adds),A
7  000002          ENDR
7.1 000002 32....          LD      (base),A
7.2 000005 32....          LD      (count),A
7.3 000008 32....          LD      (init),A
8  00000B
9  00000B          END      clear
```

STRUCTURED ASSEMBLY DIRECTIVES

The structured assembly directives allow loops and control structures to be implemented at assembly level.

IF	Specifies instructions to be executed if a condition is true.
ELSE	Introduces instructions to be executed if a condition is false.
ELSEIF	Specifies a new condition in an IF block.
ENDIF (ENDI, THEN)	Ends an IF block.
WHILE	Repeats subsequent instructions while a condition is true.
ENDW (WEND)	Ends a WHILE loop.
REPEAT	Repeats subsequent instructions until a condition is true.
UNTIL	Ends a REPEAT loop.
FOR	Repeats subsequent instructions a specified number of times.
ENDF	Ends a FOR loop.
SWITCH	Multiple case switch.
CASE	Case in SWITCH block.
DEFAULT	Default case in SWITCH block.
ENDS	Ends a SWITCH block.
BREAK	Exits prematurely from a loop or switch construct.
CONTINUE	Continues execution of a loop or switch construct.

STRUCTURED ASSEMBLY DIRECTIVES

SYNTAX

```
IF test {AND test | OR test}  
ELSE  
ELSEIF test {AND test | OR test}  
ENDIF  
  
WHILE test {AND test | OR test}  
ENDW  
  
REPEAT  
UNTIL test {AND test | OR test}  
  
FOR A = start {TO | DOWNT0} end STEP step  
ENDF  
  
SWITCH A  
CASE op  
DEFAULT  
ENDS  
  
BREAK  
CONTINUE
```

PARAMETERS

test

One of the tests given in the following table:

<i>Test</i>	<i>Instructions generated</i>
A <> <i>op</i>	JP Z
A < <i>op</i>	JP P, JP Z
A <= <i>op</i>	JP P
A = <i>op</i>	JP NZ
A >= <i>op</i>	JP M
A > <i>op</i>	JP M, JP Z

op

One of the following operands:

B, C, D, E, H, L, n.

start, end, step

Constants.

STRUCTURED ASSEMBLY DIRECTIVES

DESCRIPTION

The IAR Assemblers include a versatile range of directives for structured assembly, to make it easier to implement loops and control structures at assembly level.

The advantage of using the structured assembly directives is that the resulting programs are clearer, and their logic is easier to understand.

The directives are designed to generate simple, predictable code so that the resulting program is as efficient as if it were programmed by hand.

Conditional constructs

Use IF ... ENDIF to generate assembler source-code for comparison and jump instructions. The generated code is assembled like ordinary code, and is similar to macros. This should not be confused with conditional assembly.

IF blocks can be nested to any level.

The directive generates the statements:

```
CMP op2
JP cond, label
```

where *cond* is the inverted condition.

Use ELSE after an IF directive to introduce instructions to be executed if the IF condition is false.

Use ELSEIF to introduce a new condition after an IF directive.

Loop directives

Use WHILE ... ENDW to create a loop which is executed as long as the expression is TRUE. If the expression is false at the beginning of the loop the body will not be executed.

You can use BREAK to exit prematurely from a WHILE ... ENDW loop, and CONTINUE to continue with the next iteration of the loop.

STRUCTURED ASSEMBLY DIRECTIVES

The directive generates the statements:

```
CP op2
JP cond, label
```

where *cond* is the inverted *condition*.

Use the REPEAT ... UNTIL construct to create a loop with a body that is executed at least once, and as long as the expression is FALSE.

You can use BREAK to exit prematurely from a REPEAT ... UNTIL loop, or CONTINUE to continue with the next iteration of the loop.

The directive generates the statements:

```
CP op2
JP cond, label
```

where *cond* is the inverted *condition*.

Iteration construct

Use FOR to assemble instructions to repeat a block of instructions for a specified sequence of values.

FOR generates the following instructions:

```
LD    A,const1
JP    compare
body  ; Body of loop
inc   INC    A
      CP    const2
      JP    P,body
      JP    NZ,body
```

If DOWNT0 is specified instead of T0 the INC changes to DEC, and the test changes to JP M,body.

BREAK can be used to exit prematurely from a FOR loop, and continue execution at the instruction following the ENDF.

CONTINUE can be used to continue with the next iteration of the loop.

STRUCTURED ASSEMBLY DIRECTIVES

Switch construct

Use the SWITCH ... ENDS block to execute one of a number of sets of statements, depending on the value of a test.

CASE defines each of the tests, and DEFAULT introduces a CASE which is always true.

BREAK can be used to exit from a SWITCH ... ENDS block.

The following statements are generated:

```
CP op
JP NZ, label
```

EXAMPLES

Using conditional constructs

The following program example tests a memory location and plots 'N', 'Z', or 'P', depending on whether it is less than zero, zero, or greater than zero:

```
main    NAME    else
        EXTERN  plot
        IF      A < 0
        LD      B, 'N'
        ELSEIF  A = 0
        LD      B, 'Z'
        ELSE
        LD      B, 'P'
        ENDIF
        CALL    plot
        RET
        END     main
```

This generates the following code:

```
1  000000          NAME    else
2  000000          EXTERN  plot
3  000000          main    IF      A < 0
3.1 000000 FE00          CP      0
3.2 000002 F20D00       JP      P, _?0
```

STRUCTURED ASSEMBLY DIRECTIVES

```
3.3 000005 CA0D00      JP      Z,_?0
4   000008 064E      LD      B,'N'
5   00000A      ELSEIF  A = 0
5.1 00000A C31600      JP      _?1
5.2 00000D      _?0
5.3 00000D FE00      CP      0
5.4 00000F C21400      JP      NZ,_?2
6   000012 065A      LD      B,'Z'
7   000014      ELSE
7.1 000014      _?2
8   000014 0650      LD      B,'P'
9   000016      ENDIF
9.1 000016      _?1
10  000016 CD....      CALL    plot
11  000019 C9      RET
12  00001A
13  00001A      END      main
```

Using loop constructs

The following example uses a REPEAT ... UNTIL loop to reverse the order of bits in register A and put the result in register B:

```
NAME      repeat

reverse REPEAT
SRL      A
RL       B
UNTIL    A <> 0
RET

END
```

This generates the following code:

```
1   000000      NAME    repeat
2   000000
3   000000      reverse REPEAT
3.1 000000      _?0
4   000000 CB3F      SRL   A
5   000002 CB10      RL    B
```

STRUCTURED ASSEMBLY DIRECTIVES

```
6      000004          UNTIL  A <> 0
6.1    000004 FE00      CP      0
6.2    000006 CA0000    JP      Z, _?0
6.3    000009          _?1
7      000009 C9        RET
8      00000A
9      00000A          END
```

Using FOR ... NEXT

The following example uses a FOR block to output to a 256-byte buffer to a port:

```
NAME      for
EXTERN    port
play      FOR      A = 0 TO 255
          LD        (port),A
          ENDF
          RET

END
```

It generates the following code:

```
1      000000          NAME    for
2      000000          EXTERN   port
3      000000          play    FOR      A = 0 TO 255
3.1    000000 3E00      LD      A,0
3.2    000002 C30900    JP      _?1
3.3    000005          _?0
4      000005 32....    LD      (port),A
5      000008          ENDF
5.1    000008 3C        _?2    INC      A
5.2    000009 FEFF      _?1    CP      255
5.3    00000B F20500    JP      P, _?0
5.4    00000E C20500    JP      NZ, _?0
5.5    000011          _?3
6      000011 C9        RET
7      000012
8      000012          END
```

STRUCTURED ASSEMBLY DIRECTIVES

Using switch constructs

The following example uses a SWITCH ... ENDS block to test a character typed by a user. If the user types Y or y the program calls save and exit. If the user types N or n the program just calls exit. Any other response prints cancelled. It uses an external print routine to print an immediate string:

```
NAME      switch
EXTERN    print,save,exit

test      CALL    print
          DEFM     "Save changes? [y,n,q]:"

          SWITCH   A
          CASE     'Y'
          CASE     'y'
          CALL     save

          CASE     'N'
          CASE     'n'
          CALL     exit
          BREAK

          DEFAULT
          CALL     print
          DEFM     "Cancelled."

          ENDS

          END      test
```

This generates the following code:

```
1  000000          NAME      switch
2  000000          EXTERN    print,save,exit
3  000000
4  000000 CD....  test      CALL    print
5  000003 53617665      DEFM     "Save changes? [y,n,q]:"
6  00001B
```

STRUCTURED ASSEMBLY DIRECTIVES

```
7  00001B          SWITCH  A
8  00001B          CASE    'Y'
8.1 00001B FE59     CP      'Y'
8.2 00001D C22300   JP      NZ,_?1
9  000020          CASE    'y'
9.1 000020 C32800   JP      _?2
9.2 000023 FE79     _?1    CP      'y'
9.3 000025 C22E00   JP      NZ,_?3
9.4 000028         _?2
10 000028 CD....   CALL    save
11 00002B
12 00002B          CASE    'N'
12.1 00002B C33300   JP      _?4
12.2 00002E FE4E     _?3    CP      'N'
12.3 000030 C23600   JP      NZ,_?5
12.4 000033         _?4
13 000033          CASE    'n'
13.1 000033 C33800   JP      _?6
13.2 000036 FE6E     _?5    CP      'n'
13.3 000038 C24100   JP      NZ,_?7
13.4 00003B         _?6
14 00003B CD....   CALL    exit
15 00003E          BREAK
15.1 00003E C35000   JP      _?0
16 000041
17 000041          DEFAULT
17.1 000041         _?7
18 000041 CD....   CALL    print
19 000044 43616E63  DEFM    "Cancelled."
20 000050
21 000050          ENDS
21.1 000050         _?0
22 000050
23 000050          END     test
```

LISTING CONTROL DIRECTIVES

These directives provide control over the assembler listing.

LSTCND	Controls conditional assembly listing.
LSTCOD	Controls multi-line code listing.
LSTCYC	Controls the listing of cycle counts.
LSTEXP	Controls the listing of macro generated lines.
LSTMAC	Controls the listing of macro definitions.
LSTOUT	Controls assembly listing output.
LSTPAG	Controls the formatting of output into pages.
LSTREP	Controls the listing of lines generated by repeat directives.
LSTSAS	Controls structured assembly listing.
LSTXRF	Generates a cross reference table.
PAGSIZ	Sets the number of lines per page.
COL	Sets the number of columns per page.
PAGE	Generates a new page.
CYCLES	Sets the listed cycle count.
CYCMAX	Uses the maximum cycle count for each instruction.
CYCMEAN	Uses the mean cycle count for each instruction.
CYCMIN	Uses the minimum cycle count for each instruction.

SYNTAX

LSTCND{+ | -}
LSTCOD{+ | -}
LSTCYC{+ | -}
LSTEXP{+ | -}

LISTING CONTROL DIRECTIVES

LSTMAC{+ | -}

LSTOUT{+ | -}

LSTPAG{+ | -}

LSTREP{+ | -}

LSTSAS{+ | -}

LSTXRF{+ | -}

COL *columns*

PAGSIZ *lines*

PAGE

CYCLES *cycles*

PARAMETERS

columns An absolute expression in the range 80 to 132, default 132.

lines An absolute expression in the range 10 to 150.

cycles Value to which the listed cycle count is set.

DESCRIPTION

Turning the listing on or off

Use LSTOUT- to disable all list output except for error messages. This overrides all other list control directives.

The default is LSTOUT+, which lists the output (if a list file was specified).

Listing conditional code and strings

Use LSTCND+ to force the assembler to list source code only for the parts of the assembly that are not disabled by previous conditional .IF statements, .ELSE, or .ENDIF.

The default setting is LSTCND-, which lists all source lines.

Use LSTCOD+ to expand the listing of output code to more than one line if needed; ie long ASCII strings will produce several lines of list output.

LISTING CONTROL DIRECTIVES

The default setting is LSTCOD-, which lists only the first line of code for a source line; code generation is *not* affected.

Controlling the listing of macros

Use LSTEXP- to disable the listing of macro generated lines. The default is LSTEXP+, which lists all macro generated lines.

Use LSTMAC+ to list macro definitions. The default is LSTMAC-, which disables the listing of macro definitions.

Controlling the listing of generated lines

Use LSTREP- to turn off the listing of lines generated by REPT, REPTC, and REPTI directives.

The default is LSTREP+, which lists the generated lines.

Use LSTSAS- to disable listing of the assembly source produced by the directives for structured assembly.

The default is LSTSAS+, which lists assembly source produced by structured assembly directives.

Generating a cross reference table

Use LSTXRF+ to generate a cross reference table at the end of the assembly list for the current module. The table shows values and line numbers, and the type of the symbol.

The default is LSTXRF-, which does not give a cross reference table.

Formatting listed output

Use COL to set the number of columns per page of the assembly list. The default number of columns is 132.

Use PAGESIZ to set the number of printed lines per page of the assembly list. The default number of lines per page is 44.

Use LSTPAG+ to format the assembly output list into pages.

The default is LSTPAG-, which gives a continuous listing.

Use PAGE to generate a new page in the assembly listing if paging is active.

LISTING CONTROL DIRECTIVES

Counting cycles

Use LSTCYC+ to list cycle counts. The value displayed is the sum of processor clock cycles, and the sum can be reset to any value by the CYCLES directive.

Note that the cycle count shown assumes no caches or cycle overlap or other runtime dependencies.

Use CYCLES to set the count when calculating the cycle count in a piece of code.

The cycle count is set to 0 at the beginning of the listing.

The LSTCYC+ directive, or the command line -cC option, must be used in order to print the cycle count.

Use CYCMAX, CYCMEAN, or CYCMIN to use the maximum, mean, or minimum cycle counts for instructions that can have different cycle counts.

EXAMPLES

Turning the listing on or off

To disable the listing of a debugged section of program:

```
LSTOUT-  
; Debugged section  
  
LSTOUT+  
; Not yet debugged
```

Listing conditional code and strings

The following example shows how LSTCND+ hides a call to a subroutine that is disabled by an IF directive:

```
NAME      lstcnd  
EXTERN   print  
  
RSEG      prom  
debug    DEFL  0  
  
begin     .IF      debug  
          CALL     print  
          .ENDIF
```

LISTING CONTROL DIRECTIVES

```
                LSTCND+
begin2  .IF      debug
        CALL    print
        .ENDIF
        END      begin
```

This will produce the following listing:

```
1  000000                NAME  1stcnd
2  000000                EXTERN print
3  000000
4  000000                RSEG   prom
5  000000          debug  DEFL   0
6  000000
7  000000          begin  .IF      debug
8  000000                CALL    print
9  000000                .ENDIF
10 000000
11 000000                LSTCND+
12 000000          begin2 .IF      debug
14 000000                .ENDIF
15 000000                END      begin
```

The following example shows the effect of LSTCOD+ on the code generated by a DEFW directive:

```
1  000000                NAME  1stcod
2  000000 01000A00 table  DEFW   1,10,100,1000,10000
3  00000A
4  00000A                LSTCOD+
5  00000A 01000A00 table2 DEFW   1,10,100,1000,10000
   6400E803
   1027
6  000014                END
```

LISTING CONTROL DIRECTIVES

Controlling the listing of macros

The following example shows the effect of LSTMAC and LSTEXP:

```
NAME      lstmac

times2  MACRO    reg
        SLA      reg
        ENDM

        LSTMAC+
div2     MACRO    reg
        SRA      reg
        ENDM

begin    times2  A

        LSTEXP-
div2     B
RET

END      begin
```

This will produce the following output:

```
1      000000          NAME      lstmac
2      000000
6      000000
7      000000          LSTMAC+
8      000000          div2  MACRO    reg
9      000000          SRA      reg
10     000000          ENDM
11     000000
12     000000          begin  times2  A
12.1   000000 CB27      SLA      A
12.2   000002          ENDM
13     000002
14     000002          LSTEXP-
15     000002          div2     B
16     000004 C9       RET
```

LISTING CONTROL DIRECTIVES

```
17 000005
18 000005          END      begin
```

Controlling the listing of generated lines

The following example illustrates the effect of LSTSAS-:

```
NAME      lstsas

begin  IF      A < 7
      RL      A
      ENDIF

      LSTSAS-
      IF      A < 7
      RL      A
      ENDIF

      END      begin
```

This will generate the following listing:

```
1 000000          NAME      lstsas
2 000000
3 000000          begin  IF      A < 7
3.1 000000 FE07          CP      7
3.2 000002 F20A00        JP      P._?0
3.3 000005 CA0A00        JP      Z._?0
4 000008 CB17          RL      A
5 00000A          ENDIF
5.1 00000A          _?0
6 00000A
7 00000A          LSTSAS-
8 00000A          IF      A < 7
9 000012 CB17          RL      A
10 000014          ENDIF
11 000014
12 000014          END      begin
```

LISTING CONTROL DIRECTIVES

The following example illustrates the effect of LSTREP-:

```
NAME      tables

main      ; Generate table of powers of 3
calc      DEFL      1
          REPT      4
          DEFW      calc
calc      DEFL      calc * 3
          ENDR

LSTREP-
; Generate table of powers of 7
calc      DEFL      1
          REPT      4
          DEFW      calc
calc      DEFL      calc * 7
          ENDR

END      main
```

It generates the following listing:

```
1      000000      NAME      tables
2      000000
3      000000      main      ; Generate table of powers of 3
4      000001      calc      DEFL      1
5      000000      REPT      4
6      000000      DEFW      calc
7      000000      calc      DEFL      calc * 3
8      000000      ENDR
8.1    000000 0100      DEFW      calc
8.2    000003      calc      DEFL      calc * 3
8.3    000002 0300      DEFW      calc
8.4    000009      calc      DEFL      calc * 3
8.5    000004 0900      DEFW      calc
8.6    000018      calc      DEFL      calc * 3
8.7    000006 1800      DEFW      calc
8.8    000051      calc      DEFL      calc * 3
```

LISTING CONTROL DIRECTIVES

```
9      000008
10     000008          LSTREP-
11     000008          ; Generate table of powers of 7
12     000001      calc  DEFL    1
13     000008          REPT    4
14     000008          DEFW    calc
15     000008      calc  DEFL    calc * 7
16     000008          ENDR
17     000010
18     000010          END      main
```

Formatting listed output

The following example formats the output into pages of 66 lines each with 80 columns. The PAGE directive inserts a page break between modules:

```
PAGSIZ 66 ; Page size
COL 80
LSTPAG+
---
ENDMOD
PAGE
MODULE
---
```

Displaying cycle counts

The following example uses CYCLES 0 to calculate the cycle count for the main loop in a capture routine. The cycle count is displayed in the listing using LSTCYC+, and CYCMEAN ensures that the mean cycle count is calculated for each instruction. LSTXREF+ is used to give a cross reference listing:

```
1      000000          NAME    cycles
2      0 000000          LSTCYC+
3      0 000000          LSTXRF+
4      0 000000          EXTERN  ioport
5      0 000000
6      0 000000          RSEG    ram
7      0 000000      inbuf  DEFS    256
8      0 000100
9      0 000000          RSEG    prom
```

LISTING CONTROL DIRECTIVES

```

10      0 000000      CYCLES  0
11      0 000000      CYCMEAN
12      0 000000 21.... capture LD      HL,inbuf+256
13      1 000003 3A.... capt2  LD      A,(ioport)
14      2 000006 77      LD      (HL),A
15      4 000007 10FA      DJNZ    capt2
16      6 000009 C9      RET
17      9 00000A
18      9 00000A      END

```

Segment	Type	Mode

prom	UNTYPED	REL
ram	UNTYPED	REL

Label	Mode	Type	Segment	Value

-				
capt2	REL	CONST UNTYP.	prom	3
capture	REL	CONST UNTYP.	prom	0
inbuf	REL	CONST UNTYP.	ram	0
ioport	ABS	CONST EXT [000] UNTYP. __EXTERNS	Solved Extern	

C PREPROCESSOR DIRECTIVES

The following C-language preprocessor directives are available:

<code>#define</code>	Assigns a value to a label.
<code>#undef</code>	Undefines a label.
<code>#if</code>	Assembles instructions if a condition is true.
<code>#ifdef</code>	Assembles instructions if a symbol is defined.
<code>#ifndef</code>	Assembles instructions if a symbol is undefined.
<code>#else</code>	Assembles instructions if a condition is false.
<code>#endif</code>	Ends a <code>#if</code> , <code>#ifdef</code> , or <code>#ifndef</code> block.
<code>#include</code>	Includes a file.
<code>#error</code>	Generates an error.
<code>#message</code>	Displays a message.

SYNTAX

```
#define label text
#undef label
#if condition
#ifdef label
#ifndef label
#else
#endif
#include {"filename" | <filename>}
#error "message"
#message "message"
```

C PREPROCESSOR DIRECTIVES

PARAMETERS

<i>label</i>	Symbol to be defined, undefined, or tested.	
<i>text</i>	Value to be assigned.	
<i>condition</i>	One of the following:	
	An absolute expression	The expression must not contain forward or external references, and any non-zero value is considered as true.
	<i>string1=string2</i>	The condition is true if <i>string1</i> and <i>string2</i> have the same length and contents.
	<i>string1<>string2</i>	The condition is true if <i>string1</i> and <i>string2</i> have different length or contents.
<i>filename</i>	Name of file to be included.	
<i>message</i>	Text to be displayed.	

DESCRIPTION

Defining and undefining labels

Use `#define` to define a temporary label.

```
#define label value
```

is similar to:

```
label DEFL value
```

Use `#undef` to undefine a label; the effect is as if it had not been defined.

Conditional directives

Use the `#if ... #else ... #endif` directives to control the assembly process at assembly time. If the condition following the `#if` directive is not true, the subsequent instructions will not generate any code (ie it will not be assembled or syntax checked) until a `#endif` or `#else` directive is found.

All assembler directives (except for END), and file inclusion, may be disabled by the conditional directives. Each `#if` directive must be terminated by a `#endif` directive. The `#else` directive is optional, and if used, it must be inside a `#if ... #endif` block.

`#if ... #endif` and `#if ... #else ... #endif` blocks may be nested to any level.

Use `#ifdef` to assemble instructions up to the next `#else` or `#endif` directive only if a symbol is defined.

Use `#ifndef` to assemble instructions up to the next `#else` or `#endif` directive only if a symbol is undefined.

Including source files

Use `#include` to insert the contents of a file into the source file at a specified point.

Displaying errors and messages

Use `#error` to force the assembler to generate an error, such as in a user-defined test.

Use `#message` to display a message.

EXAMPLES

Using conditional directives

The following example defines a label `adjust`, and then uses the conditional directive `#ifdef` to use the value if it is defined. If it is not defined `#error` displays an error:

```
NAME      error
EXTERN   input,output
#define   offset  10

main      LD      A,(input)

#ifdef   offset
        ADD     A,offset
#else
#error   "No offset defined"
#endif
```

C PREPROCESSOR DIRECTIVES

```
#undef offset
      LD      (output),A
      RET

      END
```

Including a source file

The following example uses `#include` to include a file defining macros into the source file. For example, the following macros could be defined in `macros.s01`:

```
; General double-register exchange
exch  MACRO  reg1,reg2
      PUSH   reg1
      PUSH   reg2
      POP    reg1
      POP    reg2
      ENDM
```

The macro definitions can then be included, using `#include`, as in the following example.

```
      NAME    include
      ; Standard macro definitions
#include "macros.s01"
      ; Program
main   LD      DE,1234
      exch    AF,DE
      RET
      END     main
```

DATA DEFINITION OR ALLOCATION DIRECTIVES

These directives define temporary values or reserve memory.

DEFB	Defines a temporary byte value (8 bits).
DEFW	Defines a temporary word value (16 bits).
DEFA	Defines a temporary address value (24 bits).
DEFD	Defines a temporary double value (32 bits).
DEFM	Defines a temporary value (ASCII string).
DEFS	Reserves memory without initializing.

SYNTAX

DEFB *expr* [,*expr*] ...
DEFW *expr* [,*expr*] ...
DEFA *expr* [,*expr*] ...
DEFD *expr* [,*expr*] ...
DEFM *expr* [,*expr*] ...
DEFS *bytes*

PARAMETERS

<i>expr</i>	A valid absolute, relocatable, or external expression, or an ASCII string. ASCII strings will be zero filled to a multiple of the size.
<i>bytes</i>	An absolute expression specifying the number of bytes to be reserved.

DATA DEFINITION OR ALLOCATION DIRECTIVES

DESCRIPTION

Defining constant memory

Use DEFB, DEFW, DEFA, and DEFD to define constant bytes, words, addresses, and double words, respectively.

Defining strings

Use DEFM to define ASCII strings.

Allocating memory

Use DEFS to allocate space. The memory contents are not initialized in any way.

Note also DEFL, which defines a temporary value; see *Value assignment directives*, page 1-90.

EXAMPLES

Defining constant memory

The following example generates a lookup table of addresses to routines:

	NAME	table
table	DEFW	addsubr,subsubr,clrsubr
addsubr	ADD	A,B
	RET	
subsubr	SBC	A,B
	RET	
clrsubr	LD	A,0
	RET	
	END	

DATA DEFINITION OR ALLOCATION DIRECTIVES

Defining strings

To define a string:

```
message DEFM 'Please enter your name'
```

To include a single quote in a string enter it twice; for example:

```
error DEFM 'Don''t understand!'
```

Allocating memory

To reserve space for 0xA bytes:

```
table DEFS 0xA
```

ASSEMBLER CONTROL DIRECTIVES

These directives provide control over the operation of the assembler.

\$	Includes a file.
<i>/*comment*/</i>	C-style comment delimiter.
RADIX	Sets the default base.
CASEON	Enables case sensitivity.
CASEOFF	Disables case sensitivity.
OPT	Selects optimization options.
SFR	Creates a special function register label.
SFRTYPE	Creates a special function register label with specified attributes.

SYNTAX

\$filename

*/*comment*/*

RADIX *expr*

CASEON

CASEOFF

OPT *option*

SFR *register* = *value*

SFRTYPE *register attribute* [, *attribute*] = *value*

ASSEMBLER CONTROL DIRECTIVES

PARAMETERS

<i>filename</i>	Name of file to be included. The \$ character must be the first character on the line.										
<i>comment</i>	Comment ignored by the assembler.										
<i>expr</i>	Default base; default 10 (decimal).										
<i>option</i>	Either JPON to optimize jumps, or JPOFF (the default) to disable optimization.										
<i>register</i>	The special function register.										
<i>attribute</i>	One or more of the following: <table><tr><td>READ</td><td>You can read from this SFR.</td></tr><tr><td>WRITE</td><td>You can write to this SFR.</td></tr><tr><td>BYTE</td><td>The SFR must be accessed as a byte.</td></tr><tr><td>WORD</td><td>The SFR must be accessed as a word.</td></tr><tr><td>LONG</td><td>The SFR must be accessed as a long.</td></tr></table>	READ	You can read from this SFR.	WRITE	You can write to this SFR.	BYTE	The SFR must be accessed as a byte.	WORD	The SFR must be accessed as a word.	LONG	The SFR must be accessed as a long.
READ	You can read from this SFR.										
WRITE	You can write to this SFR.										
BYTE	The SFR must be accessed as a byte.										
WORD	The SFR must be accessed as a word.										
LONG	The SFR must be accessed as a long.										
<i>value</i>	The SFR value.										

DESCRIPTION

Use \$ to insert the contents of a file into the source file at a specified point.

Use /* ... */ to comment sections of the assembler listing.

Use RADIX to set the default base for use in conversion of constants from ASCII source to the internal binary format.

To reset the base from 16 to 10 *expr* must be written in hexadecimal. For example:

RADIX 0A

Controlling case sensitivity

Use CASEON or CASEOFF to turn on or off case sensitivity for user-defined symbols. By default case sensitivity is off.

ASSEMBLER CONTROL DIRECTIVES

When CASEOFF is active all symbols are stored in upper case, and all symbols used by XLINK should be written in upper case in the XLINK definition file.

Optimizing jump instructions

Use OPT JPON to optimize jump instructions, by automatically generating a two-byte JR instruction instead of a three-byte JP instruction when the address is within range.

Use OPT JPOFF to disable optimization. Optimization is off by default.

Defining special function registers

Use SFR to create special function register labels with all attributes turned on, or SFRTYPE to create special function register labels with specified attributes.

EXAMPLES

Including a source file

The following example uses \$ to include a file defining macros into the source file. For example, the following macros could be defined in macros.s01:

```
; General double-register exchange
exch    MACRO    reg1,reg2
        PUSH     reg1
        PUSH     reg2
        POP      reg1
        POP      reg2
        ENDM
```

The macro definitions can be included with a \$ directive, as in:

```
NAME    include
        ; Standard macro definitions
$macros.s01
        ; Program
main    LD      DE,1234
        exch    AF,DE
        RET
        END     main
```

ASSEMBLER CONTROL DIRECTIVES

Defining comments

The following example shows how `/* ... */` can be used for a multi-line comment:

```
/*  
Program to read serial input.  
Version 2: 19.4.93  
Author: mjp  
*/
```

Changing the base

To set the default base to 16:

```
RADIX 16  
LD    A,12
```

The immediate argument will then be interpreted as H'12.

Controlling case sensitivity

By default CASEOFF is active, so in the following example `label` and `LABEL` are identical:

```
label  NOP      ;stored as "LABEL"  
      JP      LABEL
```

However, the following will generate a duplicate label error:

```
label  NOP  
      CASEON  
LABEL  NOP      ;Error: "LABEL" already defined  
      END
```

Optimizing jump instructions

The following example illustrates how the assembler optimizes jump instructions:

```
      NAME    optim  
      ORG     100h  
      ; Jump optimization off by default  
long  JP      short2  
short JP      long2
```

ASSEMBLER CONTROL DIRECTIVES

```
        ORG      110h
short2  OPT      JPON      ; Optimize jumps
        JR       short
        JP       short

        ORG      1000h
long2   JP       long
        JR       long
        END
```

This generates the following code:

```
1  000100          NAME    optim
2  000100          ORG     100h
3  000100          ; Jump optimization off by default
4  000100 C31001   long    JP     short2
5  000103 C30010   short   JP     long2
6  000106
7  000110          ORG     110h
8  000110          OPT     JPON      ; Optimize jumps
9  000110 18F1     short2  JR     short
10 000112 18EF          JP     short
11 000114
12 001000          ORG     1000h
13 001000 C30001   long2   JP     long
14 001003 C30001   JR     long
15 001006          END
```

Defining special function registers

The following statements define P01 with any access, and P0 and P1 with byte read/write access:

```
SFR      P01          = 02h      ; port 0 & 1
SFRTYPE  P0   byte,read,write = 02h      ; port 0
SFRTYPE  P1   byte,read,write = 03h      ; port 1
```

The following instruction is illegal since BYTE access is not used, and the assembler will give a warning:

```
LD        P0,HL
```

ASSEMBLER MNEMONICS

This chapter lists the Z80 and 64180 instruction mnemonics.

ADDRESSING MODES

Instructions can contain operands that are addressed in the following modes:

- ◆ Register addressing
- ◆ Inherent addressing
- ◆ Immediate addressing
- ◆ Relative addressing
- ◆ Indexed addressing
- ◆ Register indirect addressing
- ◆ Extended addressing
- ◆ Bit addressing
- ◆ Restart addressing

These addressing modes are described below.

REGISTER ADDRESSING

In this mode, the operand is either a register or register pair. For example, the load instruction LD B, A moves data from register A to register B using register addressing.

Instructions that use register addressing can be 1, 2, 3, or 4 bytes in length.

ASSEMBLER MNEMONICS

INHERENT ADDRESSING

In this mode, the operand is either a status register, control bit, or the A register. For example, the RRA instruction inherently implies the register to be rotated is the A register.

These are 1 byte instructions.

IMMEDIATE ADDRESSING

In this mode, the operand is contained in the instruction following the opcode. Depending upon the instruction, a 1 or 2 byte immediate value follows the opcode. An error is generated for those instructions requiring a 1 byte immediate value and whose immediate value is greater than 256.

These instructions are 2 or 3 bytes in length

RELATIVE ADDRESSING

Only relative branch instructions (JR and DJNZ) use relative addressing. If the branch condition is true, the Z80 alters the program counter (PC) as follows:

$$PC = PC + x$$

where x is the 8 bit displacement in the instruction, sign extended to 16 bits. The value of x is in the range -128 to 127.

PC initially points to the start of the next instruction, two bytes away. Thus, the branch range is -126 to 129 bytes from the first byte of the instruction.

The operand of a relative address is supplied in the following way:

DJNZ MAIN

INDEXED ADDRESSING

Indexed instructions contain a 1 byte unsigned offset in the third byte of the instruction. The unsigned offset is added with carry to one of the two index registers IX or IY, and references memory. The index register is not changed. These are 3-byte instructions.

REGISTER INDIRECT ADDRESSING

This mode of addressing uses a 2 byte register pair, such as HL, as a pointer to an operand at any location in the 64 Kbyte memory space. This address mode is indicated by parentheses surrounding the register pair. An example of this instruction is LD A, (BC), which loads register A with the contents of the memory location pointed to by the register pair BC.

EXTENDED ADDRESSING

Extended addressing uses a 2-byte operand that follows the instruction opcode to directly reference a memory location. Extended addresses can address any location in the 64 Kbyte memory space.

BIT ADDRESSING

Instructions that allow bit addressing mode have two operands. The first operand specifies the bit number. The second operand is a memory location or register that contains the data for the bit addressing mode instruction. The data can be addressed with register, register indirect, or indexed addressing.

RESTART ADDRESSING

There is a special single byte call instruction that makes a call to one of eight addresses contained in low memory. This instruction is called a restart instruction. Memory space is saved whenever a single byte restart is used in place of a 3-byte call instruction.

INSTRUCTION MNEMONICS

The following symbols are used in the list of instruction mnemonics:

<i>Symbol</i>	<i>What it means</i>
n	Immediate 8-bit data.
nn	Immediate 16-bit data.
d	8-bit unsigned displacement.
e	8-bit signed displacement.
(nn)	16-bit address.
b	Bit number, 0 to 7.
PC	Program counter.
C	Carry flag.
V	Overflow flag
Z	Zero flag.
N	Negative flag.
H	Half-carry flag.
I	Interrupt control vector register.
*	Undocumented instruction; the -uu command line option must be used to interpret these instructions correctly.

8-BIT DATA LOAD

	<i>First operand (target)</i>	<i>Second operand (source)</i>
LD	A	(BC), (DE), I
	A, B, C, D, E, H, L	A, B, C, D, E, H, L, n, (HL), (IX+d), (IY+d), (nn)
	(BC), (DE), I, (nn)	A
	A, B, C, D, E, IXL*, IXH*	IXL*, IXH*
	IXL*	A, B, C, D, E
	IXH*	A, B, C, D, E

16-BIT DATA LOAD

	<i>First operand (target)</i>	<i>Second operand (source)</i>
LD	BC, DE, HL, IX, IY, SP	(nn), nn
	(nn)	BC, DE, SP
	SP	HL, IX, IY
POP, PUSH	AF, BC, DE, HL, IX, IY	

ASSEMBLER MNEMONICS

EXCHANGE

	<i>First operand</i>	<i>Second operand</i>
EX	AF	AF'
	DE	HL
EXX		

BLOCK TRANSFER/BLOCK SEARCH

	<i>First operand</i>	<i>Second operand</i>
LDI, LDIR, LDD, LDDR, CPI, CPIR, CPD, CPDR		

8-BIT ARITHMETIC/LOGICAL

	<i>First operand</i>	<i>Second operand</i>
ADD, SUB, ADC, SBC	A, B, C, D, E, H, L, n, (HL), (IX+d), (IY+d), IXL*, IXH*, IYL*, IYH*	
AND, OR, XOR, CP	A, B, C, D, E, H, L, n, (HL), (IX+d), (IY+d), IXL*, IXH*, IYL*, IYH*	
INC, DEC	A, B, C, D, E, H, L, (HL), (IX+d), (IY+d), IXL*, IXH*, IYL*, IYH*	

GENERAL ARITHMETIC AND MPU CONTROL

	<i>First operand</i>	<i>Second operand</i>
DAA, CPL, NEG, CCF, SCF, NOP, HALT, DI, EI		
IM	0, 1, 2	

16-BIT ARITHMETIC

	<i>First operand</i>	<i>Second operand</i>
ADD, SUB	HL	BC, DE, HL, SP
	IX	BC, DE, IX, SP
	IY	BC, DE, IY, SP
ADC, SBC	HL	BC, DE, HL, SP
INC, DEC	BC, DE, HL, IX, IY, SP	

ROTATE/SHIFT

	<i>First operand</i>	<i>Second operand</i>
RL, RLC, RR, RRC, SLA, SRA, SRL	A, B, C, D, E, H, L, (HL), (IX+d), (IY+d)	
RLCA, RLA, RRCA, RRA, RLD, RRD		

ASSEMBLER MNEMONICS

BIT SET/RESET/TEST

	<i>First operand</i>	<i>Second operand</i>
BIT, SET, RES	b	A, B, C, D, E, H, L, (HL), (IX+d), (IY+d)

JUMP

	<i>First operand</i>	<i>Second operand</i>
JP	nn, (HL), (IX), (IY)	
JP	NZ, Z, NC, C, PO, PE, P, N	nn
JR	C, NC, Z, NZ	e
DJNZ	e	

CALL AND RETURN

	<i>First operand</i>	<i>Second operand</i>
CALL	nn	
CALL	NZ, Z, NC, C, PO, PE, P, N	nn
RET, RETI, RETN		
RET	NZ, Z, NC, C, PO, PE, P, N	
RST	00H, 08H, 10H, 18H, 20H, 28H, 30H, 38H	

INPUT AND OUTPUT

	<i>First operand</i>	<i>Second operand</i>
IN	A	(n)
IN	A, B, C, D, E, H, L	(C)
IN	(C)	
OUT	(n)	A
OUT	(C)	A, B, C, D, E, H, L
INI, INIR, IND, INDR, OUTI, OUTR, OUTD, OTDR		

ADDITIONAL 64180 INSTRUCTIONS

These additional instructions are valid if the -v1 command line option has been specified.

MULTIPLY

	<i>First operand</i>	<i>Second operand</i>
MLT	BC, DE, HL, SP	

TEST

	<i>First operand</i>	<i>Second operand</i>
TST	A, B, C, D, E, H, L, n, (HL)	
TSTIO	n	

INPUT, OUTPUT, AND SLEEP

	<i>First operand</i>	<i>Second operand</i>
INO	A, B, C, D, E, H, L	(n)
INO	(n)	
OUTO	(n)	A, B, C, D, E, H, L
OTDM, OTDMR, OTIM, OTIMR, SLP		

ASSEMBLER DIAGNOSTICS

This chapter lists the errors and warnings for the Z80 Assembler. For details of the XLINK Linker and XLIB Librarian error messages see *XLINK and XLIB diagnostics* in the *IAR Linker & Librarian* guide.

INTRODUCTION

Error messages are printed on the terminal, as well as on the optional list file. Error messages are as few as possible and errors are reported as soon as possible.

All errors are issued as complete, self-explanatory messages. For example:

```
      ADD    B,C
-----^
"testfile.s20",4  Error[40]: bad instruction
```

The error message consists of the erroneous source line, with a pointer to the faulty spot, followed by the diagnostic and source line number. If include files are used, error messages will be preceded by the source line number and name of *current* file:

```
      ADD    B,C
-----^
"subfile.h",4  Error[40]: bad instruction
```

The error messages produced by the assembler fall into six categories:

- ◆ Command line errors.
- ◆ Assembly warning messages.
- ◆ Assembly error messages.
- ◆ Assembly fatal error messages.

ASSEMBLER DIAGNOSTICS

- ◆ Memory overflow messages.
- ◆ Assembler internal errors.

COMMAND LINE ERRORS

Command line errors occur when the assembler is invoked with bad parameters. The most common situation is when a file cannot be opened, or with duplicate, mis-spelled, or missing command line switches. The messages are self-explanatory.

ASSEMBLY WARNING MESSAGES

Assembly warning messages are produced when the assembler has found a construct which probably is due to a programming error or omission. These are listed in the section *Warning messages*, page 1-147.

ASSEMBLY ERROR MESSAGES

Assembly error messages are produced when the assembler has found a construct which violates the language rules. These are listed in the section *Error messages*, page 1-150.

ASSEMBLY FATAL ERRORS

Assembly fatal error messages are produced when the assembler has found a user error so severe that further processing is not considered meaningful. After the diagnostic message has been issued the assembly is immediately terminated. The fatal error messages are identified as 'Fatal' in the error messages list.

MEMORY OVERFLOW MESSAGE

The assembler is a memory-based program that in the case of a system with a small primary memory or in the case of very large source files may run out of memory. This is identified by the special message:

```
* * * ASSEMBLER OUT OF MEMORY * * *
```

```
Dynamic memory used: nnnnnn bytes
```

If such a situation occurs the solution is either to add system memory or to split source files into smaller modules. However, with 1 Mbyte RAM the assembler capacity should be sufficient for all reasonably sized source files.

ASSEMBLER INTERNAL ERRORS

During assembly a number of internal consistency checks are performed and if any of these checks fail the assembler will terminate after giving a short description of the problem. Such errors should normally not occur and should be reported to the IAR Systems technical support group. Please include all possible information about the problem and, preferably, a disk containing a copy of the program that generated the internal error.

WARNING MESSAGES

GENERAL

<i>No</i>	<i>Warning message</i>	<i>Suggestion</i>
0	Unreferenced label	The label was not used as an operand nor was it declared public.
1	Nested comment	A C comment was nested.
2	Unknown escape sequence	A backslash (\) found in a character constant or string literal was followed by an unknown escape character.
3	Non-printable character	A non-printable character was found in a literal or character constant.
4	Macro or define expected	
5	Floating point value out-of-range	Floating point value is too large to be represented by the floating point system of the target.

ASSEMBLER DIAGNOSTICS

<i>No</i>	<i>Warning message</i>	<i>Suggestion</i>
6	Floating point division by zero	
8	Macro parameter(s) not used	
9	Macro redefined	
10	Unknown macro	
11	Empty macro argument	
12	Recursive macro	
13	Redefinition of Special Function Register	The SFR has already been defined.
14	Division by zero	Division by 0 in constant expression.
15	Constant truncated	The constant was longer than the size of the destination.
16	SFR not read or writeable	The SFRTYPE directive was used in such a way that the Special Function Register is inaccessible.
17	Suspicious sfr expression	A Special Function Register SFR is used in an expression, and the assembler can not check access rights.
18	Empty module '<module name>', module skipped	An empty module was created by using END directly after ENDMOD or MODULE, followed by ENDMOD with no statements in between.
19	End of program while in include file	The program ended while a file was being included.
20	Symbol '<symb>' duplicated	

ASSEMBLER DIAGNOSTICS

<i>No</i>	<i>Warning message</i>	<i>Suggestion</i>
21	Bit symbol cannot be used as operand	A symbol was declared using the bit directive, but since the bit address is not calculated the symbol should not be used.

Z80 ASSEMBLER

The following warnings are specific to the Z80:

<i>No</i>	<i>Warning message</i>	<i>Suggestion</i>
400	External untyped label, using the largest addressing mode possible	
401	Untyped label, using the largest addressing mode possible	
402	Number out of range	Immediate data too large. Check syntax for instruction.
403	SFR not expected here	Check syntax for instruction.
404	SFR not accessible here	SFR defined without read or write attribute using SFRTYPE directive.
405	No size defined for SFR	SFR defined without size attribute using SFRTYPE directive.
406	More than one size defined for SFR	SFR defined with more than one size attribute using SFRTYPE directive.
407	Jump out of segment	A short jump has been made out of the segment; this may cause an error when the range check is performed at link time.

ERROR MESSAGES

GENERAL

<i>No</i>	<i>Error message</i>	<i>Suggestion</i>
0	Invalid syntax	The assembler could not decode the expression.
1	Too deep #include nesting (max. is 10)	Fatal. Assembler limit for nesting of #include files exceeded. Recursive #include could be the reason.
2	Failed to open #include file 'name'	Fatal. Could not open a #include file. File does not exist in specified directories. Check -I prefixes.
3	Invalid #include file name	Fatal. #include file name must be written <file> or "file".
4	Unexpected end of file encountered	Fatal. End of file encountered within a conditional assembly, the repeat directive or during macro expansion. Probable cause is a missing end of conditional assembly etc.
5	Too long source line (max. is 512 characters) truncated	Source line length exceeds assembler limit.
6	Bad constant	Character that is not a legal digit was encountered.
7	Hexadecimal constant without digits	Prefix 0x or 0X of hexadecimal constant found without following hexadecimal digits.

ASSEMBLER DIAGNOSTICS

<i>No</i>	<i>Error message</i>	<i>Suggestion</i>
8	Invalid floating point constant	Too large or invalid syntax of floating-point constant.
9	Too many errors encountered (>100).	
10	Space or tab expected	
11	Too deep block nesting (max is 50)	Preprocessor directives are nested too deep.
12	String too long (max is 509)	Assembler string length limit exceeded.
13	Missing delimiter in literal or character constant	No closing delimiter ' or " was found in character or literal constant.
14	Missing #endif	A #if, #ifdef, or #ifndef was found but had no matching #endif.
15	Invalid character encountered: <char>; ignored	
16	Identifier expected	A name of a label or symbol was expected.
17	')' expected	
18	No such pre-processor command: <command>	# was followed by an unknown identifier.
19	Unexpected token found in pre-processor line	The pre-processor line was not empty after the argument part was read.
20	Argument to #define too long (max is <max>)	
21	Too many formal parameters for #define (max is 127)	

ASSEMBLER DIAGNOSTICS

<i>No</i>	<i>Error message</i>	<i>Suggestion</i>
22	Macro parameter <parameter> redefined	A #define symbol's formal parameter was repeated.
23	',' or ')' expected	
24	Unmatched #else, #endif or #elif	Fatal. Missing #if, #ifdef, or #ifndef.
25	#error <error>.	Printout via the #error directive.
26	(' expected	
27	Too many active macro parameters (max is 256)	Fatal. Pre-processor limit exceeded.
28	Too many nested parameterized macros (max is <max>)	Fatal. Pre-processor limit exceeded.
29	Too deep macro nesting (max is 100)	Fatal. Pre-processor limit exceeded.
30	Actual macro parameter too long (max is 512)	A single macro (in #define) argument may not exceed the length of a source line.
31	Macro <macro> called with too many parameters	The number of parameters used was more than the number in the macro declaration.
32	Macro <macro> called with too few parameters	The number of parameters used was less than the number in the macro declaration (#define).
33	too many MACRO arguments	The number of assembler macros exceed 32.
34	may not be redefined	Assembler macros may not be redefined.

ASSEMBLER DIAGNOSTICS

<i>No</i>	<i>Error message</i>	<i>Suggestion</i>
35	no name on macro	Assembler macro definition without a label was encountered.
36	Illegal formal parameter in macro	A parameter that was not an identifier was found.
37	ENDM or EXITM not in macro	ENDM directive or EXITM directive encountered while not inside macro.
38	'>' expected but found end-of-line	A < was found but no matching >.
39	END before start of module	End-of-module directive has no matching MODULE directive.
40	bad instruction	The mnemonic/directive does not exist.
41	bad label	Labels must begin with A-Z, a-z, _, or ?. The succeeding characters must be A-Z, a-z, 0-9, _, or ?. Labels cannot have the same name as a predefined symbol.
42	duplicate label	The label has already appeared in the label field or been declared as EXTERN.
43	illegal effective address	The addressing mode (operands) is not allowed for this mnemonic.
44	',' expected	A comma was expected but not found.
45	name duplicated	The name of RSEG, STACK, or COMMON segment is already used but for something else.

ASSEMBLER DIAGNOSTICS

<i>No</i>	<i>Error message</i>	<i>Suggestion</i>
46	segment type expected	In RSEG, STACK, or COMMON directive : was found but the segment type that should follow was not valid.
47	segment name expected	The RSEG, STACK, and COMMON directives need a name.
48	value out of range '<range>'	The value exceeds its limits.
49	alignment already set	RSEG, STACK, and COMMON segment do not allow alignment to be set more than once. Use ALIGN, EVEN, or ODD instead.
50	undefined symbol:<symbol>	The symbol did not appear in label field nor in an EXTERN or sfr declaration.
51	Can't be both PUBLIC and EXTERN	Symbols can be declared as either PUBLIC or EXTERN.
52	EXTERN not allowed	Reference to EXTERN symbols is not allowed in this context.
53	expression must be absolute	The expression cannot involve relocatable or external symbols.
54	expression can not be forward	The assembler must be able to solve the expression the first time this expression is encountered.
55	illegal size	The maximum size for expressions is 32 bits.
56	too many digits	The value exceeds the size of the destination.
57	unbalanced conditional assembly directives	Missing conditional assembly IF or ENDIF.

ASSEMBLER DIAGNOSTICS

<i>No</i>	<i>Error message</i>	<i>Suggestion</i>
58	ELSE without IF	Missing conditional assembly IF.
59	ENDIF without IF	Missing conditional assembly IF.
60	unbalanced structured assembly directives	Missing structured assembly IF or ENDIF.
61	'+' or '-' expected	Plus or minus sign missing.
62	Illegal operation on extern or public symbol	An illegal operation has been used on a public or external symbol; eg SET.
63	Illegal operation on non-constant label	It is not allowed to make a non-constant symbol PUBLIC or EXTERN.
64	Extern or unsolved expression	The expression must be solved at assembly time, ie not include external references.
65	'=' expected	Equals sign was missing.
66	Segment too long (max is <max>)	The length of ASEG, RSEG, STACK, or COMMON segments is larger than the addressable length.
67	Public did not appear in label field	A symbol was declared PUBLIC but no label with the same name was found in the source file.
68	End of block-repeat without start	The repeat directive REPT was not found although the ENDR directive was.
69	Segment must be relocatable	The operation is not allowed on ASEG.

ASSEMBLER DIAGNOSTICS

<i>No</i>	<i>Error message</i>	<i>Suggestion</i>
70	Limit exceeded: <error text>, value is: <value> (decimal)	The value exceeded the limits set with the LIMIT directive. The error text is set by the user in the LIMIT directive.
71	Symbol '<symbol>' has already been declared EXTERN	An attempt to redeclare a EXTERN as EXTERN was made.
72	Symbol '<symbol>' has already been declared PUBLIC	An attempt to redeclare a PUBLIC as PUBLIC was made.
73	End-of-module missing	A PROGRAM or MODULE directive was encountered before ENDMOD was found.
74	Expression must yield non-negative result	The expression was evaluated to a negative number, whereas a positive number was required.
75	Repeat directive unbalanced	This error is caused by a REPT directive without a matching ENDR, or a an ENDR directive without a matching REPT.
76	End of repeat directive is missing	A REPT directive without a closing ENDR was encountered.
77	LOCALs not allowed in this context. (<symbol>)	Local symbols must be declared within macro definitions.
78	End of macro expected	An assembler macro is being defined but there was no end-of-macro.
79	End of repeat expected	One of the repeat directives is active, but there was no end-of-repeat found.

ASSEMBLER DIAGNOSTICS

<i>No</i>	<i>Error message</i>	<i>Suggestion</i>
80	End of conditional assembly expected	Conditional assembly is active but there was no end of if.
81	End of structured assembly expected	One of the directives for structured assembly is active but has no matching END.
82	Misplaced end of structured assembly	A directive that terminates one of the structured assembly directives was found but no matching START directive is active.

Z80 ASSEMBLER

In addition to the general warnings, the Z80 Assembler can generate the following errors:

<i>No</i>	<i>Error message</i>	<i>Suggestion</i>
400	ELSEIF with no matching IF	Check use of structured assembly directives.
401	ELSE used more than once	Check use of structured assembly directives.
402	ELSE without matching IF	Check use of structured assembly directives.
403	THEN without matching IF	Check use of structured assembly directives.
404	ENDW without matching WHILE	Check use of structured assembly directives.
405	UNTIL without matching REPEAT	Check use of structured assembly directives.
406	Too large or negative step value	Use DOWNT0 or smaller STEP in FOR loop.

ASSEMBLER DIAGNOSTICS

<i>No</i>	<i>Error message</i>	<i>Suggestion</i>
407	ENDF without matching FOR	Check use of structured assembly directives.
408	Nothing to BREAK out of	BREAK can only be used inside structured assembly constructions.
409	Nothing to CONTINUE to	CONTINUE can only be used inside structured assembly directives.
410	CASE after DEFAULT	Move DEFAULT after the last CASE label.
411	CASE outside SWITCH	CASE can only be used inside SWITCH statements.
412	Multiple DEFAULT	Only one DEFAULT label is legal in a SWITCH statement.
413	DEFAULT outside SWITCH	DEFAULT can only be used inside SWITCH statements.
414	ENDS without matching SWITCH	Check use of structured assembly directives.
415	Cannot solve value	Use absolute STEP value in FOR loop.
416	Cannot break that number of times	Argument to BREAK is greater than the number of loops to break out of.
417	Unterminated C-comment	Check your C comments /* comment */.
418	Jump too long	Jump can only be 256 bytes long.
419	Accessing SFR using incorrect size	Check size of SFR defined with SFRTYPE directive.

ASSEMBLER DIAGNOSTICS

<i>No</i>	<i>Error message</i>	<i>Suggestion</i>
420	Accessing SFR incorrectly with respect to read/write flags of SFR	Check read/write attributes of SFR defined with SFRTYPE directive.
421	'=' or '!' = 'expected	
422	ELSEIF cannot be used after ELSE	Check use of structured assembly directives.
423	Operand must be absolute	Relocatable operands are not allowed here.
424	Only register IX and IY allowed	Indirect addressing with offset only allowed with IX and IY registers.
425	The option is not valid	
426	Wrong number of arguments for this instruction	Check syntax for instruction.
427	Wrong type of addressing mode	Check syntax for instruction.
428	Register not allowed in this instruction	Check syntax for instruction.
429	Only register HL allowed in this operation	Check syntax for instruction.
430	Argument must be 0,1 or 2	Check syntax for instruction.
431	Wrong value of restart address	Check syntax for instruction.
432	Bit value not between 0 and 7	Check syntax for instruction.

ASSEMBLER DIAGNOSTICS

IAR LINKER & LIBRARIAN

INTRODUCTION

XLINK LINKER

The IAR XLINK Linker converts simple assembler programs to hex code, and can link high language modules together in a multi-segmented system. It reads one or more relocatable object files produced by the IAR Assembler or IAR C Compiler, and produces machine code for a specified target processor. It supports a wide range of industry standard loader formats, in addition to the IAR debug format used by the C-SPY high level debugger.

XLINK performs two distinct functions during program linking. It selectively loads modules containing executable code or data from the input files (it locates each segment of code or data at a user-specified address) and it links the various modules together by resolving all global symbols that could not be resolved by the assembler or compiler.

XLINK supports user libraries, and will load only those modules that are actually needed by the program you are linking.

The final output produced by XLINK is an absolute, target executable object file that can be programmed into a EPROM, down-loaded to a hardware emulator, or run directly on the host PC using the IAR C-SPY debugger.

Through its support for over thirty different symbolic and non-symbolic object file formats, programs produced by XLINK can be used by a wide variety of hardware emulators, software simulators, target boards, and EPROM programmers. XLINK can also produce a symbol cross-reference and segment/module map file, to help during software debugging and documentation.

INTRODUCTION

FEATURES OF XLINK

XLINK offers the following important features and characteristics:

- ◆ Unlimited number of input files.
- ◆ Fast memory-based linking, or optional disk-based operation for programs with large amounts of code, data, or symbols.
- ◆ Linker commands can be entered on the XLINK command line, read from an extended command file, or a combination of the two, making XLINK easy and flexible to use.
- ◆ Searches user-defined library files and loads only those modules needed by the application.
- ◆ Symbols may be up to 255 characters long with all characters being significant. Both upper and lower case may be used.
- ◆ Global symbols can be defined at link time.
- ◆ Flexible segment commands allow full control of the locations of relocatable code and data in memory.
- ◆ Supports bank-switched code with C and assembler programs.

XLIB LIBRARIAN

The IAR XLIB Librarian is a program that enables you to manipulate the relocatable object files produced by the IAR Assembler and IAR C Compiler.

FEATURES OF XLIB

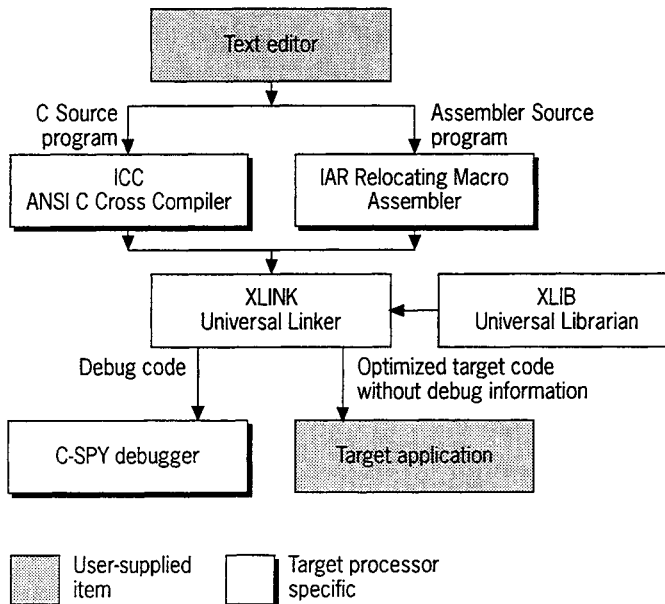
XLIB provides the following features:

- ◆ Support for modular programming.
- ◆ Modules can be listed, added, inserted, replaced, deleted, or renamed.
- ◆ Segments can be listed and renamed.
- ◆ Symbols can be listed and renamed.

- ◆ A module can be changed from program to library type, and vice versa.
- ◆ Interactive or batch mode operation.
- ◆ A full set of library listing operations.
- ◆ A command to display a directory of relevant files on disk.
- ◆ On-line help.

OVERVIEW

The following diagram shows how the IAR Assembler is used in conjunction with the XLINK Linker, XLIB Librarian, and C-SPY debugger:



For more information refer to the appropriate *IAR Assembler* user guide for the target processor you are using.

INTRODUCTION

For more information about the IAR C Compiler refer to the *IAR C Compiler* guide for the target processor you are using.

SYSTEM REQUIREMENTS

The IAR Assembler, XLINK Linker, and XLIB Librarian are available in versions for three different host computer systems: IBM PC compatibles, UNIX, or VAX/VMS. The versions supplied for the IBM PC provide default support for up to 16 Mbytes of extended memory.

IBM PC

Operating system:	Minimum MS-DOS 4.01.
Memory:	1 Mbyte minimum, 4 Mbytes recommended. Extended memory is supported up to 16 Mbytes.
Screens:	Monochrome, CGA, EGA, or VGA.

UNIX, VAX/VMS

Operating system:	No special requirements.
-------------------	--------------------------

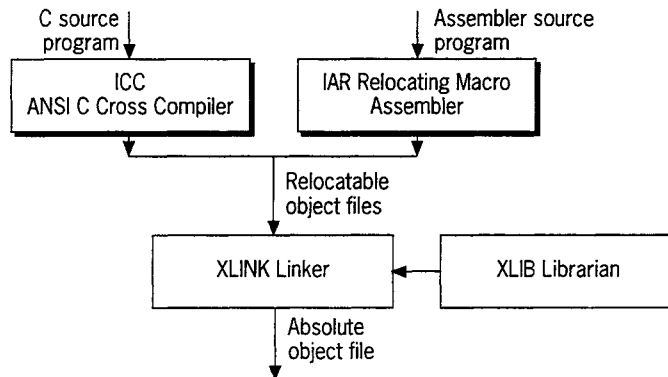
XLINK LINKER

This chapter describes the IAR XLINK Linker, and gives examples of how it can be used.

INTRODUCTION

The IAR XLINK Linker is a powerful, flexible software tool for use in the development of embedded-controller applications. XLINK reads one or more relocatable object files produced by the IAR Assembler or C Compiler and produces absolute, machine-code program as output. It is equally well-suited for linking small, single-file, absolute assembler programs as it is for linking large, relocatable, multi-module, mixed C and assembler programs.

The following diagram illustrates the linking process:



XLINK LINKER

OBJECT FORMAT

The object files produced by the IAR Assembler and C Compiler use an IAR proprietary format called UBROF, which stands for Universal Binary Relocatable Object Format. An application can be made up of any number of UBROF relocatable files, in any combination of assembler and C.

XLINK FUNCTIONS

XLINK performs three distinct functions when you link a program:

- ◆ It loads modules containing executable code or data from the input file(s).
- ◆ It locates each segment of code or data at a user-specified address.
- ◆ It links the various modules together by resolving all global (ie non-local, program-wide) symbols that could not be resolved by the assembler or compiler.
- ◆ It loads modules needed by the program from user-defined libraries.

LIBRARIES

When XLINK reads a library file (which can contain multiple C or assembler modules) it will only load those modules which are actually needed by the program you are linking. This avoids having to load all the modules in a library file when you only need one routine. The XLIB Librarian is used to manage these library files.

OUTPUT FORMAT

The final output produced by XLINK is an absolute, executable object file that can be put into an EPROM, downloaded to a hardware emulator, or executed on the PC using the IAR C-SPY debugger.

FILE NAMING CONVENTIONS

The XLINK Linker uses the following default naming conventions for the various files that it processes, where *xx* is a target-specific number.

<i>Extension</i>	<i>Description</i>
<code>.rxx</code>	Relocatable files created by the IAR Assembler or C Compiler that are read by XLINK.
<code>.xcl</code>	XLINK Linker command files (default).
<code>.axx</code>	Absolute, linked code in symbolic format output by XLINK.
<code>.lst</code>	Default file type for cross-reference listings created by XLINK.
<code>.dxx</code>	Absolute, linked code in IAR debug format.

USING XLINK

XLINK is run with the command:

```
xlink [options] objectfiles
```

If no parameters are specified a list of all the linker options will be displayed.

The command line options may be specified in any order. The order in which you specify the object files determines the order in which they are linked.

Upper or lower case is of importance in the following two cases:

- ◆ In linker command names; eg -f and -F.
- ◆ In symbol and segment names.

XLINK LINKER

As an alternative to supplying options on the command line you can use the `-f` option to read commands from a file.

For a full list of linker command line options, see *XLINK command reference*, page 2-25.

ERROR RETURN CODES

XLINK returns status information to the operating system which can be tested in a batch file. The supported error codes for MS-DOS are listed below:

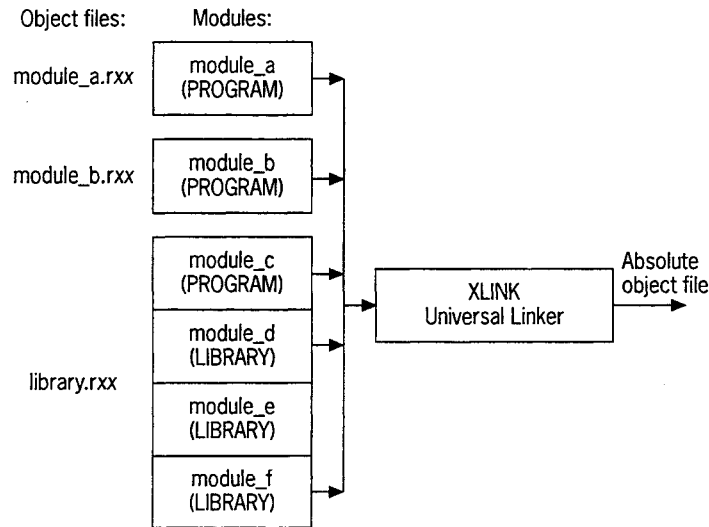
<i>Code</i>	<i>Description</i>
0	Linking successful.
1	There were warnings generated during the link unless the XLINK <code>-w</code> option was specified, in which case XLINK returns a 0 on warnings.
2	There was a non-fatal error.
3	Fatal error detected (XLINK aborted).

These error codes can be used as follows in a batch file:

```
.
xlink -f testlnk
if errorlevel 3 goto errors
if errorlevel 1 goto warnings
echo  the link was successful!
.
.
:warnings
echo  the link ended with a warning or non-fatal error ...
.
:errors
echo  the link ended with a fatal error ...
.
```

INPUT FILES AND MODULES

The following diagram shows how XLINK processes input files and load modules for a typical assembler or C program:



The main program has been assembled from two source files, `module_a.sxx` and `module_b.sxx`, to produce two relocatable files. Each of these files consists of a single module `module_a` and `module_b`. By default, the assembler assigns the PROGRAM attribute to both `module_a` and `module_b`. This means that they will always be loaded and linked whenever the files they are contained in are processed by XLINK; ie the filenames are given as arguments.

The code and data from a single C source file ends up as a single module in the file produced by the compiler. In other words, there is a one to one relationship between C source files and C modules. By default, the compiler gives this module the same name as the original C source file. Libraries of multiple C modules can only be created using XLIB.

XLINK LINKER

IAR Assembler programs can be constructed so that a single source file contains multiple modules, each of which can be a program module or a library module.

LIBRARIES

In the previous diagram, the file `library.rxx` consists of multiple modules, each of which could have been produced by the assembler or the C compiler.

The module `module_c`, which has the `PROGRAM` attribute will *always* be loaded whenever the `library.rxx` file is listed among the input files for the linker. In the run-time libraries, the startup module `cstartup` (which is a required module in all C programs) has the `PROGRAM` attribute so that it will always get included when you link a C project.

The other modules in the `library.rxx` file have the `LIBRARY` attribute. Library modules are only loaded if they contain an entry (a function, variable, or other symbol declared as `PUBLIC`) that is referenced in some way by another module that is loaded. This way, `XLINK` only gets the modules from the library file that it needs to build the program, and no more. For example, if the entries in `module_e` are not referenced by any loaded module, `module_e` will not be loaded.

This works as follows:

If `module_a` makes a reference to an external symbol, `XLINK` will search the other input files for a module containing that symbol as a `PUBLIC` entry; ie a module where the entry itself is located. If it finds the symbol declared as `PUBLIC` in `module_c`, it will then load that module (if it has not already been loaded). This procedure is iterative, so if `module_c` makes a reference to an external the same thing happens.

It is important to understand that a library file is just like any other relocatable object file. There is really no distinct type of file called a library (modules have a `LIBRARY` or `PROGRAM` attribute). What makes a file a library is what it contains and how it is used. Put simply, a library is a `.rxx` file that contains a group of related, often-used modules, most of which have a `LIBRARY` attribute so that they can be loaded on a demand-only basis.

CREATING LIBRARIES

You can create your own libraries, or add to existing libraries, using C or assembler modules. The C compiler `-b` option can be used to force a C module to have a `LIBRARY` attribute instead of the default `PROGRAM` attribute. In assembler programs, the `MODULE` directive is used to give a module the `LIBRARY` attribute, and the `NAME` directive is used to give a module the `PROGRAM` attribute.

The `XLIB` Librarian is used to create and manage libraries. Among other tasks, it can be used to alter the attribute (`PROGRAM/LIBRARY`) of any other module after it has been compiled or assembled.

SEGMENT LOCATION

Once `XLINK` has identified the modules to be loaded for a program, one of its most important functions is to assign load addresses to the various code and data segments that are being used by the program.

In assembly language programs the programmer is responsible for declaring and naming relocatable segments and determining how they are used. In C programs the compiler creates and uses a set of pre-defined code and data segments, and you have only limited control over segment naming and usage.

LISTING FORMAT

The XLINK listing format is shown below:

```
#####
#
#   Micro Series Universal Linker V4.45B/DXT      28/Apr/93  15:45:00
#
#   Target CPU   - 2000
#   List file    - day.lse
#   Output file 1 - aout.d00
#   Output format - debug
#   Command line - day -c2000 -r -l day.lse -xsme -ZDPRSEG,RCODE
#
#   (c) Copyright IAR Systems 1993
#
#####
```

Header

```
*****
*
*   CROSS REFERENCE
*
*****

Program entry at : 00000002 Relocatable, from module : DayofWeek

*****
*
*   ENTRY LIST
*
*****

DayofWeek ( day.r00 )

*****
*
*   MODULE MAP
*
*****

FILE NAME : day.r28
PROGRAM MODULE, NAME : DayofWeek

SEGMENTS IN THE MODULE

DPRSEG
Relative segment, address : 00000000 - 00000001
-----
RCODE
Relative segment, address : 00000002 - 00000044

*****
*
*   SEGMENTS IN DUMP ORDER
*
*****

SEGMENT      START ADDRESS  END ADDRESS  TYPE  ORG  P/N  ALIGN
DPRSEG       00000000 - 00000001  rel  flt  pos  1
RCODE        00000002 - 00000044  rel  flt  pos  1

*****
*
*   END OF CROSS REFERENCE
*
*****

Errors: none
Warnings: none
```

Cross reference

Entry list

Module map

Segment map

It consists of the following sections:

HEADER

Shows the command line, and options selected for the XLINK command:

	Target CPU type	Output file or device name for the listing	
#####			Time and date of the linkage
#			#
#	Micro Series Universal Linker V4.4 B/DXT	28/Apr/93 15:45:00	#
#			#
#	Target CPU = 2000		#
#	List file = day.lse		#
#	Output file 1 = aout.d00		#
#	Output format = debug		#
#	Command line = day -c2000 -l day.lse -xsme -ZDPRSEG,RCODE		#
#			#
#		(c) Copyright IAR Systems 1993	#
#####			Full list of options
	Output file format	Absolute output file name	

The full list of options shows the options specified on the command line. Options in command files specified with the -f option are also shown, in brackets.

CROSS REFERENCE

The cross reference consists of the entry list, module map and/or the segment map. It includes the program entry point, used in some output formats for hardware emulator support; see the assembler END directive.

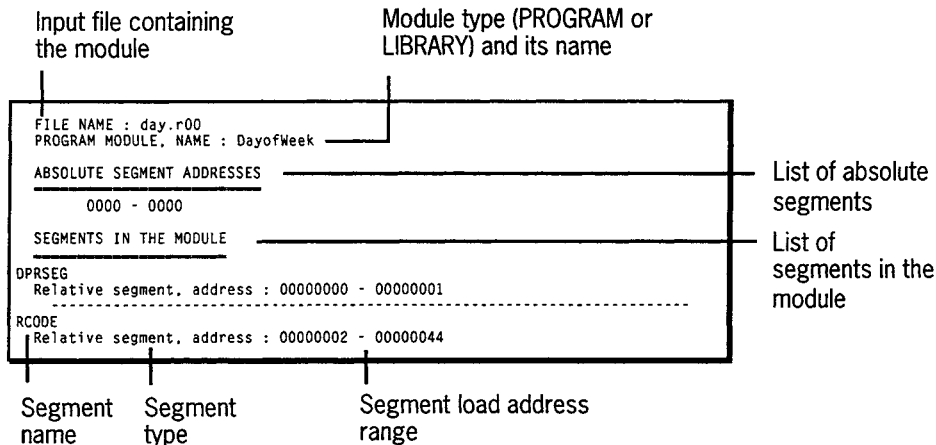
Entry list

Shows the entry name and address for each module and filename.

XLINK LINKER

Module map

The module map consists of a subsection for each module that was loaded as part of the program. Each subsection shows the following information:



For each segment the module map also lists locals and entries.

Segment map

The segment map lists the segments in the order in which they were linked:

SEGMENT	START ADDRESS	END ADDRESS	TYPE	ORG	P/N	ALIGN
DPRSEG	00000000	- 00000001	rel	flt	pos	1
RCODE	00000002	- 00000044	rel	flt	pos	1

This lists the start and end address for each segment, and the following parameters:

<i>Parameter</i>	<i>Description</i>
TYPE	The type of segment: rel Relative stc Stack bnk Banked com Common
ORG	The origin; the type of segment start address: stc Absolute flt Floating
P/N	Positive/Negative; how the segment is allocated: pos Upwards neg Downwards
ALIGN	2**align factor

XLINK LINKER

XLINK ENVIRONMENT VARIABLES

XLINK uses a number of environment variables. They can be defined in the PC host environment using the MS-DOS set command. These variables can be used to create defaults for various XLINK options so they do not have to be specified on the command line.

Except for the XLINK_ENVPAR and XLINK_TFILE environment variables, the default values can be overruled by the corresponding command line option.

For example, the -FMPDS command line argument will supersede the default format selected with the XLINK_FORMAT environment variable.

To make these settings automatic, you can place the set commands in your system's autoexec.bat file (or in your login script if you are running on a network).

XLINK_COLUMNS

Sets the number of columns per line.

DESCRIPTION

Sets the number of columns per line. The default is 80 columns.

EXAMPLE

To set the number of columns to 132:

```
set XLINK_COLUMNS=132
```

XLINK_CPU

Sets the target CPU type.

DESCRIPTION

The target CPU types are as follows:

The environment variable `XLINK_CPU` can be set to install default for the `-c` option so that it does not have to be specified on the command line; see `-c`, page 2-34.

EXAMPLE

To set the target CPU type to IAR2000:

```
set XLINK_CPU=2000
```

RELATED COMMANDS

This is equivalent to the `-c` command; see `-c`, page 2-34.

XLINK_DFLTDIR

Sets a path to a default directory for object files.

DESCRIPTION

The specified path, which should end with `\`, is prefixed to the object filename.

EXAMPLE

To specify the path for object files as `C:\iar\lib`:

```
set XLINK_DFLTDIR=C:\iar\lib\
```

XLINK_ENVPAR

Creates a default XLINK command line.

DESCRIPTION

Use XLINK_ENVPAR to specify XLINK commands that you want to execute each time you run XLINK.

EXAMPLE

To create a default XLINK command line:

```
set XLINK_ENVPAR=-FAOMF2000
```

RELATED COMMANDS

See also *-f*, page 2-40, which reads linker commands from a file.

XLINK_FORMAT

Sets the output format.

DESCRIPTION

For a list of the available output formats see *XLINK output formats*, page 2-61.

EXAMPLE

To set the output format to AOMF2000:

```
set XLINK_FORMAT=AOMF2000
```

RELATED COMMANDS

This is equivalent to the *-F* command; see *-F*, page 2-39.

XLINK_MEMORY

XLINK_MEMORY

Specifies whether XLINK is file bound or memory bound.

DESCRIPTION

If set to 0 the linker is file bound; otherwise it is memory bound.

EXAMPLE

To specify that XLINK is file bound:

```
set XLINK_MEMORY=0
```

RELATED COMMANDS

This is equivalent to the `-m` command; see *-m*, page 2-44.

XLINK_PAGE

Sets the number of lines per page.

DESCRIPTION

Sets the number of lines per page (20-150). The default is a non-paged list.

EXAMPLES

To set the number of lines per page to 64:

```
set XLINK_PAGE=64
```

RELATED COMMANDS

This is equivalent to the `-p` command; see *-p*, page 2-47.

XLINK_TFILE

Specifies the temporary file.

DESCRIPTION

Sets the name and location of the temporary file which is used when the -t command is specified; see -t, page 2-51.

EXAMPLE

To specify a temporary file E:\xlink.tmp:

```
set XLINK_TFILE=E:\xlink.tmp
```

XLINK_TFILE

XLINK COMMAND SUMMARY

This chapter summarizes the XLINK commands, classified according to their function.

A full alphabetical reference list of commands is given in the next chapter.

INPUT/OUTPUT FILE OPTIONS

-A <i>file</i> , ...	Forces input files to be loaded as program modules.
-B	Forces the generation of an output file.
-C <i>file</i> , ...	Conditionally loads input files.
-d	Disables code generation.
-E <i>file</i> , ...	Links without generating object code for specified files.
-F <i>format</i>	Selects the output format.
-o <i>file</i>	Sets the output filename.
-r{t}	Enables C-SPY debug output.
-Y[<i>char</i>]	Selects a variation of an output format.

SEGMENT CONTROL OPTIONS

-bbank_def	Defines banked segments.
-Zseg_def	Defines segments.
-z	Reduces segment overlap errors to warnings.

XLINK COMMAND SUMMARY

SYMBOL CONTROL OPTIONS

- a Disables overlaying of locals.
- Dsym=val* Defines symbols.
- new=old,old ...* Renames external symbols.
- G Disables global type checking.
- n Ignores local symbols.

LISTING CONTROL OPTIONS

- l *file* Specifies the listing filename.
- p*nnn* Sets the number of lines per page.
- x{ems} Generates a cross-reference list.

COMMAND FILE OPTIONS

- f *file* Reads linker commands from a file.
- ! *comment* -! Comment delimiter.

MISCELLANEOUS XLINK OPTIONS

- c*cpu* Specifies the processor type.
- K Suppresses the loading of default libraries.
- m Uses less host memory.
- R Disables address range check.
- S Selects silent operation.
- t Enables temporary file.
- w Disables warning messages.

XLINK COMMAND REFERENCE

This section gives an alphabetical list of the XLINK commands with a full description of their operation, and the options available for each one.

The format of each command is as follows:

Name	
-C	
Conditionally loads input files.	
SYNTAX <i>-C filename [. filename] _</i>	Syntax
PARAMETERS <i>filename</i> The relocatable object file to be conditionally loaded.	Parameters
DESCRIPTION Use -C to temporarily cause all of the modules within the specified input files to be treated as if they were all Library modules, even if some of the modules have the PROGRAM attribute. This means that the modules in the input files will be loaded only if they contain an entry that is referenced by another loaded module.	Description
EXAMPLES To load the user-defined <i>cstartup</i> module from the file <i>cstartup</i> instead of the program module of the same name in CLIB : ~! these lines are in an XCL file ~! <i>cstartup</i> -C CLIB This allows you to test the <i>cstartup</i> module before installing it in the library.	Examples
RELATED COMMANDS See also -A , which forces input files to be loaded as program modules.	Related commands

XLINK COMMAND REFERENCE

PARAMETERS

The following table shows the correct form of the most commonly-used types of parameter:

<i>Parameter</i>	<i>What it consists of</i>
<i>filenames</i>	One or more filenames. The filenames must be separated from the command by at least one tab or space.
<i>value</i>	An address or value. It is interpreted in hexadecimal unless preceded by a dot, in which case it is interpreted as decimal.

-!

Comment delimiter.

SYNTAX

`-! comments -!`

PARAMETERS

comments One or more lines of arbitrary text.

DESCRIPTION

Use `-!` to bracket off comments in an XLINK .xcl file. Unless the `-!` is at the beginning of a line, it must be preceded by a space or tab.

EXAMPLES

The following line includes a comment:

```
-c2000 -! set the CPU type for the IAR2000 -!
```

RELATED COMMANDS

See also `-f`, which allows you to supply commands to XLINK from a file.

-A

-A

Forces input files to be loaded as program modules.

`-A filename [,filename] ...`

PARAMETERS

filename Relocatable object file to be force-loaded.

DESCRIPTION

Use `-A` to temporarily force all of the modules within the specified input files to be loaded as if they were all program modules, even if some of the modules have the `LIBRARY` attribute.

This option is particularly suited for testing library modules before they are installed in a library file, since the `-A` option will override an existing library module with the same entries. In other words, `XLINK` will load the module from the *input file* specified in the `-A` argument instead of one with an entry with the same name in a library module.

EXAMPLE

To load the user-written library module `putchar.r00` instead of the standard one in the `CLIB` library:

```
#! these lines are in an .XCL file ... -!  
-A putchar  
CLIB
```

This assumes that the `putchar` file contains the same global entry as one of the modules in `CLIB`.

RELATED COMMANDS

See also `-C`, which conditionally loads input files as library modules.

-a

Disables overlaying of locals.

SYNTAX

-a [*options*]

PARAMETERS

options One of the following:

- | | |
|---------------------------------|--|
| -a | Disable overlay totally. This is for debug purposes. |
| -ai | Disable indirect function tree overlaying. |
| -aw | Disable warning number 16, function is called from two function trees. You can do this if you are sure that the code is correct. |
| -a(module:function, ...) | Do not overlay the function trees with the root functions <i>module: function</i> . |
| -a[module:function, ...] | Do not allocate parameter and auto space for the function trees with the root functions <i>module: function</i> . |
| -a{module:function, ...} | Define these functions as interrupt functions. |

Note that these options are only usable for ICC8051/V4 and V5, and ICC65000.

-B

-B

Forces the generation of an output file.

SYNTAX

-B

DESCRIPTION

Use **-B** to generate an output file even if a non-fatal error was encountered during the linking process, such as a missing global entry or a duplicate declaration. Normally, **XLINK** will not generate an output file if an error is encountered. Note that **XLINK** always aborts on fatal errors, even with **-B**.

The **-B** option allows missing entries to be patched in later in the absolute output image.

-b

Defines banked segments.

SYNTAX

-b [*addrtype*] [(*type*)] *segments*=*start*,*length*,*increment*

PARAMETERS

<i>addrtype</i>	The type of load addresses used when dumping the code:	
	omitted	Logical addresses with bank number.
	#	Linear physical addresses.
	@	64180-type physical addresses.
<i>type</i>	The type for all segments specified in <i>segments</i> .	
<i>segments</i>	The list of banked segments to be linked.	
	The delimiter between segments in the list determines how they are packed:	
:	(colon)	The next segment will be placed in a new bank.
,	(comma)	The next segment will be placed in the same bank as the previous one.
<i>start</i>	The start address of the first segment in the segment list. This is a 32-bit value: the high-order 16 bits represent the starting bank number while the low-order 16 bits represent the start address for the banks in the logical address area.	
<i>length</i>	The length of each bank, in bytes. This is a 16-bit value.	

-b

increment The incremental factor between banks, ie the number that will be added to *start* to get to the next bank. This is a 32-bit value: the high-order 16 bits are the bank increment, and the low-order 16 bits are the increment from the start address in the logical address area.

DESCRIPTION

Use **-b** to allocate banked segments for a program that is designed for bank-switched operation. It also enables the banking mode of linker operation.

EXAMPLES

To specify that the three code segments BSEG1, BSEG2, and BSEG3 should be linked into banks starting at 8000, each with a length of 4000, with an increment between banks of 10000:

```
-b (CODE) BSEG1,BSEG2,BSEG3=8000,4000,10000
```

RELATED COMMANDS

See also **-Z**, which defines a list of segments.

-C

Conditionally loads input files.

SYNTAX

-C *filename* [, *filename*] ...

PARAMETERS

filename The relocatable object file to be conditionally loaded.

DESCRIPTION

Use **-C** to temporarily cause all of the modules within the specified input files to be treated as if they were all library modules, even if some of the modules have the **PROGRAM** attribute. This means that the modules in the input files will be loaded only if they contain an entry that is referenced by another loaded module.

EXAMPLES

To load the user-defined **cstartup** module from the file **cstartup** instead of the program module of the same name in **CLIB**:

```
-! these lines are in an XCL file -!  
cstartup  
-C CLIB
```

This allows you to test the **cstartup** module before installing it in the library.

RELATED COMMANDS

See also **-A**, which forces input files to be loaded as program modules.

-c

-C

Specifies the processor type.

SYNTAX

-c cpu

PARAMETERS

cpu

One of the following:

68hc11	7700	16032	h8300
68k	7800	65000	h8500
1802	8048	78000	t7000
6301	8051	78200	t99
6502	8085	78300	z8
6801	8086	78600	z80
6805	8096	h16	z8002
6809			

DESCRIPTION

Use *-c* to set the CPU type.

The environment variable *XLINK_CPU* can be set to install a default for the *-c* option so that it does not have to be specified on the command line; see *XLINK_CPU*, page 2-18.

EXAMPLE

To set the processor type to IAR2000:

-c2000

-D

Defines symbols.

SYNTAX

`-D symbol=value`

PARAMETERS

symbol Any external (EXTERN) symbol in the program that is not defined elsewhere.

value The value to be assigned to *symbol*.

DESCRIPTION

Use `-D` to define absolute symbols at link time. This is especially useful for configuration purposes. Any number of symbols can be defined using the XCL file mode of XLINK operation. The symbol(s) defined in this manner will belong to a special module generated by the linker called `?ABS_ENTRY_MOD`.

XLINK will display an error message if you attempt to redefine an existing symbol.

EXAMPLE

To set `version` to 42 (decimal) and `monitor_entry` to F010 (hex):

```
-D version=.42
```

```
-D monitor_entry=F010
```

d

-d

Disables code generation.

SYNTAX

-d

DESCRIPTION

Use -d to disable the generation of output code from XLINK. This option is useful for the trial linking of programs; eg checking for syntax errors, missing symbol definitions, etc. XLINK will run slightly faster for larger programs when this option is used.

-E

Links without generating object code for specified files.

SYNTAX

-E *filename* [, *filename*] ...

PARAMETERS

filename The relocatable object file to be empty loaded.

DESCRIPTION

Use -E to empty load specified input files; they will be processed normally in all regards by the linker but output code will not be generated for these files.

One potential use for this feature is in creating separate output files for programming multiple EPROMs. This is done by empty loading all input files except the one(s) you want to appear in the output file.

EXAMPLES

In the following example a project consists of four files, `file1` to `file4`, but we only want to generate object for `file4` to be put into an EPROM:

```
-E file1,file2,file3
file4
-o project.hex
```

-e

-e

Renames external symbols.

SYNTAX

`-enew=old[,old] ...`

PARAMETERS

<i>new</i>	The new name to be assigned to an existing external symbol.
<i>old</i>	One or more existing external symbol names to be renamed, separated by commas.

DESCRIPTION

Use `-e` to configure a program at link time by redirecting a function call from one function to another function.

This can also be used for creating stub functions; ie when a system is not yet complete, undefined function calls can be directed to a dummy routine until the real function has been written.

EXAMPLES

To cause all calls to the `printf` function to be redirected to `small_printf`:

```
-esmall_printf=printf
```

-F

Selects the output format.

SYNTAX

-F*format*

PARAMETERS

format One of the supported XLINK output formats; for details of the formats see *XLINK output formats*, page 2-61.

If not specified, a default format will be used which depends on the target processor. This is usually INTEL-STANDARD or MOTOROLA.

DESCRIPTION

Use -F to select the output format.

The environment variable XLINK_FORMAT can be set to install an alternate default format on your system; see *XLINK_FORMAT*, page 2-19.

-f

-f

Reads linker commands from a file.

SYNTAX

-f *commandfile*

PARAMETERS

commandfile A text file containing linker commands. If not specified, an extension of `.xcl` is assumed.

DESCRIPTION

Use **-f** to extend the `XLINK` command line by reading arguments from a command file, just as if they were typed in on the command line.

Arguments are entered into the `XCL` file with a text editor using the same syntax as on the command line. However, in addition to spaces and tabs, the end-of-line CR is also treated as a valid delimiter between arguments. A command line may be extended by the `\[` sequence.

The `-!` comment delimiter can be used to bracket comments placed in the file for documentation.

EXAMPLE

To read commands from the file `test.xd`:

```
-f test.xd
```

-G

Disables global type checking.

SYNTAX

-G

DESCRIPTION

Use **-G** to disable type checking at link time. While a well-written program should not need this option, there may be occasions where it is helpful.

By default, XLINK performs link-time type checking between modules by comparing the external references to an entry with the **PUBLIC** entry (if the information exists in the object modules involved). A warning is printed if there are mismatches; otherwise the linker will continue and not abort.

-K

-K

Suppresses the loading of default libraries

SYNTAX

-K

DESCRIPTION

Every module loaded will set one of 256 flag bits, where each bit represents a type value.

XLINK will try to open and load files with the name `df1t1b##.rxx`, where `##` is the hexadecimal representation of a set flag bit. It will stop when no unresolved externals remain, or if no flag bits are set, and no error will be caused if it fails. Use the `-K` option to suppress this load function.

A default library pathname may be set through the environment variable, `XLINK_DFLTDIR`. The linker will first search the current directory, then the directory specified by `XLINK_DFLTDIR` for the library.

Note that the current IAR Compilers do not use the `-K` option.

-1

Specifies the listing filename.

SYNTAX

-1 *listfile*

PARAMETERS

listfile The name of the file or device to which a listing is directed. If an extension is not specified, `.lst` is used by default. However, an extension of `.map` is recommended to avoid confusing linker list files with assembler or compiler list files.

DESCRIPTION

Use **-1** to set the name of the file or device to receive the linker listing.

RELATED COMMANDS

See also **-x**, which determines the contents of the listing.

-m

-m

Uses less host memory.

SYNTAX

-m

DESCRIPTION

Use **-m** to reduce the amount of host system memory needed by using file pointers to all segments and modules, instead of reading all input files into RAM. If XLINK runs out of host memory during a link, this option will often help. However, XLINK will run more slowly if the **-m** option is used.

The **-m** option is equivalent to:

```
set XLINK_MEMORY=0
```

See *XLINK_MEMORY*, page 2-20.

EXAMPLES

To reduce the amount of memory needed to a minimum:

```
-m -n
```

RELATED COMMANDS

See also **-n**, which also reduces the amount of host memory needed by ignoring local symbols, and **-t**, which reduces host memory by storing linker tables on disk.

-n

Ignores local symbols.

SYNTAX

-n

DESCRIPTION

Use -n to ignore all local (non-public) symbols in the input modules. This option speeds up the linking process and can also reduce the amount of host memory needed to complete a link. If -n is used, locals will not appear in the listing cross-reference and will not be passed on to the output file.

Note that local symbols are only included in files if they were compiled or assembled with the appropriate option to specify this.

RELATED COMMANDS

See also -m, which also reduces the amount of host memory needed by using file bound processing.

-o

-O

Sets the output filename.

SYNTAX

-o filename

PARAMETERS

filename The name of the file to contain the linked, absolute output code. The default extension is *.axx*.

DESCRIPTION

Use *-o* to specify the name of the XLINK output file. If a name is not specified the linker will use the name *about.xxx* where *xxx* is dependent on the output format. If a name is supplied without a file type, the default file type for the selected output format (*-F* option) will be used.

If a format is selected that generates two output files, the user-specified file type (*.axx*) will only affect the primary output file (first format).

EXAMPLES

To create an output file *final.a00* using the debug format:

```
-o final.a00 -FDEBUG
```

RELATED COMMANDS

See also *-F*, which sets the output format.

-p

Sets the number of lines per page.

SYNTAX

-p *lines*

PARAMETERS

lines Number of lines per page; a decimal number from 10 to 150. The default is 80.

DESCRIPTION

Use **-p** to set the page length for the XLINK listings. The environment variable `XLINK_PAGE` can be set to install a default page length on your system; see *XLINK_PAGE*, page 2-20.

EXAMPLES

To set a page length of 66 lines, as used by some dot-matrix printers:

-p66

-R

-R

Disables address range check.

SYNTAX

-R

DESCRIPTION

Use -R to disable the address range check.

If an address is relocated out of the target CPU's address range (code, external data, or internal data address) an error message is generated. This usually indicates an error in an assembly language module or in the XLINK segment definition list (-Z command).

-r

Enables C-SPY debug output.

SYNTAX

-r[*option*]

PARAMETERS

<i>option</i>	An optional character to determine C-SPY options; see the <i>Using C-SPY</i> guide, and the appropriate C-SPY Target Supplement.
---------------	--

DESCRIPTION

Use **-r** to output a file in DEBUG (AUBROF) format, with a **.dxx** extension, to be used with the C-SPY debugger, or emulators which support the IAR Systems DEBUG format.

-S

-S

Selects silent operation.

SYNTAX

-S

DESCRIPTION

Use -S to turn off the XLINK sign-on message and final statistics report so that nothing appears on your screen while it runs. However, it does not disable error and warning messages or the listing output.

RELATED COMMANDS

See also -w, which disables warning messages.

-t

Enables temporary file.

SYNTAX

-t

DESCRIPTION

Use **-t** to force XLINK to use a temporary file, with the default name `xlink.tmp` in the current directory, to store a large part of the linker symbol tables. This can significantly reduce the amount of host system memory needed to link a program with a large number of symbols; eg more than 1500. In some cases, it may be necessary to use this option to complete a link process.

Note that the **-t** option can significantly increase the time it takes to link a program. The **-m** file bound processing option will also be enabled automatically when **-t** is used.

The environment variable `XLINK_TFILE` can be set to an alternate filename (with drive and directory path) to use for the temporary file; see *XLINK_TFILE*, page 2-21.

RELATED COMMANDS

See also **-m**, which selects file bound processing, and **-n**, which reduces the amount of host memory needed by ignoring local symbols.

-w

-W

Disables warning messages.

SYNTAX

-w

DESCRIPTION

Use **-w** to suppress all warning messages. They will, however, still be counted and shown in the linker's final statistics.

-X

Generates a cross-reference list.

SYNTAX

-x options

PARAMETERS

<i>options</i>	Any of the following:
e	Generates a module entry list.
m	Generates a module map.
s	Generates a segment map.

Note that e and m are mutually exclusive options.

DESCRIPTION

Use -x to determine the contents of the XLINK listing file. When -x is specified without any of the optional parameters, a default cross-reference listing will be generated which includes (this is equivalent to -xms):

- ◆ A header section with basic program information.
- ◆ A module load map with symbol cross-reference.
- ◆ A segment load map in dump order.

If the `-x` option is succeeded by one or more of the arguments `e`, `m`, or `s`, the listing will be modified as follows:

<i>Option</i>	<i>What it consists of</i>
<code>e</code> (entry list)	An abbreviated list of every entry (global symbol) in every module. This entry map is useful for quickly finding the address of a routine or data element.
<code>m</code> (module map)	A list of all segments, local symbols, and entries (public symbols) for every module in the program.
<code>s</code> (segment map)	A list of all the segments in dump order.

If a `-l` list filename option is not specified in the link, the listing will be sent to your screen.

EXAMPLES

To generate a cross-reference listing with an entry and segment map to the file `program.map`:

```
-xes -l program.map
```

RELATED COMMANDS

See also `-l`, which sets the listing filename.

-Y

Selects a variation of an output format.

SYNTAX

-Y[*character*]

PARAMETERS

character Specifies an optional variation of the format chosen by the -F option. The default is 0.

DESCRIPTION

Use -Y to select enhancements available for some output formats.

The following options are available:

<i>Output format</i>	<i>Option</i>	<i>Description</i>
PENTICA-A,B,C,D and MPDS-SYMB	Y0	Symbols as <i>module:symbolname</i> .
	Y1	Labels and lines as <i>module:symbolname</i> .
	Y2	Lines as <i>module:symbolname</i> .
AOMF8051	Y0	Extra type information for Hitex.
INTEL-STANDARD	Y0	End only with :00000001FF.
	Y1	End with PGMENTRY, else :00000001FF.
MPDS-CODE	Y0	Fill with 0xFF instead.
DEBUG, -r	Y#	Old UBROF version.
INTEL-EXTENDED	Y0	Segmented variant.
	Y1	32-bit linear variant.

RELATED COMMANDS

See also -F, which selects the output format.

-Z

Defines segments.

SYNTAX

-Z [(*type*)] *segments* [=|#] [*start-end*,] ... [*address*]

PARAMETERS

<i>type</i>	Specifies the memory type for all segments in <i>segments</i> , if applicable for the target processor. If omitted it defaults to UNTYPED.
<i>segments</i>	A list of one or more segments to be linked, separated by commas. The segments are allocated in memory in the same order as they are listed. Appending <i>+nnnn</i> to a segment name increases the amount of memory that XLINK will allocate for that segment by <i>nnnn</i> bytes.
= or #	Specifies how segments are allocated. = Allocates the segments so they begin at the start of the specified range (upwards allocation). # Allocates the segments so they finish at the end of the specified range (downwards allocation). If an allocation operator (and range) is not specified, the segments will be allocated upwards from the last segment that was linked, or from address 0 if no segments have been linked.
<i>start, end</i>	Addresses defining a range within which the listed <i>segments</i> should be placed.
<i>address</i>	Start address for placing any remaining segments to be allocated.

DESCRIPTION

Use `-Z` to specify how and where segments will be allocated in the memory map.

If the linker finds a segment in an input file that is not defined either with `-Z` or `-b` (banked definition command), a warning will be displayed by the linker. However, the segment will still be allocated as if it were listed in the last segment definition; ie at the next available address.

There can be more than one `-Z` definition.

Additional related topics and optional forms for `-Z` are described below.

Allocation segment types

The following table lists the different types of segments that can be processed by XLINK:

<i>Segment type</i>	<i>Description</i>
STACK	Allocated from high to low addresses by default. The aligned segment size is subtracted from the load address before allocation, and successive segments are placed below the preceding segment.
RELATIVE COMMON	Allocated from low to high addresses by default.

If stack segments are mixed with relative or common segments in a segment definition, the linker will produce a warning message but will allocate the segments according to the default allocation set by the first segment in the segment list.

Common segments have a size equal to the largest declaration found for the particular segment. That is, if module A declares a common segment COMSEG with size 4, while module B declares this segment with size 5, the latter size will be allocated for the segment.

Be careful not to overlay common segments containing code or initializers.

Relative and stack segments have a size equal to the sum of the different (aligned) declarations.

Memory types of segments

The optional *type* parameter is used to assign a type to all of the segments in the list. The *type* parameter affects how XLINK processes the segment overlaps. Additionally, it generates information in some of the output formats that are used by some hardware emulators and by C-SPY.

Segment type* *Description

FAR	Data in FAR memory, as defined by MELPS and other processors. XLINK will not check access to it, and a part of a segment straddling a 64 Kbytes boundary will be moved upwards to start at the boundary.
FARC	Constant in FAR memory (behaves as above).
HUGE	Data in HUGE memory. No straddling problems.
HUGECL	Constant in HUGE memory.
CODE	Code memory.
DATA	Data memory.
UNTYPED	Default type.
XDATA	8051 XDATA memory.
IDATA	8051 IDATA memory.
BIT	Bit memory.
IDATA0	65000 IDATA0 memory.
IDATA1	65000 IDATA1 memory.

Range errors

If the ranges specified in the -Z command are too short, it will cause either error 24 Segment *segment* overlaps segment *segment*, if any segment overlaps another, or error 26 Segment *segment* is too long, if the ranges are too small.

EXAMPLES

To locate SEGA at address 0, followed immediately by SEGB:

-Z(CODE)SEGA, SEGB=0

To allocate SEGA downwards from 1000H, followed by SEGB below it:

-Z(CODE)SEGA, SEGB#1000

To allocate specific areas of memory to SEGA and SEGB:

-Z(CODE)SEGA, SEGB=100-200, 400-700, 1000

In this example SEGA will be placed between address 100 and 200, if it fits in that amount of space. If it does not, XLINK will try the range 400 – 700. If none of these ranges are large enough to hold SEGA, it will start at 1000.

SEGB will be placed, according to the same rules, after segment SEGA. If SEGA fits the 100–200 range then XLINK will try to put SEGB there as well (following SEGA). Otherwise, SEGB will go into the 400 – 700 range if it is not too large, or else it will start at 1000.

-Z

-Z

Reduces segment overlap errors to warnings.

SYNTAX

-Z

DESCRIPTION

By default, XLINK checks to be sure that the various segments that have been defined (by the -Z command and absolute segments) do not overlap in memory.

Use -z to reduce segment overlap errors to warnings, making it possible to produce cross-reference maps, etc.

XLINK OUTPUT FORMATS

This chapter gives a summary of the XLINK output formats.

SINGLE OUTPUT FILE

The following formats result in the generation of a single output file:

<i>Format</i>	<i>Type</i>	<i>Extension</i>	<i>Address type</i>
AOMF8051†	binary	from CPU	N
AOMF8096†	binary	from CPU	N
ASHLING	binary	none	N
ASHLING-6301†	binary	from CPU	N
ASHLING-64180†	binary	from CPU	NS
ASHLING-6801†	binary	from CPU	N
ASHLING-8080†	binary	from CPU	NS
ASHLING-8085†	binary	from CPU	NS
ASHLING-Z80†	binary	from CPU	NS
DEBUG†	binary	.dbg	NL
EXTENDED-TEKHEX†	ASCII	from CPU	NLPS
HP-CODE	binary	.x	NLPS
HP-SYMB	binary	.l	NLPS
INTEL-STANDARD	ASCII	from CPU	N
INTEL-EXTENDED	ASCII	from CPU	NLPS
MILLENIUM (Tektronix)	ASCII	from CPU	N
MOTOROLA	ASCII	from CPU	NLPS

XLINK OUTPUT FORMATS

<i>Format</i>	<i>Type</i>	<i>Extension</i>	<i>Address type</i>
MPDS-CODE	binary	.tsk	N
MPDS-SYMB	binary	.sym	NLPS
MSD	ASCII	.sym	N
NEC-SYMBOLIC†	ASCII	.sym	N
NEC2-SYMBOLIC†	ASCII	.sym	N
NEC78K-SYMBOLIC†	ASCII	.sym	N
PENTICA-A	ASCII	.sym	NLPS
PENTICA-B	ASCII	.sym	NLPS
PENTICA-C	ASCII	.sym	NLPS
PENTICA-D	ASCII	.sym	NLPS
RCA	ASCII	from CPU	N
SYMBOLIC	ASCII	from CPU	NLPS
SYSROF†	binary	mabs	NLPS
TEKTRONIX (Millenium)	ASCII	.hex	N
TI7000 (TMS7000)	ASCII	from CPU	N
TYPED	ASCII	from CPU	NLPS
ZAX	ASCII	from CPU	NLPS

Address type

The address type is one of the following:

N = Non-banked address.

L = Banked logical address.

P = Banked physical address.

S = Banked 64180 physical address.

† format depends on the typing of the segments; ie the *type* field specified in the XLINK -Z option is important:

XLINK OUTPUT FORMATS

TWO OUTPUT FILES

The following formats result in the generation of two output files:

<i>Format</i>	<i>Code format</i>	<i>Extension</i>	<i>Symbolic format</i>	<i>Extension</i>
HP	HP-CODE	.x	HP-SYMB	.l
MPDS	MPDS-CODE	.tsk	MPDS-SYMB	.sym
MPDS-I	INTEL-STANDARD	.hex	MPDS-SYMB	.sym
MPDS-M	MOTOROLA	.s19	MPDS-SYMB	.sym
MSD-I	INTEL-STANDARD	.hex	MSD	.sym
MSD-M	MOTOROLA	.hex	MSD	.sym
MSD-T	Millenium	.hex	MSD	.sym
NEC	INTEL-STANDARD	.hex	NEC-SYMB	.sym
NEC2	INTEL-STANDARD	.hex	NEC2-SYMB	.sym
PENTICA-AI	INTEL-STANDARD	.obj	PENTICA-A	.sym
PENTICA-AM	MOTOROLA	.obj	PENTICA-A	.sym
PENTICA-BI	INTEL-STANDARD	.obj	PENTICA-B	.sym
PENTICA-BM	MOTOROLA	.obj	PENTICA-B	.sym
PENTICA-CI	INTEL-STANDARD	.obj	PENTICA-C	.sym
PENTICA-CM	MOTOROLA	.obj	PENTICA-C	.sym
PENTICA-DI	INTEL-STANDARD	.obj	PENTICA-D	.sym
PENTICA-DM	MOTOROLA	.obj	PENTICA-D	.sym
ZAX-I	INTEL-STANDARD	.hex	ZAX	.sym
ZAX-M	MOTOROLA	.hex	ZAX	.sym

XLINK OUTPUT FORMATS

XLIB LIBRARIAN

This chapter describes the XLIB Librarian, which is designed to allow you to create and maintain relocatable libraries of routines.

INTRODUCTION

Like the XLINK Linker, the XLIB Librarian, uses the UBROF standard object format (Universal Binary Relocatable Object Format) to allow it to support a wide range of 32 bit byte-oriented processors (applies to almost all current major microprocessors).

LIBRARIES

In the terms of the IAR C Compiler, Assembler, Linker, and Librarian, a library is a single file that contains a number of relocatable object modules, each of which can be loaded independently from other modules in the file as it is needed.

Normally, the modules in a library file all have the `LIBRARY` attribute, which means that they will only be loaded by the linker if they are actually needed in the program. This is referred to as demand loading of modules.

On the other hand, a module with the `PROGRAM` attribute is *always* loaded when the file in which it is contained is processed by the linker.

A library file is no different from any other relocatable object file produced by the assembler or C compiler, except that it would include a number of modules of the `LIBRARY` type.

USING LIBRARIES WITH C PROGRAMS

All C programs make use of libraries, and the IAR C Compilers are supplied with a number of standard library files.

Most C programmers will use the XLIB Librarian at some point, for one of the following reasons:

XLIB LIBRARIAN

- ◆ To replace or modify a module in one of the standard libraries. For example, the librarian can be used to replace the distribution versions of the `cstartup` and/or `putchar` modules with ones that they have customized.
- ◆ To add C or assembler modules to the standard library file so they will always be available whenever a C program is linked.
- ◆ To create custom library files that can be linked into their programs, as needed, along with the standard C library.

USING LIBRARIES WITH ASSEMBLER PROGRAMS

Assembler-only users do not need to use libraries. However, libraries provide the following advantages, especially when writing medium- and large-sized assembler applications:

- ◆ They allow you to combine utility modules used in more than one project into a simple library file. This simplifies the linking process by eliminating the need to include a list of input files for all the modules you need. Only the library module(s) needed for the program will be included in the output file.
- ◆ They simplify program maintenance by allowing multiple modules to be placed in a single assembler source file. Each of the modules can be loaded independently as a library module.
- ◆ They reduce the number of object files that make up an application, maintenance, and documentation.

You can create your assembly language library files using one of two basic methods:

- ◆ A library file can be created by assembling a single assembler source file which contains multiple library-type modules. The resulting library file can then be modified using XLIB.
- ◆ A library file can be produced by using the XLIB Librarian to merge any number of existing modules together to form a user-created library.

The `NAME` and `MODULE` assembler directives are used to declare modules as being of `PROGRAM` or `LIBRARY` type, respectively.

RUNNING XLIB

You can run XLIB in either interactive mode or batch mode.

INTERACTIVE MODE

To run XLIB in interactive mode type:

xlib 

The librarian will then prompt:

*

At the XLIB * prompt you can enter any of the XLIB commands, including all of the necessary parameters to complete the command all on one line.

You may also type just the name of the command line and XLIB will prompt you for the necessary parameters one at a time. Using this mode of operation, you do not need to remember the order of parameters for each command.

Command Syntax

Commands and parameters should be separated by commas, , or spaces, and prompts are issued in the absence of a parameter.

BATCH MODE

To run XLIB in batch mode type:

XLIB *commandfile* [*p0,p1...p9*]

Where *commandfile* specifies a text file containing a sequence of XLIB commands.

XLIB LIBRARIAN

Parameters

Optional parameters may be added after the filename and must be separated with spaces or commas. You can set a parameter to its default value by writing `..`. Within the command file, parameters will be substituted at every occurrence of a `\` (backslash) followed by a decimal number. For example:

```
LIST-MOD \4 PRN
```

If COMFIL contains a line like the one above, the command:

```
XLIB COMFIL,ABC G,,78,CODE SOME
```

would make the line be interpreted as:

```
LIST-MOD CODE PRN
```

A default value may be added immediately after a substitute parameter. The default value is recognized as a string enclosed in single quotes; eg `'Parm'`. If the requested parameter is missing in the XLIB invocation line or is written as `..`, the default string will replace the `\n` parameter.

An example of a line with a substitute parameter and a default value is:

```
LIST-MOD \4'OBJ' PRN
```

Interactive parameters

Interactive parameters written `\?`, will make XLIB stop and wait for a user response. The interactive parameters may also have a user prompt string attached immediately after the question mark.

For example:

```
LIST-MOD \?'YOUR FILE PLEASE: ' PRN
```

The `\n` parameter is replaced with the user input excluding the return.

Note that when XLIB is used in batch mode with a command file, it operates completely silently except when errors occur. This batch operation can be further controlled by the commands `ON-ERROR-EXIT`, `ECHO-INPUT`, and `REMARK`.

XLIB ENVIRONMENT VARIABLES

XLIB supports a number of environment variables. Under MS-DOS they can be set using the MS-DOS set command. These variables can be used to create defaults for various XLIB options so they do not have to be specified on the command line.

To make these settings automatic, you can place the set commands in your system's autoexec.bat file (or in your login script if you are running on a network).

XLIB_COLUMNS

Sets the number of columns/line (80–132) for listings. The default is 80 columns.

EXAMPLE

To set the width to 132 columns:

```
set XLIB_COLUMNS=132
```

XLIB_CPU

Sets the CPU type. If this variable is set to 2000, the DEFINE-CPU command does not need to be entered at the beginning of an XLIB session.

EXAMPLE

To set the CPU type to IAR2000:

```
set XLIB_CPU=2000
```

XLIB_PAGE

XLIB_PAGE

Sets the number of lines/page (10–100) for the list files. The default is a non-paged list.

EXAMPLE

To set the length to 66 lines:

```
set XLIB_PAGE=66
```

XLIB_SCROLL_BREAK

By setting this variable to *nnn* lines (16–100), XLIB will pause and wait for the  key to be pressed after *nnn* lines on the screen have scrolled by.

EXAMPLE

To pause every 22 lines:

```
set XLIB_SCROLL_BREAK=22
```

XLIB COMMAND SUMMARY

This chapter summarizes the librarian commands, classified according to their function.

A full alphabetical reference list of commands is given in the next chapter.

LIBRARY LISTING COMMANDS

LIST-ALL-SYMBOLS	Lists every symbol in modules.
LIST-CRC	Lists CRC values of modules.
LIST-DATE-STAMPS	Lists dates of modules.
LIST-ENTRIES	Lists PUBLIC symbols in modules.
LIST-EXTERNALS	Lists EXTERN symbols in modules.
LIST-MODULES	Lists modules.
LIST-OBJECT-CODE	Lists low-level relocatable code.
LIST-SEGMENTS	Lists segments in modules.

LIBRARY EDITING COMMANDS

DELETE-MODULES	Removes modules from a library.
FETCH-MODULES	Adds modules to a library.
INSERT-MODULES	Moves modules in a library.
MAKE-LIBRARY	Changes a module to library type.
MAKE-PROGRAM	Changes a module to program type.
RENAME-ENTRY	Renames PUBLIC symbols.
RENAME-EXTERNAL	Renames EXTERN symbols.
RENAME-GLOBAL	Renames EXTERN and PUBLIC symbols.

XLIB COMMAND SUMMARY

RENAME-MODULE	Renames one or more modules.
RENAME-SEGMENT	Renames one or more segments.
REPLACE-MODULES	Updates executable code.

MISCELLANEOUS LIBRARY COMMANDS

COMPACT-FILE	Shrinks library file size.
DEFINE-CPU	Specifies CPU type.
DIRECTORY	Displays available object files.
DISPLAY-OPTIONS	Displays XLIB options.
ECHO-INPUT	Command file diagnostic tool.
EXIT	Returns to operating system.
HELP	Displays help information.
ON-ERROR-EXIT	Quits on a batch error.
QUIT	Returns to operating system.
REMARK	Comment in command file.

XLIB COMMAND REFERENCE

This chapter gives a full syntactic and functional description of all librarian commands.

The individual words of an identifier can be abbreviated to the limit of ambiguity. For example, LIST-MODULES can be abbreviated to L-M.

PARAMETERS

The following parameters are common to many of the XLIB commands.

<i>Parameter</i>	<i>What it means</i>										
<i>objectfile</i>	File containing object modules.										
<i>start, end</i>	The first and last modules to be processed, in one of the following forms: <table><tr><td><i>n</i></td><td>The <i>n</i>th module.</td></tr><tr><td><i>\$</i></td><td>The last module.</td></tr><tr><td><i>name</i></td><td>Module <i>name</i>.</td></tr><tr><td><i>name+n</i></td><td>The module <i>n</i> modules after <i>name</i>.</td></tr><tr><td><i>\$-n</i></td><td>The module <i>n</i> modules before the last.</td></tr></table>	<i>n</i>	The <i>n</i> th module.	<i>\$</i>	The last module.	<i>name</i>	Module <i>name</i> .	<i>name+n</i>	The module <i>n</i> modules after <i>name</i> .	<i>\$-n</i>	The module <i>n</i> modules before the last.
<i>n</i>	The <i>n</i> th module.										
<i>\$</i>	The last module.										
<i>name</i>	Module <i>name</i> .										
<i>name+n</i>	The module <i>n</i> modules after <i>name</i> .										
<i>\$-n</i>	The module <i>n</i> modules before the last.										
<i>listfile</i>	File to which a listing will be sent.										
<i>source</i>	A file from which modules will be read.										
<i>destination</i>	The file to which modules will be sent.										

MODULE EXPRESSIONS

In most of the XLIB commands you can or must specify a source module (like *oldname* in RENAME-MODULE), or a range of modules (*startmodule*, *endmodule*).

XLIB COMMAND REFERENCE

Internally in all XLIB operations modules are numbered upwards from one. Modules may be referred to by the actual name of the module, by the name plus or minus a relative expression, or by an absolute number. The latter is very useful when a module name is very long, unknown, or contains unusual characters (like space or comma). Below is a list of the available variations on module expressions:

<i>Name</i>	<i>Description</i>
3	The third module.
\$	The last module.
<i>name</i> +4	The module 4 modules after <i>name</i> .
<i>name</i> -12	The module 12 modules before <i>name</i> .
\$-2	The module 2 modules before the last module.

The command `LIST-MOD FILE,,$-2` will thus list the three last modules in `FILE` on the user terminal.

LIST FORMAT

The `LIST` commands give a list of symbols, where each symbol has one of the following prefixes:

<i>Prefix</i>	<i>Description</i>
<i>nn</i> .Pgm	A program module with relative number <i>nn</i> .
<i>nn</i> .Lib	A library module with relative number <i>nn</i> .
Ext	An external in the current module.
Ent	An entry in the current module.
Loc	A local in the current module.
Rel	A standard segment in the current module.
Stk	A stack segment in the current module.
Com	A common segment in the current module.

COMPACT-FILE

Shrinks library file size.

SYNTAX

COMPACT-FILE *objectfile*

DESCRIPTION

Use **COMPACT-FILE** to concatenate short, absolute records into longer records of variable length. This is useful for library files which, due to the decreased size, will take up less time during the loader/linker process.

DEFINE-CPU

Specifies CPU type.

SYNTAX

DEFINE-CPU *cpu*

PARAMETERS

CPU The target processor.

DESCRIPTION

This command must be issued before any operations on object files can be done.

DELETE-MODULES

DELETE-MODULES

Removes modules from a library.

SYNTAX

DELETE-MODULES *objectfile start end*

DESCRIPTION

Use DELETE-MODULES to delete the specified modules.

DIRECTORY

Displays available object files.

SYNTAX

DIRECTORY [*specifier*]

DESCRIPTION

Use DIRECTORY to display on the user terminal all files of the type that applies to the target processor. If no *specifier* is given, the current directory is listed.

DISPLAY-OPTIONS

Displays XLIB options.

SYNTAX

DISPLAY-OPTIONS [*listfile*]

DESCRIPTION

Use DISPLAY-OPTIONS to list on the *listfile* the names of all the CPUs which are recognized by this version of XLIB. The default file types of object files for the different CPUs are also listed. After that a list of all UBROF tags is output.

ECHO-INPUT

Command file diagnostic tool.

SYNTAX

ECHO-INPUT

DESCRIPTION

ECHO-INPUT is useful when debugging command files in batch mode because it makes all command input visible on the terminal. In the interactive mode it has no effect.

EXIT

EXIT

Returns to operating system.

SYNTAX

EXIT

DESCRIPTION

Use EXIT to exit from XLIB after an interactive session.

FETCH-MODULES

Adds modules to a library.

SYNTAX

FETCH-MODULES *source destination* [*start*] [*end*]

DESCRIPTION

Use FETCH-MODULES to append the specified modules to *destination* file. If *destination* already exists, it must be empty or contain valid object modules; otherwise it will be created.

HELP

Displays help information

SYNTAX

HELP [*command*] [*listfile*]

PARAMETERS

command Command to display help about.

DESCRIPTION

If the **HELP** command is followed by *.*, only a list of the available commands will be displayed on the terminal. If a parameter is specified, all commands which match the parameter will be displayed with a brief explanation of their syntax and function. A *** matches all commands. **HELP** output can be directed to any file.

INSERT-MODULES

Moves modules in a library.

SYNTAX

INSERT-MODULES *objectfile start end* {**BEFORE** | **AFTER**} *dest*

DESCRIPTION

Use **INSERT-MODULES** to move the specified modules before or after the *dest*.

LIST-ALL-SYMBOLS

LIST-ALL-SYMBOLS

Lists every symbol in modules.

SYNTAX

LIST-ALL-SYMBOLS *objectfile* [*listfile*] [*start*] [*end*]

DESCRIPTION

Use LIST-ALL-SYMBOLS to list all symbols (module names, segments, externals, entries, and locals) for the specified modules in the *objectfile*. They are listed to the *listfile*.

Each symbol is identified with a prefix; see *List format*, page 2-74.

LIST-CRC

Lists CRC values of modules.

SYNTAX

LIST-CRC *objectfile* [*listfile*] [*start*] [*end*]

DESCRIPTION

Use LIST-CRC to list the module names and their associated CRCs for the specified modules.

Each symbol is identified with a prefix; see *List format*, page 2-74.

LIST-DATE-STAMPS

Lists dates of modules.

SYNTAX

LIST-DATE-STAMPS *objectfile* [*listfile*] [*start*] [*end*]

DESCRIPTION

Use LIST-DATE-STAMPS to list the module names and their associated generation dates for the specified modules.

Each symbol is identified with a prefix; see *List format*, page 2-74.

LIST-ENTRIES

Lists PUBLIC symbols in modules.

SYNTAX

LIST-ENTRIES *objectfile* [*listfile*] [*start*] [*end*]

DESCRIPTION

Use LIST-ENTRIES to list the names and associated entries for the specified modules.

Each symbol is identified with a prefix; see *List format*, page 2-74.

LIST-EXTERNALS

LIST-EXTERNALS

Lists EXTERN symbols in modules.

SYNTAX

LIST-EXTERNALS *objectfile* [*listfile*] [*start*] [*end*]

DESCRIPTION

Use LIST-EXTERNALS to list the module names and associated externals for the specified modules.

Each symbol is identified with a prefix; see *List format*, page 2-74.

LIST-MODULES

Lists modules.

SYNTAX

LIST-MODULES *objectfile* [*listfile*] [*start*] [*end*]

DESCRIPTION

Use LIST-MODULES to list the module names for the specified modules.

Each symbol is identified with a prefix; see *List format*, page 2-74.

LIST-OBJECT-CODE

Lists low-level relocatable code.

SYNTAX

LIST-OBJECT-CODE *objectfile* [*listfile*]

DESCRIPTION

Use **LIST-OBJECT-CODE** to list the contents of the *objectfile* on the *listfile* in an ASCII format.

Each symbol is identified with a prefix; see *List format*, page 2-74.

LIST-SEGMENTS

Lists segments in modules.

SYNTAX

LIST-SEGMENTS *objectfile* [*listfile*] [*start*] [*end*]

DESCRIPTION

Use **LIST-SEGMENTS** to list the module names and associated segments for the specified modules.

Each symbol is identified with a prefix; see *List format*, page 2-74.

MAKE-LIBRARY

MAKE-LIBRARY

Changes a module to library type.

SYNTAX

MAKE-LIBRARY *objectfile* [*start*] [*end*]

DESCRIPTION

Use MAKE-LIBRARY to change the module header attributes to conditionally loaded for the specified modules.

MAKE-PROGRAM

Changes a module to program type.

SYNTAX

MAKE-PROGRAM *objectfile* [*start*] [*end*]

DESCRIPTION

Use MAKE-PROGRAM to change the module header attributes to unconditionally loaded for the specified modules.

ON-ERROR-EXIT

Quits on a batch error.

SYNTAX

ON-ERROR-EXIT

DESCRIPTION

Use ON-ERROR-EXIT to make the librarian abort if an error is found. Most suited for use in batch mode.

QUIT

Returns to operating system.

SYNTAX

QUIT

DESCRIPTION

Use QUIT to exit and return to the operating system.

REMARK

REMARK

Comment in command file.

SYNTAX

REMARK *text*

DESCRIPTION

Use REMARK to include a comment.

RENAME-ENTRY

Renames PUBLIC symbols.

SYNTAX

RENAME-ENTRY *objectfile old new [start] [end]*

DESCRIPTION

Use RENAME-ENTRY to rename all occurrences of an entry from *old* to *new* in the specified modules.

RENAME-EXTERNAL

Renames EXTERN symbols.

SYNTAX

RENAME-EXTERNAL *objectfile old new [start] [end]*

DESCRIPTION

Use RENAME-EXTERNAL to rename all occurrences of an external from *old* to *new* in the specified modules.

RENAME-GLOBAL

Renames EXTERN and PUBLIC symbols.

SYNTAX

RENAME-GLOBAL *objectfile [start] [end]*

DESCRIPTION

Use RENAME-GLOBAL to rename all occurrences of an external or entry from *old* to *new* in the specified modules.

RENAME-MODULE

RENAME-MODULE

Renames one or more modules.

SYNTAX

RENAME-MODULE *objectfile old new*

DESCRIPTION

Use RENAME-MODULE to rename a module. Note that if there is more than one module with name *old*, only the first encountered is changed.

RENAME-SEGMENT

Renames one or more segments.

SYNTAX

RENAME-SEGMENT *objectfile old new [start] [end]*

DESCRIPTION

Use RENAME-SEGMENT to rename all occurrences of a segment from name *old* to *new* in the specified modules.

REPLACE-MODULES

Updates executable code.

SYNTAX

REPLACE-MODULES *source destination*

DESCRIPTION

Use REPLACE-MODULES to replace modules with the same name from *source* to *destination*. All replacements are logged on the terminal. The main application for this command is to update large run-time libraries etc.

REPLACE-MODULES

XLINK AND XLIB DIAGNOSTICS

This chapter describes the errors and warnings produced by the XLINK Linker and the XLIB Librarian.

XLINK MESSAGES

The error messages produced by the XLINK Linker fall into five categories:

- ◆ Linker warning messages.
- ◆ Linker error messages.
- ◆ Linker fatal error messages.
- ◆ Memory overflow message.
- ◆ Linker internal errors.

XLINK WARNING MESSAGES

XLINK warning messages will appear when the linker detects something that may be wrong. The code generated may still be correct.

XLINK ERROR MESSAGES

XLINK error messages are produced when the linker detects something wrong. The linking process will not be aborted but the code produced may be faulty.

XLINK FATAL ERRORS

XLINK fatal error messages abort the linking process. They occur when continued linking is useless, ie the fault is irrecoverable.

XLINK AND XLIB DIAGNOSTICS

MEMORY OVERFLOW MESSAGE

XLINK is a memory-based linker. If run on a system with a small main memory or if very large source files are being used, XLINK may run out of memory. This is recognized by the following message:

* * * LINKER OUT OF MEMORY * * *

Dynamic memory used: *nnnnnn* bytes

If this occurs, the solution is either to add system memory, or to enable file bound processing with the *-m* option. The *-t* option can also be used to save memory.

XLINK INTERNAL ERRORS

During linking, a number of internal consistency checks are performed. If any of these checks fail, the linker will terminate after giving a short description of the problem. These errors will not normally occur, but if they do please report them to the IAR technical support group. Please include all possible information about the problem and also a diskette with the program that generated the error.

ERROR MESSAGE SUMMARY

If you get a message that indicates a corrupt object file, reassemble or recompile the faulty file since an interrupted assembly or compilation may produce an invalid object file

<i>No</i>	<i>Error message</i>	<i>Suggestion</i>
0	Format chosen cannot support banking	Format unable to support banking.
1	Corrupt file. Unexpected end of file in module <i>module</i> (<i>file</i>) encountered	Linker aborts immediately. Recompile or reassemble, or check the compatibility between the linker and C compiler.
2	Too many errors encountered (>100)	Linker aborts immediately.

XLINK AND XLIB DIAGNOSTICS

<i>No</i>	<i>Error message</i>	<i>Suggestion</i>
3	Corrupt file. Checksum failed in module <i>module</i> (file). Linker checksum is linkcheck, module checksum is modcheck	Linker aborts immediately. Recompile or reassemble.
4	Corrupt file. Zero length identifier encountered in module <i>module</i> (file)	Linker aborts immediately. Recompile or reassemble.
5	Address type for CPU incorrect. Error encountered in module <i>module</i> (file)	Linker aborts immediately. Check the files and library.
6	Program module <i>module</i> declared twice, redeclaration in file <i>file</i> . Ignoring second module	XLINK will not produce code unless the -B option (forced dump) is used.
7	Corrupt file. Unexpected UBROF - format end of file encountered in module <i>module</i> (file)	Linker aborts immediately. Recompile or reassemble.
8	Corrupt file. Unknown or misplaced tag encountered in module <i>module</i> (file). Tag <i>tag</i>	Linker aborts immediately. Recompile or reassemble.
9	Corrupt file. Module <i>module</i> start unexpected in file <i>file</i>	Linker aborts immediately. Recompile or reassemble.
10	Corrupt file. Segment no. <i>segno</i> declared twice in module <i>module</i> (file)	Linker aborts immediately. Recompile or reassemble.
11	Corrupt file. External no. <i>ext no</i> declared twice in module <i>module</i> (file)	Linker aborts immediately. Recompile or reassemble.

XLINK AND XLIB DIAGNOSTICS

<i>No</i>	<i>Error message</i>	<i>Suggestion</i>
12	Unable to open file <i>file</i>	Linker aborts immediately. Check environment variable XLINK_DFLTDIR.
13	Corrupt file. Error tag encountered in module <i>module</i> (file)	A UBROF error tag was encountered. Linker aborts immediately. Recompile or reassemble.
14	Corrupt file. Local <i>local</i> defined twice in module <i>module</i> (file)	Linker aborts immediately. Recompile or reassemble.
15	Faulty bank definition -bbank def	Incorrect syntax. Linker aborts immediately.
16	Segment <i>segment</i> is too long for segment definition	The segment defined does not fit into the memory area reserved for it. Linker aborts immediately.
17	Segment <i>segment</i> is defined twice in segment definition -Zsegdef	Linker aborts immediately.
18	Range error in module <i>module</i> (file), segment <i>segment</i> at address <i>address</i> . Value <i>value</i> , in tag <i>tag</i> , is out of bounds	The address is out of the CPU address range.
19	Corrupt file. Undefined segment referenced in module <i>module</i> (file)	Linker aborts immediately. Recompile or reassemble.
20	Undefined external referenced in module <i>module</i> (file)	Linker aborts immediately. Recompile or reassemble.
21	Segment <i>segment</i> in module <i>module</i> does not fit bank	The segment is too long. Linker aborts immediately.

XLINK AND XLIB DIAGNOSTICS

<i>No</i>	<i>Error message</i>	<i>Suggestion</i>
22	Paragraph no. is not applicable for the wanted CPU. Tag encountered in module <i>module</i> (file)	Linker aborts immediately.
23	Corrupt file. T_REL_FI_8 or T_EXT_FI_8 is corrupt in module <i>module</i> (file)	The tag T_REL_FI_8 or T_EXT_FI_8 is faulty. Linker aborts immediately. Recompile or reassemble.
24	Segment <i>segment</i> overlaps segment <i>segment</i>	The segments overlap each other; ie both have code on the same address.
25	Corrupt file. Unable to find module <i>module</i> (file)	A module is missing. Linker aborts immediately.
26	Segment <i>segment</i> is too long	This error should never occur unless the program is extremely large. Linker aborts immediately.
27	Entry <i>entry</i> in module <i>module</i> (file) redefined in module <i>module</i> (file)	There are two or more entries with the same name. Linker aborts immediately.
28	File <i>file</i> is too long	This error should never occur unless the program is extremely large. Linker aborts immediately.
29	No object file specified in command-line	There is nothing to link. Linker aborts immediately.
30	Option <i>-option</i> also requires the <i>-option option</i>	Linker aborts immediately.
31	Option <i>-option</i> cannot be combined with the <i>-option option</i>	Linker aborts immediately.

XLINK AND XLIB DIAGNOSTICS

<i>No</i>	<i>Error message</i>	<i>Suggestion</i>
32	Option <i>-option</i> cannot be combined with the <i>-option</i> option and the <i>-option</i> option	Linker aborts immediately.
33	Faulty value <i>val</i> (in command line or in XLINK_PAGE), (range is 10-150)	Faulty page setting. Linker aborts immediately.
34	Filename too long	The filename is more than 255 characters long. Linker aborts immediately.
35	Unknown flag <i>flag</i> in cross reference option <i>option</i>	Linker aborts immediately.
36	Option <i>op</i> does not exist	Linker aborts immediately.
37	- not succeeded by character	The - marks the beginning of an option, and must be followed by a character. Linker aborts immediately.
38	Option <i>option</i> multiply defined	Linker aborts immediately.
39	Illegal character specified in option <i>op</i>	Linker aborts immediately.
40	Argument expected after option <i>op</i>	This option must be succeeded by an argument. Linker aborts immediately.
41	Unexpected '-' in option <i>op</i>	Linker aborts immediately.
42	Faulty symbol definition -Dsymbol definition	Incorrect syntax. Linker aborts immediately.
43	Symbol in <i>symbol</i> definition too long	The symbol name is more than 255 characters. Linker aborts immediately.

XLINK AND XLIB DIAGNOSTICS

<i>No</i>	<i>Error message</i>	<i>Suggestion</i>
44	Faulty value <i>val</i> (in command line or in XLINK_COLUMNS), (range 80-300)	Faulty column setting. Linker aborts immediately.
45	Unknown CPU <i>CPU</i> encountered in command line (or in XLINK_CPU)	Linker aborts immediately.
46	Undefined external <i>external</i> referred in module (file)	Entry to external is missing.
47	Unknown format <i>format</i> encountered in command line or XLINK_FORMAT	Linker aborts immediately.
48	Faulty segment definition -Zsegdef	Incorrect syntax. Linker aborts immediately.
49	Segment name in segment definition too long	The segment name is more than 255 characters long. Linker aborts immediately.
50	Paragraph no. not allowed for this CPU, encountered in option <i>option</i>	Linker aborts immediately.
51	Hexadecimal or decimal value expected in option <i>option</i>	Linker aborts immediately.
52	Overflow on value in option <i>option</i>	Linker aborts immediately.
53	Parameter exceeded 255 characters in extended command line file <i>file</i>	Linker aborts immediately.
54	Extended command line file <i>file</i> is empty	Linker aborts immediately.

XLINK AND XLIB DIAGNOSTICS

<i>No</i>	<i>Error message</i>	<i>Suggestion</i>
55	Extended command line variable XLINK_ENVPAR is empty	Linker aborts immediately.
56	Overlapping ranges in segment definition <i>segment def</i>	Linker aborts immediately.
57	No CPU defined	No CPU defined, either in the command line or in XLINK_CPU. Linker aborts immediately.
58	No format defined	No format defined, either in the command line or in XLINK_FORMAT. Linker aborts immediately.
59	Revision no. for file is incompatible with XLINK revision no.	Linker aborts immediately. If this error occurs after recompilation or reassembly, the wrong version of XLINK is being used. Check with your distributor.
60	Segment <i>segment</i> defined in bank definition and segment definition.	Linker aborts immediately.
61	Symbol in bank definition is too long	Linker aborts immediately.
62	File <i>file</i> multiply defined in command line	Linker aborts immediately.
63	Trying to pop an empty stack in module <i>module</i> (file)	Linker aborts immediately. Recompile or reassemble.

XLINK AND XLIB DIAGNOSTICS

<i>No</i>	<i>Error message</i>	<i>Suggestion</i>
64	module <i>module</i> (file) has not the same debug type as the other modules	Linker aborts immediately.
65	Faulty replacement definition <i>-rrreplacement</i> definition	Incorrect syntax. Linker aborts immediately.
66	Function with F-index <i>index</i> has not been defined before indirect reference in module <i>module</i> (file)	Indirect call to an undefined in module. Probably caused by an omitted function declaration.
67	Function <i>name</i> has same F-index as function- <i>name</i> , defined in module <i>module</i> (file)	Probably a corrupt file. Recompile file.
68	External function <i>name</i> in module <i>module</i> (file) has no global definition	If no other errors have been encountered, this error is generated by an assembly language call from C where the required declaration using the \$DEFFN assembly language support directive is missing. The declaration is necessary to inform the linker of the memory requirements of the function.

XLINK AND XLIB DIAGNOSTICS

<i>No</i>	<i>Error message</i>	<i>Suggestion</i>
69	Indirect or recursive function <i>name</i> in module <i>module</i> (file) has parameters or auto variables in nondefault memory	The recursively or indirectly called function <i>name</i> is using extended language memory specifiers (bit, data, idata, etc) to point to non-default memory, which is not allowed. Function parameters to indirectly called functions must be in the default memory area for the memory model in use, and for recursive functions, both local variables and parameters must be in default memory.
70	Module <i>module</i> (file) has not the same memory as previously linked modules	Only modules compiled under the same memory model may be linked together.
71	Segment <i>name</i> is incorrectly defined (in a bank definition, has wrong segment type or mixed segment types)	This is usually due to misuse of a predefined segment; see the explanation of <i>name</i> in the <i>IAR C Compiler</i> guide. It may be caused by changing the predefined linker control file.
72	Segment <i>name</i> must be defined in a -Z definition	This is either an omission of a segment in the linker (usually a segment needed by the C system control) file or a spelling error (segment names are case sensitive).
73	Label ?ARG_MOVE not found (recursive function need it)	In the library there should be a module containing this label. If it has been removed it must be restored.

XLINK AND XLIB DIAGNOSTICS

<i>No</i>	<i>Error message</i>	<i>Suggestion</i>
74	There was an error when writing to file <i>file</i>	Either the linker or your host system is corrupt, or the two are incompatible. Call your supplier.
75	SFR address in module <i>module</i> (file), segment <i>segment</i> at address <i>address</i> , value <i>value</i> is out of bounds	An SFR has been defined to a bad address. Change the definition.
76	Absolute segments overlap in module <i>module</i>	The linker has found two or more absolute segments in <i>module</i> overlapping each other.
77	Absolute segments in module <i>module</i> (file) overlaps absolute segment in module <i>module</i> (file)	The linker has found two or more absolute segments in <i>module</i> (file) and <i>module</i> (file) overlapping each other.
78	Absolute segment in module <i>module</i> (file) overlaps segment <i>segment</i>	The linker has found an absolute segment in <i>module</i> (file) overlapping a relocatable segment.
79	Faulty allocation definition <i>-adefinition</i>	The linker has discovered an error in an overlay control definition.
80	Symbol in allocation definition (-a) too long	A symbol in the -a command is too long.
81	Unknown flag in extended format option -Y	Check flags.
82	Conflict in segment 'name'. Mixing overlayable and not overlayable segment parts.	These errors only occur with the 8051 and converted PL/M code.

XLINK AND XLIB DIAGNOSTICS

<i>No</i>	<i>Error message</i>	<i>Suggestion</i>
83	The overlayable segment 'name' may not be banked.	These errors only occur with the 8051 and converted PL/M code.
84	The overlayable segment 'name' must be of relative type.	These errors only occur with the 8051 and converted PL/M code.

WARNING MESSAGE SUMMARY

The following table lists the linker warning messages:

<i>No</i>	<i>Warning message</i>	<i>Suggestion</i>
0	Too many warnings	Too many warnings encountered.
1	Error tag encountered in module <i>module</i> (file)	A UBROF error tag was encountered when loading file <i>file</i> . This indicates a corrupt file and will generate an error in the linking phase.
2	Symbol <i>symbol</i> is redefined in command-line	A symbol has been redefined.
3	Type conflict. Segment <i>segment</i> , in module <i>module</i> , is incompatible with earlier segment(s) of the same name	Segments of the same name should have the same type.
4	Close/open conflict. Segment <i>segment</i> , in module <i>module</i> , is incompatible with earlier segment of the same name	Segments of the same name should be either open or closed.
5	Segment <i>segment</i> cannot be combined with previous segment	The segments will not be combined.

XLINK AND XLIB DIAGNOSTICS

<i>No</i>	<i>Warning message</i>	<i>Suggestion</i>
6	Type conflict for external/entry <i>entry</i> , in module <i>module</i> , against external/entry in module <i>module</i>	Entries and their corresponding externals should have the same type.
7	Module <i>module</i> declared twice, once as program and once as library. Redeclared in file <i>file</i> , ignoring library module	The program module is linked.
8	Segment <i>segment</i> undefined in segment or bank definition	Undefined segment exists. All segments should be defined in either the segment or the bank definition.
9	Ignoring redeclared program entry	Only the program entry found first is chosen.
10	No modules to link	The linker has no modules to link.
11	Module <i>module</i> declared twice as library. Redeclared in file <i>file</i> , ignoring second module	The module found first is linked.
12	Using SFB in banked segment <i>segment</i> in module <i>module</i> (file)	The SFB assembler directive may not work in a banked segment.
13	Using SFE in banked segment <i>segment</i> in module <i>module</i> (file)	The SFE assembler directive may not work in a banked segment.

XLINK AND XLIB DIAGNOSTICS

<i>No</i>	<i>Warning message</i>	<i>Suggestion</i>
14	Entry <i>entry</i> duplicated. Module <i>module</i> (file) loaded, module <i>module</i> (file) discarded	Duplicated entries exist in conditionally loaded modules; ie library modules or conditionally loaded program modules (with the -C option).
15	Predefined type sizing mismatch between modules <i>module</i> (file) and <i>module</i> (file)	The modules have been compiled with different options for predefined types, such as different sizes of basic C types (eg integer, double).
16	Function <i>name</i> in module <i>module</i> (file) is called from two function trees (with roots <i>name1</i> and <i>name2</i>)	The probable cause is that an interrupt function calls another function that also could be executed by a foreground program, and this could lead to execution errors.
17	Segment <i>name</i> is too large or placed at wrong address	This error occurs if a given segment overruns the available address space in the named memory area. To find out the extent of the overrun do a dummy link, moving the start address of the named segment to the lowest address, and look at the linker map file. Then relink with the correct address specification.
18	Segment <i>segment</i> overlaps segment <i>segment</i>	The linker has found two relocatable segments overlapping each other. Check the -Z option parameters.

XLINK AND XLIB DIAGNOSTICS

<i>No</i>	<i>Warning message</i>	<i>Suggestion</i>
19	Absolute segments overlaps in module <i>module (file)</i>	The linker has found two or more absolute segments in module <i>module</i> overlapping each other.
20	Absolute segment in module <i>module (file)</i> overlaps absolute segment in module <i>module (file)</i>	The linker has found two or more absolute segments in module <i>module (file)</i> and module <i>module (file)</i> overlapping each other. Change the ORG directives.
21	Absolute segment in module <i>module (file)</i> overlaps segment <i>segment</i>	The linker has found an absolute segment in module <i>module (file)</i> overlapping a relocatable segment. Change either the ORG directive or the -Z relocation command.
22	Interrupt function <i>name</i> in module <i>module (file)</i> is called from other functions	Interrupt functions may not be called.

XLIB MESSAGES

Commands flagged as erroneous never alter object files.

<i>No</i>	<i>Error message</i>	<i>Suggestion</i>
1	Bad object file, EOF encountered	Bad or empty object file, which could be the result of an aborted assembly or compilation.
2	Unexpected EOF in batch file	The last command in a command file must be EXIT.

XLINK AND XLIB DIAGNOSTICS

<i>No</i>	<i>Error message</i>	<i>Suggestion</i>
3	Unable to open file <i>file</i>	Could not open the command file or, if ON-ERROR-EXIT has been specified, this message is issued on any failure to open a file.
4	Variable length record out of bounds	Bad object module, could be the result of an aborted assembly.
5	Missing or non-default parameter	A parameter was missing in the direct mode.
6	No such CPU	A list with the possible choices is displayed when this error is found.
7	CPU undefined	DEFINE-CPU must be issued before object file operations can begin. A list with the possible choices is displayed when this error is found.
8	Ambiguous CPU type	A list with the possible choices is displayed when this error is found.
9	No such command	Use the HELP command.
10	Ambiguous command	Use the HELP command.
11	Invalid parameter(s)	Too many parameters or a misspelled parameter.
12	Module out of sequence	Bad object module, could be the result of an aborted assembly.

XLINK AND XLIB DIAGNOSTICS

<i>No</i>	<i>Error message</i>	<i>Suggestion</i>
13	Incompatible object, consult distributor!	Bad object module, could be the result of an aborted assembly, or that the assembler/compiler revision used is incompatible with the version of XLIB used.
14	Unknown tag: hh	Bad object module, could be the result of an aborted assembly.
15	Too many errors	More than 32 errors will make XLIB abort.
16	Assembly/compilation error?	The T_ERROR tag was found. Edit and re-assemble/re-compile your program.
17	Bad CRC, hhhh expected	Bad object module; could be the result of an aborted assembly.
18	Can't find module: xxxxx	Check the available modules with LIST-MOD <i>file</i> .
19	Module expression out of range	Module expression is less than one or greater than \$.
20	Bad syntax in module expression: xxxxx	The syntax is invalid.
21	Illegal insert sequence	The specified <i>destination</i> in the INSERT-MODULES command must not be within the <i>start-end</i> sequence.
22	<End module> found before <Start module>!	Source module range must be from low to high order.
23	Before or after!	Bad BEFORE AFTER specifier in the INSERT-MODULES command.

XLINK AND XLIB DIAGNOSTICS

<i>No</i>	<i>Error message</i>	<i>Suggestion</i>
24	Corrupt file, error occurred in <i>tag</i>	A fault is detected in the object file <i>tag</i> . Reassembly or recompilation may help. Otherwise contact an IAR technical support center.
25	<i>File</i> is write protected	The file <i>file</i> is write protected and cannot be written to.
26	Non-matching replacement module <i>name</i> found in source file	In the source file, a module name with no corresponding entry in the destination file was found.

INDEX

~ (assembler operator)	1-62, 1-63	-c (command line option)	1-32
^ (assembler operator)	1-63	-c (XLINK command)	2-34
! (assembler operator)	1-70	-D (command line option)	1-33
!= (assembler operator)	1-69	-D (XLINK command)	2-35
#define (assembler directive)	1-123	-d (XLINK command)	2-36
#else (assembler directive)	1-123	-E (command line option)	1-33
#endif (assembler directive)	1-123	-E (XLINK command)	2-37
#error (assembler directive)	1-123	-e (XLINK command)	2-38
#if (assembler directive)	1-123	-F (XLINK command)	2-39
#ifdef (assembler directive)	1-123	-f (command line option)	1-34
#ifndef (assembler directive)	1-123	-f (XLINK command)	2-40
#include (assembler directive)	1-123	-G (command line option)	1-34
#message (assembler directive)	1-123	-G (XLINK command)	2-41
#undef (assembler directive)	1-123	-I (command line option)	1-35
\$ (assembler directive)	1-130	-i (command line option)	1-35
\$ (location counter)	1-45	-K (XLINK command)	2-42
% (assembler operator)	1-69	-L (command line option)	1-35
& (assembler operator)	1-62	-l (command line option)	1-36
&& (assembler operator)	1-61	-l (XLINK command)	2-43
* (assembler operator)	1-59	-M (command line option)	1-36
+	1-59	-m (XLINK command)	2-44
- (assembler operator)	1-60	-N (command line option)	1-37
-.! (XLINK command)	2-27	-n (XLINK command)	2-45
-A (XLINK command)	2-28	-O (command line option)	1-37
-a (XLINK command)	2-29	-o (command line option)	1-37
-B (command line option)	1-31	-o (XLINK command)	2-46
-B (XLINK command)	2-30	-p (command line option)	1-38
-b (command line option)	1-32	-p (XLINK command)	2-47
-b (XLINK command)	2-31	-R (XLINK command)	2-48
-C (XLINK command)	2-33	-r (command line option)	1-38
		-r (XLINK command)	2-49
		-S (command line option)	1-39
		-S (XLINK command)	2-50
		-T (command line option)	1-39

INDEX

-t (command line option)	1-39	ASEG (assembler directive)	1-85
-t (XLINK command)	2-51	ASMZ80 (environment variable)	1-10
-U (command line option)	1-40	assembler	
-uu (command line option)	1-40	command line options	
-v (command line option)	1-41	summary	1-29
-w (command line option)	1-41	expressions	1-44
-w (XLINK command)	2-52	features	1-1
-x (command line option)	1-42	installing	1-5
-x (XLINK command)	2-53	introduction	1-1
-Y (XLINK command)	2-55	labels	1-45
-Z (XLINK command)	2-56	listing format	1-13, 1-49
.ELSE (assembler directive)	1-93	mnemonics	1-135
.ENDIF (assembler directive)	1-93	operator format	1-57
.IF (assembler directive)	1-93	operators	1-44
/ (assembler operator)	1-61	output formats	1-51
/* (assembler directive)	1-130	overview	1-3
< (assembler operator)	1-68	running	1-8, 1-12
< < (assembler operator)	1-72	source format	1-43
< = (assembler operator)	1-67	symbols	1-44
< > (assembler operator)	1-69	system requirements	1-4
= (assembler directive)	1-90	assembler diagnostics	1-145
= (assembler operator)	1-64	command line errors	1-146
= = (assembler operator)	1-64	error messages	1-146, 1-150
> (assembler operator)	1-65	fatal errors	1-146
> = (assembler operator)	1-65	internal errors	1-147
> > (assembler operator)	1-73	memory overflow	1-146
(assembler operator)	1-62	warning messages	1-146, 1-147
(assembler operator)	1-70	assembler directive syntax	

A

absolute segments,		conventions	1-78
beginning	1-86, 1-87	labels	1-79
address field, in listing	1-50	parameters	1-79
AND (assembler operator)	1-61	assembler directives	1-77
ASCII character constants	1-46	#define	1-123
		#else	1-123
		#endif	1-123
		#error	1-123
		#if	1-123

INDEX

assembler directives (*continued*)

#ifdef	1-123
#ifndef	1-123
#include	1-123
#message	1-123
#undef	1-123
\$	1-130
.ELSE	1-93
.ENDIF	1-93
.IF	1-93
/*	1-130
=	1-90
ASEG	1-85
BREAK	1-104
CASE	1-104
CASEOFF	1-130
CASEON	1-130
COL	1-113
COMMON	1-85
CONTINUE	1-104
CYCLES	1-113
CYCMAX	1-113
CYCMEAN	1-113
CYCMIN	1-113
DEFA	1-127
DEFAULT	1-104
DEFB	1-127
DEFD	1-127
DEFINE	1-90
DEFL	1-90
DEFM	1-127
DEFS	1-127
DEFW	1-127
ELSE	1-104
ELSEIF	1-104
END	1-80
ENDF	1-104
ENDI	1-104

assembler directives (*continued*)

ENDIF	1-104
ENDM	1-96
ENDMOD	1-80
ENDR	1-96
ENDS	1-104
ENDW	1-104
EQU	1-90
EXITM	1-96
EXTERN	1-83
FOR	1-104
IF	1-104
LOCAL	1-96
LSTCND	1-113
LSTCOD	1-113
LSTCYC	1-113
LSTEXP	1-113
LSTMAC	1-113
LSTOUT	1-113
LSTPAG	1-113
LSTREP	1-113
LSTSAS	1-113
LSTXRF	1-113
MACRO	1-96
MODULE	1-80
NAME	1-80
OPT	1-130
ORG	1-85
PAGE	1-113
PAGSIZ	1-113
PUBLIC	1-83
RADIX	1-130
REPEAT	1-104
REPT	1-96
REPTC	1-96
REPTI	1-96
RSEG	1-85
SFR	1-130

INDEX

assembler directives (*continued*)

SFRTYPE	1-130
STACK	1-85
SWITCH	1-104
THEN	1-104
UNTIL	1-104
WEND	1-104
WHILE	1-104

assembler operators

~	1-62, 1-63
^	1-63
!	1-70
!=	1-69
%	1-69
&	1-62
&&	1-61
*	1-59
+	1-59
-	1-60
/	1-61
<	1-68
< <	1-72
< =	1-67
< >	1-69
=	1-64
= =	1-64
>	1-65
> =	1-65
> >	1-73
AND	1-61
BINAND	1-62
BINNOT	1-62
BINOR	1-62
BINXOR	1-63
BYTE3	1-63
DATE	1-64
EQ	1-64
GE	1-65

assembler operators (*continued*)

GT	1-65
HIGH	1-66
HWRD	1-66
LE	1-67
LOW	1-67
LT	1-68
LWRD	1-68
MOD	1-69
NE	1-69
NOT	1-70
OR	1-70
precedence	1-53
SFB	1-70
SFE	1-71
SHL	1-72
SHR	1-73
SIZEOF	1-73
summary	1-53
UGT	1-74
ULT	1-74
XOR	1-75
	1-62
	1-70
assembling a program	1-20
AUTOEXEC.BAT file	1-8
az80 command	1-8

B

BINAND (assembler operator)	1-62
binary numbers	1-46
BINNOT (assembler operator)	1-62
BINOR (assembler operator)	1-62

BINXOR (assembler
 operator) 1-63
 BIT (segment type) 2-58
 BREAK (assembler
 directive) 1-104
 BYTE3 (assembler
 operator) 1-63

C

C-SPY, using 1-15, 1-18
 CASE (assembler directive) 1-104
 case sensitivity, controlling 1-131
 CASEOFF (assembler
 directive) 1-130
 CASEON (assembler
 directive) 1-130
 CODE (segment type) 2-58
 COL (assembler directive) 1-113
 command line errors 1-146
 command line options
 -B 1-31
 -b 1-32
 -c 1-32
 -D 1-33
 -E 1-33
 -f 1-34
 -G 1-34
 -I 1-35
 -i 1-35
 -L 1-35
 -l 1-36
 -M 1-36
 -N 1-37
 -O 1-37
 -o 1-37
 -p 1-38

command line options (*continued*)
 -r 1-38
 -S 1-39
 -T 1-39
 -t 1-39
 -U 1-40
 -uu 1-40
 -v 1-41
 -w 1-41
 -x 1-42

command line options
 summary 1-29
 command line, extending 1-10
 COMMON (assembler
 directive) 1-85
 COMMON (segment type) 2-57
 common segments,
 beginning 1-86, 1-89
 COMPACT-FILE (XLIB
 command) 2-75
 conditional constructs 1-106
 constants 1-45
 CONTINUE (assembler
 directive) 1-104
 cycle count, in listing 1-50
 CYCLES (assembler
 directive) 1-113
 cycles, counting 1-116
 CYCMAX (assembler
 directive) 1-113
 CYCMEAN (assembler
 directive) 1-113
 CYCMIN (assembler
 directive) 1-113

INDEX

D

DATA (segment type) 2-58
data field, in listing 1-50
DATE (assembler operator) 1-64
decimal numbers 1-46
DEF-CPU (XLIB command) 1-27
DEFA (assembler directive) 1-127
DEFAULT (assembler directive) 1-104
DEFB (assembler directive) 1-127
DEFD (assembler directive) 1-127
DEFINE (assembler directive) 1-90
DEFINE-CPU (XLIB command) 2-75
defining macros 1-97
DEFL (assembler directive) 1-90
DEFM (assembler directive) 1-127
DEFS (assembler directive) 1-127
DEFW (assembler directive) 1-127
DELETE-MODULES (XLIB command) 2-76
diagnostics
 assembler 1-145
 XLIB 2-105
 XLINK 2-91
directives
 assembler 1-77
 structured assembly 1-21
DIRECTORY (XLIB command) 2-76
DISPLAY-OPTIONS (XLIB command) 2-77

E

ECHO-INPUT (XLIB command) 2-77
ELSE (assembler directive) 1-104
ELSEIF (assembler directive) 1-104
END (assembler directive) 1-80
ENDF (assembler directive) 1-104
ENDI (assembler directive) 1-104
ENDIF (assembler directive) 1-104
ENDM (assembler directive) 1-96
ENDMOD (assembler directive) 1-80
ENDR (assembler directive) 1-96
ENDS (assembler directive) 1-104
ENDW (assembler directive) 1-104
entry list, linker 2-13
environment variables
 ASMZ80 1-10
 XLIB_COLUMNS 2-69
 XLIB_CPU 2-69
 XLIB_PAGE 2-69
 XLIB_SCROLL_BREAK 2-70
 XLINK_COLUMNS 2-17
 XLINK_CPU 2-18
 XLINK_DFLTDIR 1-8, 2-18
 XLINK_ENVPAR 2-19
 XLINK_FORMAT 2-19
 XLINK_MEMORY 2-20
 XLINK_PAGE 2-20
 XLINK_TFILE 2-21
EQ (assembler operator) 1-64
EQU (assembler directive) 1-90

error messages
 assembler 1-150
 XLIB 2-105
 XLINK 2-91
 error return codes 1-9
 assembler 1-9
 XLINK 2-8
 errors, displaying 1-125
 EXIT (XLIB command) 1-27, 2-78
 EXITM (assembler directive) 1-96
 expressions, in assembler 1-44
 extensions for files 1-9
 EXTERN (assembler directive) 1-83

F

FAR (segment type) 2-58
 FARC (segment type) 2-58
 FETCH-MODULES (XLIB command) 1-27, 2-78
 file naming conventions 1-9
 linker 2-7
 floating-point numbers 1-47
 FOR (assembler directive) 1-104

G

GE (assembler operator) 1-65
 global value, defining 1-91
 GT (assembler operator) 1-65

H

header, in listing 1-50
 HELP (XLIB command) 2-79
 hexadecimal numbers 1-45
 HIGH (assembler operator) 1-66
 HUGE (segment type) 2-58
 HUGECC (segment type) 2-58
 HWRD (assembler operator) 1-66

I

IDATA (segment type) 2-58
 IDATA0 (segment type) 2-58
 IDATA1 (segment type) 2-58
 IF (assembler directive) 1-104
 in-line coding using macros 1-100
 INSERT-MODULES (XLIB command) 2-79
 installation 1-5
 instruction set, extending 1-100
 iteration construct 1-107

J

jump instructions, optimizing 1-132

L

labels
 defining and undefining 1-124
 in assembler 1-45
 in assembler directives 1-79

INDEX

- LE (assembler operator) 1-67
- librarian 2-2
 - batch mode 2-67
 - command summary 2-71
 - environment variables 2-69
 - error messages 2-105
 - features 2-2
 - interactive mode 2-67
 - introduction 2-65
 - overview 2-3
 - running 2-67
 - system requirements 2-4
 - using 1-26
- librarian commands *see* XLIB
- commands
- libraries 2-10, 2-65
 - using 1-26
- library modules, beginning 1-81
- library routines, creating 1-24
- linker 2-1
 - command summary 2-23
 - environment variables 2-17
 - error messages 2-91
 - error return codes 2-8
 - features 2-2
 - file naming conventions 2-7
 - input files and modules 2-9
 - introduction 2-5
 - libraries 2-6
 - listing format 2-12
 - object format 2-6, 2-10
 - operation 2-7
 - output formats 1-51, 2-6, 2-61
 - overview 2-3
 - system requirements 2-4
 - warning messages 2-102
- linker commands *see* XLINK
- commands
- linking 1-14, 1-18
- LIST-ALL-SYMBOLS (XLIB
 - command) 2-80
- LIST-CRC (XLIB command) 2-80
- LIST-DATE-STAMPS (XLIB
 - command) 2-81
- LIST-ENTRIES (XLIB
 - command) 2-81
- LIST-EXTERNALS (XLIB
 - command) 2-82
- LIST-MODULES (XLIB
 - command) 1-27, 2-82
- LIST-OBJECT-CODE (XLIB
 - command) 2-83
- LIST-SEGMENTS (XLIB
 - command) 2-83
- listings
 - address and data fields 1-50
 - assembler 1-49
 - conditional code and strings 1-114
 - cross reference table 1-115
 - cycle count 1-50
 - formatting 1-115
 - generated lines 1-115
 - header 1-50
 - macros 1-115
 - segment list 1-50
 - source line 1-50
 - source line number 1-50
 - symbol list 1-50
 - turning on and off 1-114
- LOCAL (assembler
 - directive) 1-96
- local symbols, using 1-92
- local value, defining 1-90
- location counter 1-45
 - setting 1-87

loop directives	1-106	MAKE-PROGRAM (XLIB	
LOW (assembler operator)	1-67	command)	2-84
LSTCND (assembler directive)	1-113	memory overflow error	1-146
LSTCOD (assembler directive)	1-113	mnemonics, assembler	1-135
LSTCYC (assembler directive)	1-113	MOD (assembler operator)	1-69
LSTEXP (assembler directive)	1-113	MODULE (assembler directive)	1-80
LSTMAC (assembler directive)	1-113	module map, linker	2-14
LSTOUT (assembler directive)	1-113	modules	
LSTPAG (assembler directive)	1-113	terminating	1-81
LSTREP (assembler directive)	1-113	using	1-23
LSTSAS (assembler directive)	1-113	MS-DOS error return codes	1-9
LSTXRF (assembler directive)	1-113		
LT (assembler operator)	1-68	N	
LWRD (assembler operator)	1-68	NAME (assembler directive)	1-80
		NE (assembler operator)	1-69
		NOT (assembler operator)	1-70
		numbers	
		binary	1-46
		decimal	1-46
		hexadecimal	1-45
		octal	1-46
		real	1-47
M		O	
MACRO (assembler directive)	1-96	octal numbers	1-46
macro-generated lines	1-18	ON-ERROR-EXIT (XLIB	
macros		command)	2-85
defining	1-97	operators	
processing	1-99	in assembler	1-44
tutorial	1-15	__TID__	1-44, 1-48
using special characters	1-98	OPT (assembler directive)	1-130
MAKE-LIBRARY (XLIB		OR (assembler operator)	1-70
command)	2-84	ORG (assembler directive)	1-85

INDEX

output formats, assembler 1-51

P

PAGE (assembler directive) 1-113

PAGSIZ (assembler directive) 1-113

path for installed files 1-6

program modules, beginning 1-80

PUBLIC (assembler directive) 1-83

Q

QUIT (XLIB command) 2-85

R

RADIX (assembler directive) 1-130

real number constants 1-47

RELATIVE (segment type) 2-57

relocatable segments, beginning 1-86, 1-88

REMARK (XLIB command) 2-86

RENAME-ENTRY (XLIB command) 2-86

RENAME-EXTERNAL (XLIB command) 2-87

RENAME-GLOBAL (XLIB command) 2-87

RENAME-MODULE (XLIB command) 2-88

RENAME-SEGMENT (XLIB command) 2-88

REPEAT (assembler directive) 1-104

repeating statements 1-99

REPLACE-MODULES (XLIB command) 2-89

REPT (assembler directive) 1-96

REPTC (assembler directive) 1-96

REPTI (assembler directive) 1-96

requirements
assembler 1-4

linker and librarian 2-4

return codes 1-9

RSEG (assembler directive) 1-85

running a program 1-15, 1-18

running the assembler 1-8, 1-12

S

segment list in listing 1-50

segment location 2-11

segment map, linker 2-14

segment types

BIT 2-58

CODE 2-58

COMMON 2-57

DATA 2-58

FAR 2-58

FARC 2-58

HUGE 2-58

HUGEC 2-58

IDATA 2-58

IDATA0 2-58

IDATA1 2-58

RELATIVE 2-57

STACK 2-57

UNTYPED 2-58

XDATA 2-58

segments, defining 2-56
 SFB (assembler operator) 1-70
 SFE (assembler operator) 1-71
 SFR (assembler directive) 1-130
 SFRTYPE (assembler directive) 1-130
 SHL (assembler operator) 1-72
 SHR (assembler operator) 1-73
 SIZEOF (assembler operator) 1-73
 source files,
 including 1-125, 1-132
 source format, assembler 1-43
 source line in listing 1-50
 source line number in listing 1-50
 special function registers,
 defining 1-132
 STACK (assembler directive) 1-85
 STACK (segment type) 2-57
 stack segments,
 beginning 1-86, 1-88
 structured assembly 1-19, 1-21
 SWITCH (assembler directive) 1-104
 symbol list, in listing 1-50
 symbols
 exporting to other modules 1-83
 importing from other modules 1-83
 redefining 1-91
 symbols, in assembler 1-44

T

target identity symbol 1-44, 1-48
 temporary value, defining 1-90
 THEN (assembler directive) 1-104
 __TID__ (operator) 1-44, 1-48
 tutorial 1-11
 tutorial programs
 DayofWeek 1-16
 first 1-11
 GCD 1-19
 using modules 1-23

U

UGT (assembler operator) 1-74
 ULT (assembler operator) 1-74
 UNTIL (assembler directive) 1-104
 UNTYPED (segment type) 2-58

W

warning messages, XLINK 2-102
 WEND (assembler directive) 1-104
 WHILE (assembler directive) 1-104

INDEX

X

XDATA (segment type)	2-58	XLIB Librarian (<i>continued</i>)	
XLIB commands	2-73	environment variables	2-69
COMPACT-FILE	2-75	error messages	2-105
DEF-CPU	1-27	features	2-2
DEFINE-CPU	2-75	interactive mode	2-67
DELETE-MODULES	2-76	introduction	2-65
DIRECTORY	2-76	overview	2-3
DISPLAY-OPTIONS	2-77	running	2-67
ECHO-INPUT	2-77	system requirements	2-4
EXIT	1-27, 2-78	using	1-26
FETCH-MODULES	1-27, 2-78	XLIB_COLUMNS (XLIB	
HELP	2-79	environment variable)	2-69
INSERT-MODULES	2-79	XLIB_CPU (XLIB	
LIST-ALL-SYMBOLS	2-80	environment variable)	2-69
LIST-CRC	2-80	XLIB_PAGE (XLIB	
LIST-DATE-STAMPS	2-81	environment variable)	2-69
LIST-ENTRIES	2-81	XLIB_SCROLL_BREAK (XLIB	
LIST-EXTERNALS	2-82	environment variable)	2-70
LIST-MODULES	1-27, 2-82	XLINK command format	2-25
LIST-OBJECT-CODE	2-83	XLINK commands	2-25
LIST-SEGMENTS	2-83	-!	2-27
MAKE-LIBRARY	2-84	-A	2-28
MAKE-PROGRAM	2-84	-a	2-29
ON-ERROR-EXIT	2-85	-B	2-30
QUIT	2-85	-b	2-31
REMARK	2-86	-C	2-33
RENAME-ENTRY	2-86	-c	2-34
RENAME-EXTERNAL	2-87	-D	2-35
RENAME-GLOBAL	2-87	-d	2-36
RENAME-MODULE	2-88	-E	2-37
RENAME-SEGMENT	2-88	-e	2-38
REPLACE-MODULES	2-89	-F	2-39
XLIB Librarian	2-2	-f	2-40
batch mode	2-67	-G	2-41
command summary	2-71	-K	2-42
		-l	2-43
		-m	2-44
		-n	2-45

XLINK commands (<i>continued</i>)		XLINK Linker (<i>continued</i>)	
-o	2-46	object format	2-6, 2-10
-p	2-47	operation	2-7
-R	2-48	output formats	1-51, 2-6, 2-61
-r	2-49	overview	2-3
-S	2-50	system requirements	2-4
-t	2-51	warning messages	2-102
-w	2-52	XLINK_COLUMNS (XLINK	
-x	2-53	environment variable)	2-17
-Y	2-55	XLINK_CPU (XLINK	
-Z	2-56	environment variable)	2-18
-z	2-60	XLINK_DFLTDIR (XLINK	
XLINK Linker	2-1	environment variable)	1-8, 2-18
command summary	2-23	XLINK_ENVPAR (XLINK	
environment variables	2-17	environment variable)	2-19
error messages	2-91	XLINK_FORMAT (XLINK	
error return codes	2-8	environment variable)	2-19
features	2-2	XLINK_MEMORY (XLINK	
file naming conventions	2-7	environment variable)	2-20
functions	2-6	XLINK_PAGE (XLINK	
input files and modules	2-9	environment variable)	2-20
introduction	2-5	XLINK_TFILE (XLINK	
libraries	2-6, 2-10	environment variable)	2-21
listing format	2-12	XOR (assembler operator)	
			1-75

INDEX
